



**UNIVERSIDAD  
DE GRANADA**

TRABAJO FIN DE GRADO  
INGENIERÍA INFORMÁTICA

# Integración de LLMs con Wazuh para enriquecer las alertas y apoyar a la toma de decisiones

---

**Autor:**

Máximo Martín Moreno

**Directores:**

Antonio Miguel Mora García

Jesús Chamorro Martínez



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE  
TELECOMUNICACIÓN

Granada, septiembre de 2026







# Integración de LLMs con Wazuh para enriquecer las alertas y apoyar a la toma de decisiones

---

## **Autor**

Máximo Martín Moreno

## **Directores**

Antonio Miguel Mora García

Jesús Chamorro Martínez



DEPARTAMENTO DE TEORÍA DE LA SEÑAL, TELEMÁTICA Y  
COMUNICACIONES



**DECSAI**

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA  
ARTIFICIAL

Granada, septiembre de 2026

# Integración de LLMs con Wazuh para enriquecer las alertas y apoyar a la toma de decisiones

Máximo Martín Moreno

**Palabras clave:** Wazuh, LLM (*Large Language Models*), SIEM (*Security Information and Event Management*), XDR (*Extended Detection and Response*), MITRE ATT&CK, Cyber Kill Chain, Ollama, Gemini, Groq, ciberseguridad, SOC (*Security Operations Center*), *threat hunting*, respuesta activa, multi-modelo, interfaz web, Kubernetes

## Resumen

La gestión de alertas de seguridad en un Centro de Operaciones de Seguridad (SOC) se enfrenta a un volumen creciente de eventos que los analistas de seguridad deben interpretar y priorizar bajo estrictos Acuerdos de Nivel de Servicio. Este Trabajo Fin de Grado abarca el diseño y la implementación de un *middleware* que conecta Wazuh, una plataforma SIEM/XDR de código abierto, con modelos de lenguaje de gran tamaño (LLM), con el fin de enriquecer las alertas y asistir en la toma de decisiones.

El sistema opera en tres modos: el **análisis forense**, que transforma las alertas en formato JSON en informes de incidente comprensibles, las correlaciona con el marco MITRE ATT&CK y la *Cyber Kill Chain*, infiere los objetivos del ataque y propone un plan de respuesta; el *threat hunting*, que traduce consultas en lenguaje natural a consultas estructuradas sobre el almacén de eventos; y la **respuesta activa**, que permite bloquear direcciones IP maliciosas siempre bajo confirmación humana explícita (*Human-in-the-Loop*). Una interfaz web desarrollada con Streamlit expone los tres modos de operación sin necesidad de línea de comandos y permite seleccionar el proveedor de inferencia (Ollama con Llama 3.2 en local, Gemini Flash y Groq) en tiempo de ejecución.

La validación experimental, con 45 ejecuciones sobre cinco vectores de ataque reales y tres modelos distintos, mostró que Gemini Flash alcanzó el 87 % de acierto en la identificación de la fase de la *Kill Chain* y cero alucinaciones en sus 15 ejecuciones, frente al 67 % y 12 alucinaciones del modelo local Llama 3.2 (3B de parámetros). El ciclo de respuesta activa completó el bloqueo de IP de extremo a extremo con traza verificable en el cortafuegos del agente. La ejecución local del modelo garantiza la privacidad de los datos de la infraestructura cuando la situación lo requiere. El sistema incluye además un despliegue para un entorno productivo sobre un clúster Kubernetes local, con manifiestos YAML versionados en git, gestión de secretos y control de tráfico.

# Integrating LLMs with Wazuh to enrich alerts and support decision-making

Máximo Martín Moreno

**Keywords:** Wazuh, LLM (Large Language Models), SIEM (Security Information and Event Management), XDR (Extended Detection and Response), MITRE ATT&CK, Cyber Kill Chain, Ollama, Gemini, Groq, cybersecurity, SOC (Security Operations Center), threat hunting, active response, multi-model, web interface, Kubernetes

## Abstract

Security alert management in a Security Operations Center (SOC) faces a growing volume of events that security analysts must interpret and prioritise under strict Service Level Agreements. This Bachelor's Thesis encompasses the design and implementation of a middleware that connects Wazuh, an open-source SIEM/XDR platform, with large language models (LLMs), in order to enrich alerts and support decision-making.

The system operates in three modes: **forensic analysis**, which transforms JSON-formatted alerts into readable incident reports, correlates them with the MITRE ATT&CK framework and the Cyber Kill Chain, infers the attacker's objectives, and proposes a response plan; *threat hunting*, which translates natural-language queries into structured queries over the event store; and **active response**, which blocks malicious IP addresses, always under explicit human confirmation (Human-in-the-Loop). A web interface built with Streamlit exposes the three operating modes without requiring the command line and allows the inference provider (Ollama with Llama 3.2 locally, Gemini Flash, or Groq) to be selected at runtime.

Experimental validation, comprising 45 executions across five real attack vectors and three distinct models, showed that Gemini Flash achieved 87 % accuracy in Kill Chain phase identification with zero hallucinations across its 15 executions, compared with 67 % and 12 hallucinations for the local Llama 3.2 model (3 B parameters). The active-response cycle completed end-to-end IP blocking with a verifiable trace in the agent firewall. Running the model locally guarantees the privacy of infrastructure data when required. The system also includes a production deployment on a local Kubernetes cluster, with YAML manifests versioned in git, secret management via Kustomize, and traffic control through NetworkPolicy and Ingress.



---

Yo, **Máximo Martín Moreno**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77148636N, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Máximo Martín Moreno

Granada, a 4 de septiembre de 2026.



---

D. **Antonio M. Mora García**, Profesor del Área de Ingeniería Telemática del Departamento de Teoría de la Señal, Telemática y Comunicaciones de la Universidad de Granada.

D. **Jesús Chamorro Martínez**, Profesor del Área de Ciencias de la Computación e Inteligencia Artificial del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

**Informan:**

Que el presente trabajo, titulado ***Integración de LLMs con Wazuh para enriquecer las alertas y apoyar a la toma de decisiones***, ha sido realizado bajo su supervisión por **Máximo Martín Moreno**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada, a 4 de septiembre de 2026.

**Los directores:**

**Antonio M. Mora García**

**Jesús Chamorro Martínez**



# Agradecimientos

En primer lugar, gracias a mis padres y mis hermanas por ser un apoyo a diario y confiar en mí siempre. El pequeño pollito de la casa está a un paso de ser Ingeniero Informático y eso es sobre todo gracias a vosotros. A mi padre, gracias por mostrarme esta profesión y enseñarme cómo es un informático tan profesional y con tanto conocimiento como él. A mi madre, por ser una persona excepcional, estar siempre tan atenta y motivarme a acabar esta última etapa de la carrera. A mis hermanas, por ser mis mayores referentes en la vida. También agradecer en especial a Lore, te doy las gracias por ser mi compañera de vida estos años y por todos los que nos quedan juntos.

A mi familia, por el orgullo que me da tener una familia como la que tenemos y todas las reuniones tan divertidas que hacemos. A mi prima Blanca, por compartir todas las etapas de nuestra vida desde que nacimos. A mis abuelas Loli, Encarnita y Antonia, por tratarme siempre de esa manera tan especial y mostrarme la importancia de mantener a la familia siempre unida.

A mis amigos que he hecho durante estos cuatro años y previos, por hacerme sentir la persona con más suerte del mundo de teneros. Mención especial a mi grupo del Erasmus en Brno, porque hemos vivido y aprendido muchísimo juntos. Me he llevado gente magnífica para el resto de mi vida. Gracias por hacerme sentir como en familia estando tan lejos de casa.

A mis compañeros de Telefónica, por abrirme un mundo nuevo y enseñarme cómo se trabaja en el ámbito de ciberseguridad en un equipo tan cercano, respetuoso y especial.

Finalmente, a mis tutores, Antonio y Jesús, muchas gracias por ayudarme siempre que lo he necesitado, por todo lo aprendido gracias a vosotros, abrirme las puertas para realizar un proyecto de ciberseguridad y por vuestra increíble paciencia.



# Índice general

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                               | <b>1</b>  |
| 1.1. Contexto y motivación . . . . .                                                 | 1         |
| 1.2. Objetivos . . . . .                                                             | 3         |
| 1.3. Contribuciones del trabajo . . . . .                                            | 3         |
| 1.4. Estructura de la memoria . . . . .                                              | 4         |
| <b>2. Conceptos preliminares y estado del arte</b>                                   | <b>7</b>  |
| 2.1. Centros de operaciones de seguridad: SIEM y XDR . . . . .                       | 7         |
| 2.2. Wazuh . . . . .                                                                 | 8         |
| 2.3. Modelos de lenguaje de gran tamaño . . . . .                                    | 9         |
| 2.4. MITRE ATT&CK y la Cyber Kill Chain . . . . .                                    | 9         |
| 2.5. Trabajos relacionados . . . . .                                                 | 11        |
| 2.5.1. Revisiones del estado del arte . . . . .                                      | 11        |
| 2.5.2. Propuestas de sistema . . . . .                                               | 12        |
| 2.5.3. Comparativa y posicionamiento . . . . .                                       | 13        |
| <b>3. Análisis y planificación</b>                                                   | <b>15</b> |
| 3.1. Requisitos del sistema . . . . .                                                | 15        |
| 3.1.1. Requisitos funcionales . . . . .                                              | 15        |
| 3.1.2. Requisitos no funcionales . . . . .                                           | 16        |
| 3.2. Casos de uso . . . . .                                                          | 16        |
| 3.3. Metodología de desarrollo . . . . .                                             | 20        |
| 3.4. Selección de tecnologías . . . . .                                              | 20        |
| 3.4.1. Plataforma SIEM y XDR: Wazuh frente a Splunk y Elastic SIEM . . . . .         | 20        |
| 3.4.2. Motor de inferencia local: Ollama frente a llama.cpp y Transformers . . . . . | 21        |
| 3.4.3. Lenguaje de implementación: Python frente a Go y Java . . . . .               | 22        |
| 3.4.4. Marco de la interfaz web: Streamlit frente a Flask, FastAPI y Dash . . . . .  | 22        |
| 3.4.5. Orquestación del despliegue: Kubernetes con kind y podman . . . . .           | 23        |
| 3.5. Planificación temporal . . . . .                                                | 23        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 3.6. Presupuesto . . . . .                                                 | 25        |
| <b>4. Diseño e implementación</b>                                          | <b>27</b> |
| 4.1. Arquitectura general . . . . .                                        | 27        |
| 4.2. Decisiones de diseño . . . . .                                        | 28        |
| 4.3. Módulo orquestador . . . . .                                          | 30        |
| 4.4. Módulo de <i>threat hunting</i> . . . . .                             | 32        |
| 4.5. Módulo de respuesta activa . . . . .                                  | 32        |
| 4.6. Ingeniería del <i>prompt</i> . . . . .                                | 34        |
| 4.7. Interfaz web . . . . .                                                | 35        |
| 4.7.1. Arquitectura de la aplicación . . . . .                             | 36        |
| 4.7.2. Páginas de la aplicación . . . . .                                  | 37        |
| 4.7.3. Respuesta activa en la interfaz web . . . . .                       | 41        |
| 4.8. Pruebas unitarias . . . . .                                           | 42        |
| 4.9. Guía de instalación y uso . . . . .                                   | 45        |
| 4.9.1. Requisitos del sistema . . . . .                                    | 45        |
| 4.9.2. Despliegue de la infraestructura de pruebas . . . . .               | 45        |
| 4.9.3. Instalación del <i>middleware</i> . . . . .                         | 47        |
| 4.9.4. Configuración . . . . .                                             | 47        |
| 4.9.5. Uso de la interfaz web . . . . .                                    | 48        |
| 4.9.6. Uso por línea de comandos . . . . .                                 | 48        |
| 4.10. Despliegue en producción: arquitectura Kubernetes . . . . .          | 49        |
| 4.10.1. Motivación y alcance . . . . .                                     | 49        |
| 4.10.2. Herramientas: kind sobre podman . . . . .                          | 50        |
| 4.10.3. Arquitectura del despliegue . . . . .                              | 51        |
| 4.10.4. Contenerización de la aplicación . . . . .                         | 51        |
| 4.10.5. Gestión de configuración y secretos . . . . .                      | 53        |
| 4.10.6. Configuración del <i>Deployment</i> . . . . .                      | 53        |
| 4.10.7. Integración de servicios externos mediante EndpointSlice . . . . . | 54        |
| 4.10.8. Control de tráfico: Ingress y NetworkPolicy . . . . .              | 55        |
| 4.10.9. Triage por lotes mediante CronJob . . . . .                        | 56        |
| 4.10.10. Verificación del despliegue . . . . .                             | 57        |
| 4.10.11. Limitaciones conocidas del despliegue . . . . .                   | 59        |
| <b>5. Experimentos y resultados</b>                                        | <b>65</b> |
| 5.1. Entorno de pruebas . . . . .                                          | 65        |
| 5.2. Justificación y secuencia del diseño experimental . . . . .           | 66        |
| 5.3. Prueba 1: análisis forense de alertas . . . . .                       | 69        |
| 5.3.1. Escenario: T1110.001, fuerza bruta SSH . . . . .                    | 69        |
| 5.3.2. Alertas generadas . . . . .                                         | 69        |
| 5.3.3. Informe generado por el <i>middleware</i> . . . . .                 | 70        |
| 5.3.4. Evaluación de la calidad . . . . .                                  | 70        |
| 5.4. Prueba 2: <i>threat hunting</i> en lenguaje natural . . . . .         | 71        |
| 5.4.1. Evaluación de la calidad . . . . .                                  | 74        |

|                                                                        |            |
|------------------------------------------------------------------------|------------|
| 5.5. Prueba 3: respuesta activa con confirmación humana . . . . .      | 75         |
| 5.5.1. Configuración previa . . . . .                                  | 75         |
| 5.5.2. Ejecución . . . . .                                             | 75         |
| 5.5.3. Verificación del bloqueo . . . . .                              | 76         |
| 5.5.4. Evaluación de la calidad . . . . .                              | 77         |
| 5.6. Prueba 4: comparativa multi-modelo . . . . .                      | 77         |
| 5.6.1. Configuración del experimento . . . . .                         | 77         |
| 5.6.2. Ataque 1: T1110.001, fuerza bruta SSH . . . . .                 | 78         |
| 5.6.3. Ataque 2: T1548.003, escalada de privilegios con sudo . . . . . | 79         |
| 5.6.4. Ataque 3: T1046, escaneo de puertos . . . . .                   | 80         |
| 5.6.5. Ataque 4: T1136.001, creación de cuenta no autorizada . . . . . | 82         |
| 5.6.6. Ataque 5: T1078, acceso con cuentas válidas . . . . .           | 83         |
| 5.6.7. Resumen comparativo . . . . .                                   | 84         |
| 5.6.8. Patrones por vector de ataque . . . . .                         | 88         |
| 5.7. Limitaciones detectadas y soluciones implementadas . . . . .      | 89         |
| 5.8. Cobertura de requisitos . . . . .                                 | 91         |
| <b>6. Conclusiones y trabajo futuro . . . . .</b>                      | <b>93</b>  |
| 6.1. Objetivos cumplidos . . . . .                                     | 93         |
| 6.2. Limitaciones del sistema . . . . .                                | 94         |
| 6.3. Consideraciones éticas y de sostenibilidad . . . . .              | 95         |
| 6.4. Trabajo futuro . . . . .                                          | 96         |
| 6.5. Valoración personal y aprendizajes adquiridos . . . . .           | 99         |
| <b>Sobre el uso de la inteligencia artificial generativa . . . . .</b> | <b>101</b> |
| <b>Bibliografía . . . . .</b>                                          | <b>103</b> |
| <b>A. Prompts utilizados . . . . .</b>                                 | <b>105</b> |



# Índice de figuras

|                                                                                                                                                                                                                                                                                                                                                                                                                                             |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1. Estructura del repositorio del proyecto. . . . .                                                                                                                                                                                                                                                                                                                                                                                       | 6  |
| 2.1. Las siete fases de la <i>Cyber Kill Chain</i> de Lockheed Martin [5]: desde el reconocimiento inicial hasta las acciones sobre los objetivos finales del atacante. Fuente: INCIBE [6]. . . . .                                                                                                                                                                                                                                         | 10 |
| 2.2. Extracto de la matriz MITRE ATT&CK [17]: las columnas representan tácticas (objetivos del adversario) y las filas agrupan las técnicas concretas empleadas para lograrlos. Disponible en <a href="https://attack.mitre.org/">https://attack.mitre.org/</a> . . . . .                                                                                                                                                                   | 11 |
| 3.1. Planificación temporal del proyecto (febrero – septiembre de 2026). . . . .                                                                                                                                                                                                                                                                                                                                                            | 24 |
| 4.1. Página de inicio de la interfaz web. Los indicadores de estado reflejan la conectividad con el Manager (puerto 55000) y el Indexer (puerto 9200); el panel inferior presenta el recuento de alertas por nivel de severidad y el feed de los eventos más recientes con su marca de tiempo y agente. . . . .                                                                                                                             | 36 |
| 4.2. Explorador de alertas de la interfaz web. La tabla superior muestra las alertas recuperadas del Indexer con sus campos MITRE ATT&CK (técnica y táctica) y la IP de origen; los controles permiten ajustar el número de alertas y el nivel mínimo de severidad. El panel inferior presenta el detalle estructurado de la alerta seleccionada: metadatos, registro en bruto ( <i>raw log</i> ) y opción de ver el JSON completo. . . . . | 38 |
| 4.3. Página de análisis LLM: sección de configuración. Los tres botones permiten seleccionar el proveedor activo (Ollama local, Gemini o Groq); el proveedor seleccionado se resalta con borde y fondo diferenciados. Los controles inferiores permiten elegir el modo de selección de alertas y el nivel mínimo de severidad. . . . .                                                                                                      | 39 |

|       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.4.  | Página de análisis LLM: resultados. Tras ejecutar el análisis, la página muestra las métricas de inferencia (alertas procesadas, tiempo total y media por alerta) y los informes generados en desplegables que incluyen el tiempo de inferencia, el modelo empleado y el texto estructurado con correlación MITRE ATT&CK. . . . .                                                                                                                                                                  | 40 |
| 4.5.  | Página de <i>Threat Hunting</i> de la interfaz web. Tras introducir una consulta en lenguaje natural, el sistema muestra la Query DSL generada en un desplegable y presenta los resultados en una tabla con los campos de agente, nivel, descripción e IP de origen. . . . .                                                                                                                                                                                                                       | 41 |
| 4.6.  | Flujo completo de la respuesta activa en la interfaz web. El aviso superior identifica <b>192.168.64.1</b> como IP candidata; el analista ha introducido la cadena exacta <b>CONFIRMAR</b> y pulsado el botón de bloqueo; el banner verde inferior confirma que la regla <b>DROP</b> fue aplicada sobre el agente <b>Ubuntu-Victima</b> a través de la API de Wazuh. . . . .                                                                                                                       | 42 |
| 4.7.  | Arquitectura del despliegue Kubernetes. El <i>dashboard</i> corre dentro del clúster; Wazuh y Ollama permanecen en el <i>host</i> macOS y se alcanzan a través de <i>EndpointSlice</i> manuales. . .                                                                                                                                                                                                                                                                                               | 51 |
| 4.8.  | Página de inicio del <i>dashboard</i> Streamlit desplegado en Kubernetes, con las dos conexiones a Wazuh verificadas y 50 alertas cargadas. . . . .                                                                                                                                                                                                                                                                                                                                                | 58 |
| 4.9.  | Diagrama de componentes del sistema. La zona de <i>endpoints</i> genera eventos que el agente Wazuh reenvía al Manager; el <i>middleware</i> consulta las alertas al Indexer (:9200) y obtiene el token de autenticación del Manager (:55000); Ollama (:11434) recibe el <i>prompt</i> y devuelve el informe. Las flechas de <b>respuesta_activa.py</b> y <b>threat_hunting.py</b> al Manager y al Indexer, respectivamente, se activan solo cuando se invocan los modos correspondientes. . . . . | 60 |
| 4.10. | Diagrama de secuencia del flujo completo del <i>middleware</i> . El bloque <b>loop</b> central muestra el análisis forense de cada alerta; los bloques <b>opt</b> inferiores representan los modos <b>--hunting</b> (consultas en lenguaje natural al Indexer vía Query DSL) y <b>--respuesta-activa</b> (confirmación humana y ejecución de <b>firewall-drop</b> ). . . . .                                                                                                                       | 61 |
| 4.11. | Flujo del módulo de <i>threat hunting</i> . La consulta en lenguaje natural se traduce a Query DSL mediante el LLM; si el JSON devuelto está truncado, <b>reparar_json_truncado()</b> añade los cierres necesarios y se reintenta la interpretación. Solo cuando el JSON es válido se lanza <b>ejecutar_query()</b> contra el Indexer. . .                                                                                                                                                         | 62 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.12. Flujo del módulo de respuesta activa con patrón <i>Human-in-the-Loop</i> . La validación actúa en dos capas independientes: <code>validar_respuesta_ia()</code> inspecciona el informe y añade un aviso si el modelo sugirió bloquear una dirección protegida; <code>bloquear_ip_en_agente()</code> comprueba de nuevo la lista blanca antes de llamar a la API. Solo la cadena exacta <b>CONFIRMAR</b> desencadena la orden de bloqueo. . . . .                                                                  | 63 |
| 5.1. Secuencia del diseño experimental. Las Pruebas 1 a 3 reproducen el ciclo de vida de un incidente tal y como lo recorre un analista de Nivel 1: interpretar la alerta, buscar actividad relacionada en el histórico y contener el ataque. La Prueba 4 no continúa ese ciclo, sino que evalúa de forma transversal la viabilidad del sistema comparando los tres proveedores de inferencia sobre los cinco vectores de ataque. . . . .                                                                               | 67 |
| 5.2. Consulta 1 sobre <code>rule.level</code> : diez eventos devueltos, con las alertas de fuerza bruta SSH (niveles 10–12) y eventos de sistema de nivel 7. La tabla muestra <i>timestamp</i> , agente, nivel con código de color, descripción, IP de origen e identificador de regla. . . . .                                                                                                                                                                                                                         | 72 |
| 5.3. Consulta 2 sobre <code>rule.mitre.id</code> con el código padre T1548: cero resultados. El modelo identificó el campo correcto pero buscó el código padre; el índice almacena exclusivamente sub-técnicas completas como T1548.003. . . . .                                                                                                                                                                                                                                                                        | 73 |
| 5.4. Consulta 4 sobre <code>rule.mitre.id</code> con el código completo T1548.003: diez eventos de la técnica <i>Abuse Elevation Control Mechanism: Sudo and Sudo Caching</i> , correspondientes a escaladas exitosas a <code>root</code> registradas durante las sesiones de pruebas. . . . .                                                                                                                                                                                                                          | 73 |
| 5.5. Tiempo de inferencia medio por ataque y proveedor (escala logarítmica). Cada barra es el promedio de las tres repeticiones: A1 = T1110.001 (fuerza bruta SSH), A2 = T1548.003 (escalada sudo), A3 = T1046 (escaneo de puertos), A4 = T1136.001 (creación de cuenta), A5 = T1078 (acceso con credenciales válidas). Groq (media 2,6 s) es aproximadamente 6 veces más rápido que Gemini (15,0 s) y 15 veces más que Ollama (39,5 s). . . . .                                                                        | 85 |
| 5.6. Porcentaje de identificación MITRE y acierto en la <i>Kill Chain</i> por proveedor, sobre 15 ejecuciones por modelo (5 ataques × 3 repeticiones). El porcentaje MITRE asigna peso 1,0 a los resultados <i>sí</i> y 0,5 a los <i>parcial</i> . Los tres modelos se sitúan entre el 30 % y el 37 % en identificación MITRE, con diferencias menores entre proveedores de lo que sugeriría la diferencia de tamaño de modelo; en la <i>Kill Chain</i> , Gemini (87 %) aventaja a Groq (80 %) y Ollama (67 %). . . . . | 86 |



# Índice de tablas

|                                                                                                                                                                                                                                                                                                                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Comparativa de los trabajos relacionados. "Local predeterminado" indica si el sistema procesa las alertas sin enviar datos a terceros por defecto; "Datos etiquetados" indica si requiere un conjunto de alertas anotadas para funcionar; Respuesta activa indica si puede ejecutar acciones de contención; "SIEM concreto" indica si la integración es nativa con una plataforma SIEM específica. . . . . | 13 |
| 3.1. Caso de uso CU-01: análisis de alertas de seguridad. . . . .                                                                                                                                                                                                                                                                                                                                               | 17 |
| 3.2. Caso de uso CU-02: consulta de <i>threat hunting</i> en lenguaje natural. . . . .                                                                                                                                                                                                                                                                                                                          | 18 |
| 3.3. Caso de uso CU-03: respuesta activa con confirmación humana. . . . .                                                                                                                                                                                                                                                                                                                                       | 18 |
| 3.4. Caso de uso CU-04: exploración de alertas en la interfaz web. . . . .                                                                                                                                                                                                                                                                                                                                      | 19 |
| 3.5. Caso de uso CU-05: selección del proveedor de inferencia. . . . .                                                                                                                                                                                                                                                                                                                                          | 19 |
| 3.6. Coste de personal. La tarifa del desarrollador proviene del [1]. La tarifa de los directores es una estimación basada en la retribución bruta de personal docente e investigador de universidad pública española, incluyendo la parte proporcional de cotizaciones sociales. . . . .                                                                                                                       | 25 |
| 3.7. Coste de hardware (amortización). . . . .                                                                                                                                                                                                                                                                                                                                                                  | 25 |
| 3.8. Resumen del presupuesto del proyecto. . . . .                                                                                                                                                                                                                                                                                                                                                              | 26 |
| 4.1. Comparativa de los tres proveedores de inferencia soportados. La latencia recoge el promedio de las 15 ejecuciones de la Prueba 4 (Sección 5.6); el coste es el precio de lista por inferencia (el incurrido fue nulo al operar dentro del nivel gratuito de cada API). . . . .                                                                                                                            | 28 |
| 4.2. Casos de prueba de <code>TestRepararJsonTruncado</code> . . . . .                                                                                                                                                                                                                                                                                                                                          | 43 |
| 4.3. Casos de prueba de <code>TestGuardrailSeguridad</code> . . . . .                                                                                                                                                                                                                                                                                                                                           | 44 |
| 4.4. Casos de prueba de <code>TestObtenerAlertasDelIndexer</code> . . . . .                                                                                                                                                                                                                                                                                                                                     | 44 |
| 4.5. Variables de configuración del <i>middleware</i> . . . . .                                                                                                                                                                                                                                                                                                                                                 | 47 |
| 4.6. Recursos Kubernetes desplegados en el espacio de nombres <code>wazuh-llm</code> . . . . .                                                                                                                                                                                                                                                                                                                  | 58 |

|                                                                                                                                                                                                                                                                                                                                       |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1. Resultados para T1110.001 (fuerza bruta SSH), 9 ejecuciones.                                                                                                                                                                                                                                                                     | 79 |
| 5.2. Resultados para T1548.003 (escalada de privilegios con sudo),<br>9 ejecuciones. . . . .                                                                                                                                                                                                                                          | 80 |
| 5.3. Resultados para T1046 (escaneo de puertos), 9 ejecuciones. . .                                                                                                                                                                                                                                                                   | 81 |
| 5.4. Resultados para T1136.001 (creación de cuenta no autorizada),<br>9 ejecuciones. . . . .                                                                                                                                                                                                                                          | 82 |
| 5.5. Resultados para T1078 (acceso con cuentas válidas), 9 ejecu-<br>ciones. . . . .                                                                                                                                                                                                                                                  | 84 |
| 5.6. Promedios globales de la Prueba 4 sobre 15 ejecuciones por<br>modelo (5 ataques $\times$ 3 repeticiones). Guard.: número total de<br>activaciones de la salvaguarda. Los costes de Gemini y Groq<br>corresponden al precio de lista; el coste incurrido fue nulo al<br>ejecutarse dentro del nivel gratuito de cada API. . . . . | 85 |
| 5.7. Trazabilidad entre requisitos funcionales (RF) y no funcionales<br>(RNF) y las pruebas o secciones que acreditan su cumplimiento.                                                                                                                                                                                                | 92 |
| 6.1. Líneas de trabajo futuro con estimación de complejidad de<br>implementación. . . . .                                                                                                                                                                                                                                             | 99 |

# Capítulo 1

## Introducción

### 1.1. Contexto y motivación

La detección y respuesta ante amenazas en las organizaciones modernas se articula en torno a los Centros de Operaciones de Seguridad (*Security Operations Center*, SOC). En estos centros, plataformas de tipo SIEM (*Security Information and Event Management*) recogen y correlacionan de forma centralizada los registros de eventos generados por toda la infraestructura de una organización (cortafuegos, servidores, proxies y sistemas operativos), emitiendo alertas cuando un patrón dispara una regla de detección. El reto operativo no es la falta de información, sino su exceso: los analistas de seguridad reciben un flujo continuo de alertas en formato técnico que deben interpretar, contextualizar y priorizar bajo fuertes restricciones de tiempo.

Esta problemática no es teórica para el autor de este trabajo. Durante la realización de sus prácticas de empresa en el SOC de Telefónica Cybersecurity & Cloud Tech, en Granada, dentro del Proyecto Global 24x7, se pudo observar de primera mano el funcionamiento real de un centro de operaciones de alta criticidad. Los analistas de Nivel 1 (N1) gestionan colas de alertas continuas bajo Acuerdos de Nivel de Servicio (*Service Level Agreements*, SLA) que limitan el tiempo de respuesta a ventanas muy reducidas para los incidentes más graves. El triaje (clasificación y priorización inicial de un incidente para decidir si requiere escalado y con qué urgencia) de cada evento exige correlacionar la alerta con marcos de inteligencia de amenazas, descartar falsos positivos y documentar los hallazgos, todo ello bajo presión de tiempo. Esta carga constituye un cuello de botella operativo que resta capacidad al analista para centrarse en la toma de decisiones.

La dificultad del triaje no es solo de volumen. Cada alerta exige que el analista consulte fuentes externas de inteligencia de amenazas, identifique la técnica del adversario dentro del marco MITRE ATT&CK, un catálogo

de tácticas y técnicas de ataque ampliamente utilizado en la industria para clasificar amenazas (descrito en el Capítulo 2), descarte falsos positivos cruzando el evento con el historial de la máquina afectada y documente los hallazgos antes de escalar. La alerta en bruto llega como un objeto JSON técnico que no incluye ninguno de esos contextos; el analista debe construirlo a mano, para cada evento, bajo presión de tiempo.

Los Modelos de Lenguaje de Gran Tamaño (*Large Language Models*, LLM) pueden resumir, interpretar y razonar sobre texto técnico con una fluidez notable, lo que los hace adecuados para asistir en este proceso. Sin embargo, su uso en un contexto de ciberseguridad corporativa plantea una restricción inmediata: los eventos de seguridad contienen información sensible sobre la infraestructura de la organización (direcciones IP internas, nombres de equipos, credenciales parciales, patrones de tráfico) que no pueden transmitirse a un servicio en la nube de terceros sin comprometer la confidencialidad del entorno protegido. Esta restricción descarta, en la práctica, el uso directo de las API comerciales de los grandes modelos como solución de triaje para un SOC real. El despliegue local, mediante herramientas como Ollama que permiten ejecutar modelos de pesos abiertos en hardware propio, ofrece una salida a esta tensión: la capacidad de razonamiento del LLM sin que los datos de la infraestructura abandonen la red de la organización.

Ambas observaciones (un problema operativo real y una tecnología adecuada para abordarlo bajo los requisitos de privacidad del sector) motivan este Trabajo Fin de Grado: diseñar y construir un sistema que conecte Wazuh [19], una plataforma SIEM/XDR de código abierto, con un modelo de lenguaje ejecutado en local, de modo que las alertas crudas se transformen automáticamente en informes comprensibles y accionables que asistan al analista en su labor.

La viabilidad técnica de esta aproximación ha mejorado de forma notable en los últimos años con la proliferación de modelos de lenguaje de pesos abiertos de tamaño reducido. Modelos como Llama 3.2, con 3.000 millones de parámetros, pueden ejecutarse en hardware de uso general (en este proyecto, sobre un sistema con Apple Silicon mediante Ollama) con tiempos de inferencia aceptables y sin los costes de infraestructura asociados a los servicios comerciales. Esta disponibilidad de capacidad de razonamiento lingüístico en local es la condición técnica que hace compatible el uso de LLMs con el requisito de privacidad inherente a un SOC: el análisis se produce íntegramente dentro de la red de la organización, sin que los datos de la infraestructura abandonen el perímetro controlado.

## 1.2. Objetivos

El objetivo general del proyecto es desarrollar un *middleware* que actúe como capa de inteligencia entre Wazuh y un modelo de lenguaje local, enriqueciendo las alertas de seguridad y apoyando la toma de decisiones del analista. Este objetivo general se concreta en los siguientes objetivos específicos:

1. **Transformación de alertas.** Convertir las alertas en formato JSON técnico generadas por Wazuh en informes de incidente redactados en lenguaje natural, comprensibles para un analista.
2. **Correlación con MITRE ATT&CK.** Enriquecer cada alerta situándola dentro del marco MITRE ATT&CK, identificando las tácticas y técnicas del adversario y su posición en la *Cyber Kill Chain*.
3. **Inferencia de objetivos.** Razonar, a partir de la alerta y su contexto, sobre los objetivos probables del ataque y los posibles movimientos laterales del adversario (desplazamientos del atacante desde el sistema comprometido hacia otros equipos de la misma red con el fin de ampliar su control o alcanzar activos de mayor valor).
4. **Asistencia en el triaje.** Proponer acciones de respuesta recomendadas, incluyendo medidas de contención, manteniendo siempre la supervisión humana sobre cualquier acción con impacto.
5. **Consultas en lenguaje natural.** Permitir la búsqueda proactiva de amenazas (*threat hunting*) mediante consultas formuladas en lenguaje natural, que el sistema traduce a consultas estructuradas sobre el almacén de eventos.

## 1.3. Contribuciones del trabajo

El sistema construido integra en un entorno local, sobre un despliegue real de Wazuh, capacidades de análisis forense asistido por IA, *threat hunting* en lenguaje natural y respuesta activa con supervisión humana. Sus aportaciones concretas son las siguientes:

1. **Integración directa con el ecosistema Wazuh.** El *middleware* accede a la plataforma a través de sus dos planos nativos: el Indexer basado en OpenSearch (puerto 9200) para la recuperación de alertas y el análisis del historial, y la API REST del Manager (puerto 55000) para las acciones de respuesta. No se interpone ninguna capa de abstracción adicional entre el *middleware* y la plataforma, lo que simplifica el

despliegue en instalaciones existentes de Wazuh y reduce la superficie de ataque al eliminar componentes intermedios.

2. ***Threat hunting* en lenguaje natural sobre alertas reales.** El sistema traduce consultas formuladas en español a Query DSL de OpenSearch mediante un LLM, lo que permite al analista investigar el historial de eventos sin conocer la sintaxis del lenguaje de consultas ni el esquema de índice de Wazuh. La traducción incluye un mecanismo de reparación de JSON truncado que garantiza la robustez frente a respuestas incompletas del modelo cuando se opera con límites de contexto reducidos.
3. **Patrón *Human-in-the-Loop* para la respuesta activa.** Ninguna acción con impacto sobre la infraestructura se ejecuta de forma automática. El sistema exige la confirmación explícita del operador antes de cualquier bloqueo, y añade una segunda capa de validación (una lista blanca de direcciones críticas) que protege los nodos fundamentales de la red incluso ante fallos en el razonamiento del modelo.
4. **Arquitectura multi-proveedor con interfaz web.** El sistema permite seleccionar en tiempo de ejecución entre tres proveedores de inferencia (Ollama con Llama 3.2 en local, Gemini Flash de Google y Llama 3.3-70B vía Groq) desde una interfaz web en Streamlit que hace accesibles los tres modos de operación sin necesidad de línea de comandos. Esta flexibilidad permite al operador equilibrar, según el contexto, la privacidad que ofrece el modelo local frente a la capacidad de razonamiento de los modelos de mayor tamaño.

El código fuente completo del sistema se publica en abierto bajo licencia MIT (<https://github.com/maximartinm/TFG-Wazuh-LLM>) [8]; la Figura 1.1 muestra la organización de directorios del repositorio. El despliegue en producción descrito en la Sección 4.10 se distribuye en la rama `feat/k8s-deploy` del mismo repositorio.

## 1.4. Estructura de la memoria

El resto de esta memoria se organiza en cinco capítulos siguiendo el ciclo habitual del desarrollo de software: del análisis del problema y del estado del arte hasta las conclusiones, pasando por el diseño, la implementación y la evaluación experimental.

El Capítulo 2 está dividido en dos partes. La primera establece los conceptos preliminares necesarios para seguir el resto del documento: la arquitectura de un SOC, las plataformas SIEM y XDR, el funcionamiento de

Wazuh, los modelos de lenguaje de gran tamaño y el marco MITRE ATT&CK. La segunda parte revisa el estado del arte, analizando los trabajos previos sobre integración de LLMs en sistemas de ciberseguridad e identificando las carencias que este proyecto trata de cubrir. El Capítulo 3 presenta el análisis de requisitos del sistema, la metodología de desarrollo seguida y la planificación temporal del proyecto, incluyendo la selección razonada de las tecnologías empleadas. El Capítulo 4 detalla el diseño e implementación del *middleware*, describiendo la arquitectura general, las principales decisiones de diseño, el funcionamiento de cada módulo y el despliegue del sistema en un clúster Kubernetes local (Sección 4.10).

El Capítulo 5 recoge las cuatro pruebas realizadas y los resultados obtenidos, incluyendo la evaluación comparativa de los tres proveedores de inferencia sobre cinco vectores de ataque. Finalmente, el Capítulo 6 expone las conclusiones del trabajo, las limitaciones identificadas durante la evaluación y las líneas de evolución futura. Antes de la bibliografía se incluye una sección sobre el uso de herramientas de inteligencia artificial generativa en la elaboración de este trabajo. La memoria se cierra con el Apéndice A, que recoge los *prompts* completos utilizados en los módulos del *middleware*.

## Estructura del repositorio

```
TFG-Wazuh-LLM/
├── wazuh_llm/
│   ├── __init__.py
│   ├── middleware.py          # Orquestador principal
│   ├── nav.py                 # Barra de navegación web
│   ├── respuesta_activa.py    # Módulo de bloqueo
│   └── threat_hunting.py      # Consultas en lenguaje natural
├── pages/
│   ├── 1_Alertas.py           # Explorador de alertas
│   ├── 2_Analisis_LLM.py      # Análisis con LLM
│   └── 3_Threat_Hunting.py    # Hunting interactivo
├── prompts/
│   └── mitre_prompt.txt       # Plantilla de análisis forense
├── Home.py                    # Dashboard principal (Streamlit)
├── tests/
│   └── test_middleware.py     # Tests unitarios (pytest)
├── k8s/
│   ├── kind-cluster.yaml     # Despliegue en Kubernetes
│   ├── deployment.yaml       # Clúster kind sobre podman
│   ├── externals.yaml        # Dashboard Streamlit
│   ├── ingress.yaml          # Wazuh y Ollama fuera del clúster
│   ├── networkpolicy.yaml    # nginx + timeouts para la inferencia
│   ├── cronjob-batch.yaml    # default-deny egress + allowlist
│   ├── kustomization.yaml    # Triage programado (suspendido)
│   └── README.md              # Genera el Secret desde .env.k8s
├── Dockerfile                 # Runbook de despliegue
├── .dockerignore              # Imagen del middleware
├── .env.example               # Plantilla de configuración
├── .gitignore
├── pyproject.toml
└── README.md
```

Figura 1.1: Estructura del repositorio del proyecto.

## Capítulo 2

# Conceptos preliminares y estado del arte

Este capítulo se organiza en dos partes. La primera (Secciones 2.1–2.4) establece los conceptos preliminares: los centros de operaciones de seguridad y las tecnologías SIEM y XDR que los sustentan, la plataforma Wazuh como solución de código abierto, los modelos de lenguaje de gran tamaño y las razones para su despliegue local, y el marco MITRE ATT&CK junto con la *Cyber Kill Chain*. La segunda parte (Sección 2.5) constituye el estado del arte propiamente dicho: una revisión crítica de los trabajos que combinan inteligencia artificial y operaciones de seguridad, que sirve para contextualizar la propuesta de este TFG.

### 2.1. Centros de operaciones de seguridad: SIEM y XDR

Un Centro de Operaciones de Seguridad (SOC) es la unidad responsable de la monitorización continua, la detección y la respuesta ante incidentes de seguridad de una organización. Su funcionamiento se estructura habitualmente en niveles de escalado de analistas: una primera línea (N1) encargada de la monitorización y el triaje inicial de las alertas, una segunda línea (N2) que realiza investigaciones avanzadas y acciones de contención, y niveles superiores (N3 y coordinación) responsables de las decisiones críticas y de la interlocución con el cliente. Conviene distinguir tres conceptos en este contexto: el *evento*, entendido como cualquier registro detectable en la infraestructura; la *alerta*, que es el evento o conjunto de eventos correlacionados que disparan una regla de detección y requieren revisión; y el *incidente*, que se constituye cuando una alerta es analizada y se confirma la existencia de una amenaza real.

La pieza tecnológica central de un SOC es el sistema SIEM (*Security Information and Event Management*). Un SIEM recopila, normaliza, almacena y correlaciona de forma centralizada los registros de eventos procedentes de fuentes heterogéneas, generando alertas cuando se cumplen determinadas reglas de correlación. En el mercado conviven numerosas soluciones comerciales de este tipo, como Microsoft Sentinel, Splunk, IBM QRadar, Google SecOps o ArcSight, ampliamente utilizadas en entornos de producción.

Una evolución de este modelo son las plataformas XDR (*Extended Detection and Response*) y, en el ámbito del *endpoint* (un dispositivo final conectado a la red, como un servidor, una estación de trabajo o un equipo de escritorio), las soluciones EDR (*Endpoint Detection and Response*), que monitorizan el comportamiento interno de los equipos finales (las llamadas al sistema y la ejecución de procesos) y permiten ejecutar acciones de respuesta. Soluciones como CrowdStrike Falcon, Cortex XDR, Microsoft Defender o FortiEDR son representativas de esta categoría.

Frente a estas plataformas comerciales, que conllevan costes de licencia significativos, existen alternativas de código abierto que ofrecen funcionalidades comparables. Wazuh, descrito en la siguiente sección, es una de las más relevantes y constituye la base sobre la que se desarrolla este trabajo.

## 2.2. Wazuh

Wazuh es una plataforma de seguridad de código abierto que integra capacidades de SIEM y XDR [19] (disponible en <https://wazuh.com/>). Permite la detección de intrusiones, el análisis de registros, la monitorización de la integridad de los ficheros, la evaluación de la configuración de seguridad y la respuesta activa ante amenazas. Su arquitectura se compone de tres elementos principales:

- **Wazuh Manager.** Es el componente central, encargado de recibir los datos de los agentes, analizarlos mediante reglas y decodificadores, generar alertas y orquestar las acciones de respuesta. Expone una API REST de gestión que permite consultar el estado del sistema y ordenar acciones de forma programática.
- **Wazuh Indexer.** Es el motor de almacenamiento e indexación de las alertas, basado en OpenSearch [12]. Permite realizar búsquedas complejas sobre los eventos históricos mediante un lenguaje de consultas estructurado (Query DSL).
- **Agentes Wazuh.** Son los componentes ligeros instalados en los equipos monitorizados (*endpoints*), responsables de recopilar los datos de seguridad y ejecutar las acciones de respuesta que ordena el Manager.

Esta separación entre el plano de gestión (la API REST del Manager) y el plano de datos (el Indexer basado en OpenSearch) es relevante para el diseño de este proyecto, ya que el sistema interactúa con ambos planos de manera diferenciada (véase el diagrama de secuencia completo en la Figura 4.10 del Capítulo 4). Una de las capacidades nativas de Wazuh especialmente pertinente para este trabajo es la *respuesta activa* (*active response*), que permite ejecutar de forma automatizada *scripts* de contención (como el bloqueo de una dirección IP en el cortafuegos del *endpoint*) en respuesta a determinadas condiciones.

## 2.3. Modelos de lenguaje de gran tamaño

Los modelos de lenguaje de gran tamaño (LLM) son redes neuronales entrenadas sobre grandes volúmenes de texto para predecir y generar lenguaje natural. La mayoría se basan en la arquitectura *Transformer* [18], que procesa el texto mediante mecanismos de atención capaces de relacionar simultáneamente todos los elementos de una secuencia (palabras o fragmentos de texto) sin importar la distancia entre ellos; esto le permite capturar el significado global de una frase o párrafo de forma mucho más eficiente que las redes recurrentes anteriores. Su capacidad para resumir, clasificar, traducir y razonar sobre texto los hace especialmente adecuados para una tarea como la interpretación de alertas de seguridad, que llegan al analista en un formato técnico y de difícil interpretación directa.

El uso de estos modelos puede realizarse a través de servicios en la nube de terceros o mediante su ejecución en local. En el ámbito de la ciberseguridad, la ejecución local tiene ventajas concretas para este tipo de aplicación: los datos sensibles de la infraestructura nunca salen de la red de la organización, se evitan las tarifas por uso de las API comerciales y el sistema no queda sujeto a la disponibilidad de un proveedor externo. Por estas razones se ha optado por el despliegue local en este proyecto.

Para ello se emplea Ollama [11], una herramienta que simplifica la descarga y ejecución de modelos de lenguaje de pesos abiertos en hardware local, exponiendo una API REST sencilla para la generación de texto. Entre los modelos disponibles se encuentra la familia Llama, desarrollada por Meta [9], de la que el presente trabajo utiliza Llama 3.2.

## 2.4. MITRE ATT&CK y la Cyber Kill Chain

Para enriquecer las alertas con contexto de inteligencia de amenazas, este trabajo se apoya en dos marcos ampliamente adoptados en la industria.

La *Cyber Kill Chain*, formulada por Lockheed Martin [5], modela un ataque dirigido como una secuencia de siete fases (reconocimiento, preparación, distribución, explotación, instalación, comando y control, y acciones sobre los objetivos), partiendo de la premisa de que interrumpir cualquiera de estas fases permite frustrar el ataque. Este modelo aporta una narrativa que ayuda a situar una alerta concreta dentro del avance global de una intrusión. La Figura 2.1 ilustra las siete fases y su progresión.



Figura 2.1: Las siete fases de la *Cyber Kill Chain* de Lockheed Martin [5]: desde el reconocimiento inicial hasta las acciones sobre los objetivos finales del atacante. Fuente: INCIBE [6].

El marco MITRE ATT&CK [15, 17] complementa lo anterior con una base de conocimiento estructurada de las tácticas (los objetivos del adversario, como el acceso a credenciales o la escalada de privilegios) y las técnicas (los métodos concretos empleados para lograrlos), observadas en ataques reales y accesibles en <https://attack.mitre.org/>. Cada técnica se identifica con un código único (por ejemplo, T1110 para los ataques de fuerza bruta). Wazuh etiqueta muchas de sus reglas con identificadores MITRE ATT&CK, lo que permite al sistema desarrollado correlacionar automáticamente cada alerta con la técnica correspondiente y razonar sobre ella. La Figura 2.2 muestra la estructura de tácticas y técnicas del marco.

| Tactic               | Count | Techniques                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reconnaissance       | 12    | Active Scanning (3), Gather Victim Host Information (4), Gather Victim Identity Information (3), Gather Victim Network Information (3), Gather Victim Org Information (4), Phishing for Information (4), Query Public AI Services (2), Search Closed Sources (2), Search Open Technical Databases (3), Search Open Websites/Domains (3), Search Threat Vendor Data, Search Victim-Owned Websites                                              |
| Resource Development | 9     | Acquire Access, Acquire Infrastructure (3), Compromise Accounts (3), Compromise Infrastructure (3), Develop Capabilities (4), Establish Accounts (3), Generate Content (2), Replication Through Removable Media, Stage Capabilities (3), Supply Chain Compromise (3), Trusted Relationship, Valid Accounts (4), Wi-Fi Networks                                                                                                                |
| Initial Access       | 11    | Content Injection, Drive-by Compromise, Exploit Public-Facing Application, External Remote Services, Hardware Additions, Phishing (4), Replication Through Removable Media, Supply Chain Compromise (3), Trusted Relationship, Valid Accounts (4), Wi-Fi Networks                                                                                                                                                                             |
| Execution            | 20    | BITS Jobs, Cloud Administration Command, Command and Scripting Interpreter (13), Container Administration Command, Deploy Container, ESXi Administration Command, Exploitation for Client Execution, Hijack Execution Flow (12), Input Injection, Inter-Process Communication (3), Native API, Poisoned Pipeline Execution, Scheduled                                                                                                         |
| Persistence          | 22    | Account Manipulation (7), BITS Jobs, Boot or Logon Autostart Execution (14), Boot or Logon Initialization Scripts (3), Cloud Application Integration, Compromise Host Software Binary, Create Account (3), Create or Modify System Process (2), Domain or Tenant Policy Modification (2), Event Triggered Execution (14), Event Triggered Execution (18), Exclusive Control, External Remote Services                                         |
| Privilege Escalation | 13    | Abuse Elevation Control Mechanism (4), Access Token Manipulation (3), Build Image on Host, Debugger Evasion, Delay Execution, Deobfuscate/Decode Files or Information, Direct Volume Access, Execution Guardrails (2), Exploitation for Stealth, Hide Artifacts (14), Hijack Execution Flow (12), Indicator Removal (8), Escape to Host, Event Triggered Execution (18), Exploitation for Privilege Escalation (14), Pre-OS Boot (3), Process |
| Stealth              | 30    | Access Token Manipulation (3), BITS Jobs, Build Image on Host, Debugger Evasion, Delay Execution, Deobfuscate/Decode Files or Information, Direct Volume Access, Execution Guardrails (2), Exploitation for Stealth, Hide Artifacts (14), Hijack Execution Flow (12), Indicator Removal (8), Escape to Host, Event Triggered Execution (18), Exploitation for Privilege Escalation (14), Pre-OS Boot (3), Process                             |
| Defense Impairment   | 18    | Disable or Modify System Firewall (3), Disable or Modify Tools (3), Domain or Tenant Policy Modification (2), Downgrade Attack, Exploitation for Defense Impairment, File and Directory Permissions Modification (2), Input Capture (4), Modify Authentication Process (3), Modify Cloud Compute Infrastructure (3), Modify Cloud Resource Hierarchy, Modify Registry                                                                         |
| Credential Access    | 17    | Adversary-in-the-Middle (4), Brute Force (4), Credentials from Password Stores (3), Exploitation for Credential Access, Forced Authentication, Forge Web Credentials (2), Input Capture (4), Modify Authentication Process (3), Multi-Factor Authentication Interception, Multi-Factor Authentication Request Generation, Network                                                                                                             |
| Discovery            | 34    | Account Discovery (4), Application Window Discovery, Browser Information Discovery, Cloud Infrastructure Discovery, Cloud Service Dashboard, Cloud Service Discovery, Cloud Storage Object Discovery, Container and Resource Discovery, Debugger Evasion, Device Driver Discovery, Domain Trust Discovery, File and Directory Discovery, Group Policy Discovery                                                                               |

Figura 2.2: Extracto de la matriz MITRE ATT&CK [17]: las columnas representan tácticas (objetivos del adversario) y las filas agrupan las técnicas concretas empleadas para lograrlos. Disponible en <https://attack.mitre.org/>.

## 2.5. Trabajos relacionados

Tradicionalmente, las técnicas de aprendizaje automático se han empleado para la detección de anomalías y la reducción de falsos positivos en los SIEM. La irrupción de los modelos de lenguaje de gran tamaño ha abierto una línea de trabajo centrada en el análisis de registros de seguridad, la generación de informes de incidente y el apoyo al analista en el triaje. Los trabajos recientes pueden agruparse en dos categorías: revisiones del estado del arte, que ofrecen un mapa del espacio de soluciones, y propuestas de sistema concretas, que implementan y evalúan una arquitectura de integración específica.

### 2.5.1. Revisiones del estado del arte

Motlagh et al. [10] ofrecen una caracterización amplia del estado del arte, identificando aplicaciones defensivas (detección de malware, análisis de vulnerabilidades, respuesta a incidentes) y ofensivas (generación de código malicioso, ataques de ingeniería social asistidos por LLM). Su taxonomía distingue entre el uso de LLMs como herramienta de análisis y su uso como superficie de ataque, y constata que la mayoría de los trabajos publicados hasta 2024 se apoyan en modelos de acceso remoto (GPT-4, Claude) sin

contemplar el procesamiento local como requisito de diseño. Esta omisión es relevante para el presente TFG, que trata la ejecución local como requisito primario y no como alternativa secundaria de menor rendimiento: las alertas de seguridad contienen información de infraestructura que no puede abandonar la red de la organización sin evaluación previa de las implicaciones de privacidad.

Habibzadeh et al. [4] examinan específicamente el potencial de los LLMs en el contexto de los SOC: la automatización del análisis de registros, la generación de resúmenes de incidentes y el apoyo al triaje de primera línea. Los autores señalan como limitación central el riesgo de privacidad que conlleva delegar el procesamiento de eventos de seguridad en modelos externos, ya que los registros contienen datos sensibles sobre la infraestructura (nombres de equipos, direcciones IP internas, usuarios, credenciales parciales) cuya exposición a un tercero puede suponer un incumplimiento normativo en entornos regulados. Esta limitación es precisamente la que motiva el diseño de privacidad de este TFG: el procesamiento local como opción predeterminada y la externalización a proveedores en la nube únicamente mediante decisión explícita del analista, con advertencia visible en la interfaz.

### 2.5.2. Propuestas de sistema

CORTEX [21] presenta una arquitectura multiagente en la que varios LLMs especializados colaboran en el triaje de alertas de SOC: un agente examina patrones de comportamiento a partir del registro bruto, otro recaba evidencia de fuentes externas (bases de datos de vulnerabilidades, inteligencia de amenazas en abierto) y un tercero sintetiza la decisión final en un informe auditable. La especialización de cada agente permite que cada uno opere sobre un subconjunto bien delimitado del problema, reduciendo la tasa de alucinaciones respecto a un agente único que debe razonar simultáneamente sobre todos los aspectos del incidente. Los autores evalúan el sistema sobre datos de producción de un SOC real y muestran una reducción estadísticamente significativa de los falsos positivos. El coste de esta arquitectura es la complejidad de despliegue: requiere varios modelos en ejecución simultánea y mecanismos de coordinación entre agentes que presuponen una infraestructura de inferencia robusta, inaccesible para entornos académicos o con recursos limitados.

AIDR [22] aborda el mismo problema desde el ángulo de la eficiencia computacional. Mediante compresión de razonamiento por gradiente y ajuste fino con LoRA, el sistema reduce un 68 % los tokens procesados y mejora la latencia un 40,6 % respecto al razonamiento estándar con cadena de pensamiento, manteniendo el procesamiento local para cumplir los requisitos de residencia de datos. El ajuste fino con LoRA requiere, sin embargo,

conjuntos de datos de alertas anotadas con respuestas de referencia, un recurso que raramente está disponible en organizaciones con SOC de tamaño reducido o en entornos de nueva implantación. En ausencia de datos etiquetados, el ajuste fino no es una opción viable sin incurrir en costes significativos de anotación manual, lo que lo descarta en la práctica como mecanismo de adaptación para el contexto de este TFG.

### 2.5.3. Comparativa y posicionamiento

La Tabla 2.1 sintetiza las principales características de los trabajos anteriores y las contrasta con la propuesta de este TFG a lo largo de cinco dimensiones: el objetivo principal de cada sistema, si procesa las alertas de forma local por defecto sin enviar datos a terceros, si requiere un conjunto de alertas anotadas para funcionar, si es capaz de ejecutar acciones de contención activa y si se integra de forma nativa con una plataforma SIEM concreta. Estas dimensiones se han seleccionado por ser las que mejor reflejan las restricciones de privacidad, los requisitos de datos y el alcance operativo de cada propuesta, que son precisamente los ejes en los que este TFG toma posiciones diferenciadas respecto al estado del arte.

| Trabajo               | Objetivo                         | Local predeterminado | Datos etiquetados | Respuesta activa | SIEM concreto         |
|-----------------------|----------------------------------|----------------------|-------------------|------------------|-----------------------|
| Motlagh et al. [10]   | Revisión campo                   | No                   | No                | No               | No                    |
| Habibzadeh et al. [4] | Revisión SOC+LLM                 | No                   | No                | No               | No                    |
| CORTEX [21]           | Triaje multi-agente              | Parcial              | Sí                | No               | Sí (genérico)         |
| AIDR [22]             | Eficiencia LoRA                  | Sí                   | Sí                | No               | Sí (genérico)         |
| <b>Este TFG</b>       | <b>Triaje + respuesta activa</b> | <b>Sí</b>            | <b>No</b>         | <b>Sí</b>        | <b>Wazuh (nativo)</b> |

Tabla 2.1: Comparativa de los trabajos relacionados. "Local predeterminado" indica si el sistema procesa las alertas sin enviar datos a terceros por defecto; "Datos etiquetados" indica si requiere un conjunto de alertas anotadas para funcionar; "Respuesta activa" indica si puede ejecutar acciones de contención; "SIEM concreto" indica si la integración es nativa con una plataforma SIEM específica.

El presente trabajo comparte con CORTEX y AIDR el objetivo de asistir al analista sin sustituirlo. Su diferencia principal respecto a esas propuestas es doble. En el plano de la integración, el sistema se conecta directamente con Wazuh a través de sus dos planos de comunicación (la API REST del Manager y el Indexer de OpenSearch), sin capa de abstracción intermedia ni dependencia de una plataforma SIEM genérica. En el plano de la adaptación del modelo, la personalización se realiza íntegramente mediante ingeniería

de *prompt*, lo que prescinde de datos etiquetados y de infraestructura de entrenamiento, haciendo el sistema desplegable en cualquier entorno con Wazuh y Ollama. El sistema amplía además el alcance de las propuestas revisadas con dos modos de operación no presentes en ellas: las consultas en lenguaje natural sobre el historial de alertas almacenado en el Indexer y la respuesta activa con confirmación explícita del operador, que cierran el ciclo desde la alerta bruta hasta la acción de contención dentro de un único *middleware* ligero.

En conjunto, la revisión muestra que los enfoques basados en LLM para el análisis de alertas de SOC son activos y con resultados prometedores, pero que las soluciones más avanzadas (CORTEX, AIDR) exigen datos de producción, infraestructura de ajuste fino o múltiples agentes especializados. El presente trabajo plantea una alternativa de menor coste de despliegue, orientada a un entorno concreto (Wazuh) y centrada en la ingeniería del *prompt* como mecanismo de adaptación principal.

## Capítulo 3

# Análisis y planificación

En este capítulo se analizan los requisitos que debe satisfacer el sistema, se describe la metodología de desarrollo seguida y se presenta la planificación temporal del proyecto.

### 3.1. Requisitos del sistema

A partir de los objetivos definidos en el Capítulo 1, se han identificado los siguientes requisitos funcionales y no funcionales.

#### 3.1.1. Requisitos funcionales

Los requisitos funcionales recogen las capacidades observables que el sistema debe ofrecer, derivadas de los objetivos específicos enunciados en el Capítulo 1:

- **RF1.** El sistema debe conectarse al Wazuh Indexer y recuperar un número configurable de alertas, filtrables por nivel de severidad.
- **RF2.** El sistema debe generar, para cada alerta, un informe en lenguaje natural que incluya un resumen del incidente, su análisis forense, la correlación con MITRE ATT&CK, la inferencia de objetivos y un plan de respuesta recomendado.
- **RF3.** El sistema debe permitir la realización de consultas en lenguaje natural, traduciéndolas a consultas estructuradas (Query DSL) que se ejecutan contra el Indexer, y mostrar los resultados.
- **RF4.** El sistema debe ser capaz de proponer el bloqueo de direcciones IP maliciosas y ejecutarlo, a través de la API de Wazuh, únicamente tras la confirmación explícita de un operador humano.

- **RF5.** El sistema debe ofrecer una interfaz de línea de comandos que permita seleccionar el modo de operación y sus parámetros.
- **RF6.** El sistema debe ofrecer una interfaz web que exponga de forma visual e interactiva las funcionalidades de análisis de alertas, consultas en lenguaje natural y respuesta activa.

### 3.1.2. Requisitos no funcionales

Los requisitos no funcionales establecen las restricciones de calidad y diseño que condicionan la solución sin describir comportamientos concretos:

- **RNF1 (Privacidad).** El procesamiento de las alertas mediante el modelo de lenguaje debe realizarse en local de forma predeterminada, sin que los datos abandonen la red de la organización. El sistema permite seleccionar proveedores externos (Gemini, Groq) como alternativa, tanto en la interfaz web como en la línea de comandos; al activar esta opción, el analista acepta que los eventos sean procesados en la infraestructura en la nube del proveedor elegido.
- **RNF2 (Seguridad).** Ninguna acción con impacto sobre la infraestructura (en particular, el bloqueo de IP) debe ejecutarse de forma automática; debe mediar siempre la aprobación humana. Asimismo, el sistema debe proteger las direcciones críticas frente a bloqueos accidentales.
- **RNF3 (Robustez).** El sistema debe gestionar de forma controlada los fallos previsibles (indisponibilidad de Wazuh o del modelo de lenguaje, respuestas mal formadas del LLM) sin terminar con un error genérico.
- **RNF4 (Usabilidad).** La salida debe ser legible para un analista y la herramienta debe ser accesible tanto desde la línea de comandos como a través de una interfaz web.
- **RNF5 (Mantenibilidad).** El código debe estar organizado de forma modular, separando las distintas funcionalidades para facilitar su evolución.

## 3.2. Casos de uso

El actor principal del sistema es el **Analista de Seguridad**, que interactúa con el *middleware* para analizar alertas, investigar el historial de eventos e iniciar acciones de respuesta activa. El sistema actúa como intermediario entre el analista y la infraestructura Wazuh, abstrayendo la

complejidad de las consultas al Indexer y la comunicación con la API del Manager. Se identifican cinco casos de uso.

| <b>CU-01: Análisis de alertas de seguridad</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                                          | Analista de Seguridad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Descripción                                    | El analista solicita el análisis de las $N$ alertas más recientes que superen un nivel de severidad dado; el sistema las recupera del Indexer, construye un <i>prompt</i> forense por alerta y presenta el informe generado por el LLM.                                                                                                                                                                                                                                                                                                                                                                                     |
| Precondiciones                                 | Wazuh (Manager e Indexer) accesible en red; al menos un proveedor de inferencia disponible.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Flujo principal                                | <ol style="list-style-type: none"> <li>1. El analista invoca el sistema con <code>--alertas N --nivel M</code> (CLI) o desde la página <i>Análisis con LLM</i> de la interfaz web.</li> <li>2. El sistema se autentica en la API del Manager y obtiene el token JWT.</li> <li>3. El sistema consulta el Indexer y recupera las <math>N</math> alertas de nivel <math>\geq M</math>.</li> <li>4. Para cada alerta, el sistema construye el <i>prompt</i>, lo envía al proveedor seleccionado y valida la respuesta.</li> <li>5. Se presenta un informe estructurado en cinco secciones por cada alerta procesada.</li> </ol> |
| Postcondición                                  | El analista dispone de un informe forense con correlación MITRE ATT&CK para cada alerta.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

Tabla 3.1: Caso de uso CU-01: análisis de alertas de seguridad.

| <b>CU-02: Consulta de <i>threat hunting</i> en lenguaje natural</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                                                               | Analista de Seguridad                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Descripción                                                         | El analista formula en español una consulta sobre el historial de eventos; el sistema la traduce a Query DSL y ejecuta la búsqueda contra el Indexer.                                                                                                                                                                                                                                                                                                  |
| Precondiciones                                                      | Indexer accesible en el puerto 9200; motor de inferencia disponible.                                                                                                                                                                                                                                                                                                                                                                                   |
| Flujo principal                                                     | <ol style="list-style-type: none"> <li>1. El analista activa el modo con <code>--hunting</code> (CLI) o desde la página <i>Threat Hunting</i> de la interfaz web, y escribe la consulta en lenguaje natural.</li> <li>2. El sistema traduce la consulta a Query DSL mediante el LLM (reparando el JSON si está truncado con <code>reparar_json_truncado()</code>), ejecuta la búsqueda contra el Indexer y muestra los resultados en tabla.</li> </ol> |
| Postcondición                                                       | Se presentan los eventos del Indexer que satisfacen la consulta formulada.                                                                                                                                                                                                                                                                                                                                                                             |

Tabla 3.2: Caso de uso CU-02: consulta de *threat hunting* en lenguaje natural.

| <b>CU-03: Respuesta activa con confirmación humana</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                                                  | Analista de Seguridad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Descripción                                            | Tras recibir el informe de un análisis (CU-01), el analista decide si bloquear alguna de las IPs candidatas identificadas; el bloqueo se ejecuta únicamente tras su confirmación explícita.                                                                                                                                                                                                                                                                                                                               |
| Precondiciones                                         | CU-01 completado; API del Manager accesible con token válido; <code>firewall-drop</code> habilitado en <code>ossec.conf</code> (véase la Sección 4.9.2).                                                                                                                                                                                                                                                                                                                                                                  |
| Flujo principal                                        | <ol style="list-style-type: none"> <li>1. El sistema extrae IPs candidatas del informe mediante expresión regular y aplica la lista blanca (<code>127.0.0.1, ::1</code>, IP del Manager).</li> <li>2. El sistema presenta las IPs restantes al analista; este escribe <b>CONFIRMAR</b> para cada IP que desea bloquear.</li> <li>3. El sistema llama al <i>endpoint</i> de respuesta activa con <code>firewall-drop0</code>; Wazuh aplica la regla <b>DROP</b> de <code>iptables</code> en el <i>endpoint</i>.</li> </ol> |
| Postcondición                                          | La IP confirmada queda bloqueada en el cortafuegos del <i>endpoint</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Tabla 3.3: Caso de uso CU-03: respuesta activa con confirmación humana.

| <b>CU-04: Exploración de alertas en la interfaz web</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                                                   | Analista de Seguridad                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Descripción                                             | El analista navega el historial de alertas con filtros interactivos, consulta los metadatos de cada evento y accede al JSON original generado por Wazuh.                                                                                                                                                                                                                                                                                                                             |
| Precondiciones                                          | Interfaz web activa ( <code>streamlit run Home.py</code> ); Indexer accesible.                                                                                                                                                                                                                                                                                                                                                                                                       |
| Flujo principal                                         | <ol style="list-style-type: none"> <li>1. El analista abre la interfaz web en <code>http://localhost:8501</code> y navega a la página <i>Alertas</i>.</li> <li>2. El analista ajusta los controles de nivel de severidad y número de alertas.</li> <li>3. El sistema recupera las alertas del Indexer y las muestra en tabla con sus campos MITRE ATT&amp;CK.</li> <li>4. El analista selecciona una alerta para ver su detalle estructurado y el JSON completo original.</li> </ol> |
| Postcondición                                           | El analista ha consultado el historial de alertas filtrado según sus criterios.                                                                                                                                                                                                                                                                                                                                                                                                      |

Tabla 3.4: Caso de uso CU-04: exploración de alertas en la interfaz web.

| <b>CU-05: Selección del proveedor de inferencia</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor                                               | Analista de Seguridad                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Descripción                                         | El analista elige el motor LLM que se utilizará para el análisis o las consultas de <i>hunting</i> : Ollama con Llama 3.2 en local, Gemini Flash o Llama 3.3-70B vía Groq.                                                                                                                                                                                                                                                                                                               |
| Precondiciones                                      | Al menos un proveedor disponible: Ollama activo, o clave de API de Gemini/Groq configurada en el fichero <code>.env</code> .                                                                                                                                                                                                                                                                                                                                                             |
| Flujo principal                                     | <ol style="list-style-type: none"> <li>1a. En la interfaz web: el analista pulsa el botón del proveedor deseado en la página <i>Análisis con LLM</i>; el sistema lo resalta con borde y fondo diferenciados.</li> <li>1b. En la CLI: el analista indica <code>--proveedor {ollama,gemini,groq}</code>; si omite el parámetro, el sistema lo solicita de forma interactiva.</li> <li>2. El sistema utiliza ese proveedor para todas las llamadas al LLM de la sesión en curso.</li> </ol> |
| Postcondición                                       | Las inferencias de la sesión se realizan con el proveedor seleccionado por el analista.                                                                                                                                                                                                                                                                                                                                                                                                  |

Tabla 3.5: Caso de uso CU-05: selección del proveedor de inferencia.

### 3.3. Metodología de desarrollo

El desarrollo se ha abordado mediante una **metodología iterativa e incremental**: el trabajo se organizó en fases de desarrollo progresivo e incremental, cada una de las cuales añadió funcionalidad de forma progresiva sobre una base validada. Cada fase partió de una prueba de concepto, sobre la que se fueron detectando y corrigiendo limitaciones, refactorizando el código y ampliando sus capacidades. Esta aproximación (habitual en proyectos de software unipersonales y afín a los principios ágiles de entrega continua de valor) ha permitido disponer en todo momento de una versión funcional del sistema y validar cada incremento antes de avanzar al siguiente. Aunque la metodología general sigue un enfoque iterativo, la redacción de la memoria se solapó con las fases de desarrollo.

El control de versiones se ha gestionado con Git, alojando el repositorio en GitHub [8] (<https://github.com/maximartinm/TFG-Wazuh-LLM>). El proyecto se ha empaquetado siguiendo los estándares modernos de Python (`pyproject.toml`), lo que permite su instalación como un paquete y la exposición de un comando de consola (`wazuh-ia`).

### 3.4. Selección de tecnologías

Cada componente del sistema fue elegido tras evaluar las alternativas disponibles en su categoría. Esta sección justifica las decisiones adoptadas en función de los requisitos del proyecto, con especial atención a los criterios de código abierto, integración nativa y privacidad.

#### 3.4.1. Plataforma SIEM y XDR: Wazuh frente a Splunk y Elastic SIEM

Para la plataforma SIEM se evaluaron tres opciones principales: Wazuh, Splunk y Elastic SIEM. Las tres ofrecen detección de intrusiones, indexación de eventos y alguna forma de respuesta activa, pero presentan diferencias relevantes para los objetivos de este proyecto.

Splunk es la solución comercial de referencia en entornos de producción, con una propuesta de valor madura y amplias capacidades de correlación y visualización. Su limitación principal es el modelo de licencias: el plan gratuito está limitado a 500 MB de indexación diaria, lo que lo hace inaccesible para un entorno de pruebas sin presupuesto, y las licencias de producción se calculan por volumen de datos ingestados. Elastic SIEM (anteriormente ELK Stack) es una alternativa de código abierto con capacidades comparables, pero requiere una configuración más elaborada para activar respuestas activas

y no expone una API REST de gestión con autenticación JWT equivalente a la de Wazuh para el disparo programático de acciones de contención.

Wazuh [19] fue seleccionado porque combina tres atributos que los otros no ofrecen simultáneamente: es completamente libre y sin restricciones de licencia ni de volumen, expone una API REST de gestión autenticada con JWT que permite ordenar acciones de respuesta de forma programática, y etiqueta sus reglas con identificadores MITRE ATT&CK que el *middleware* puede aprovechar directamente sin transformación adicional. La disponibilidad de imágenes Docker oficiales [20] facilitó además el despliegue en el entorno de pruebas sin instalación nativa en el sistema anfitrión, aislando los componentes del SIEM del entorno de desarrollo.

### 3.4.2. Motor de inferencia local: Ollama frente a llama.cpp y Transformers

Para la ejecución local del modelo de lenguaje se evaluaron tres alternativas: Ollama [11], llama.cpp y la biblioteca Transformers de Hugging Face [9].

llama.cpp es el motor de referencia para ejecutar modelos cuantizados en CPU con bajo consumo de memoria; de hecho, Ollama lo utiliza internamente. Su interfaz es de bajo nivel: requiere compilar el binario, gestionar manualmente los ficheros de modelo y construir la integración con la aplicación llamando directamente al proceso. Transformers es la biblioteca estándar del ecosistema PyTorch para la carga y la inferencia de modelos de lenguaje en GPU o CPU, con soporte para una gama de modelos más amplia que llama.cpp; su coste es una mayor complejidad de instalación y unos requisitos de memoria más elevados.

Ollama envuelve llama.cpp en una interfaz de alto nivel: gestiona la descarga y el almacenamiento de los modelos, expone una API REST sencilla en el puerto 11434 y hace que el cambio de modelo se reduzca a un único parámetro de configuración. Para los objetivos de este proyecto (integrar un modelo local como un servicio consumible mediante HTTP, sin gestionar manualmente ficheros de pesos ni compilar binarios) esta abstracción es la más conveniente. La API de Ollama es compatible en estructura con las de Gemini y Groq, lo que simplificó la implementación de la capa de abstracción de proveedores en `middleware.py` y permitió que los tres proveedores compartan la misma interfaz de llamada.

### 3.4.3. Lenguaje de implementación: Python frente a Go y Java

Python fue seleccionado como lenguaje de implementación por tres razones complementarias. Primera, el ecosistema de bibliotecas para la integración con servicios REST (`requests`), la lectura de variables de entorno (`python-dotenv`), el análisis de ficheros JSON (biblioteca estándar) y el procesamiento de texto mediante expresiones regulares (`re`) está maduro, bien documentado y no requiere dependencias adicionales de infraestructura. Segunda, el marco Streamlit, adoptado para la interfaz web (Sección 4.7), requiere Python y ofrece componentes nativos para la visualización de datos de seguridad sin necesidad de código JavaScript. Tercera, Python es el lenguaje predominante en los ecosistemas de inteligencia artificial y ciberseguridad, lo que facilita la comprensión del código y su integración futura con otras herramientas del sector.

Alternativas como Go habrían ofrecido mejor rendimiento en tiempo de ejecución y un binario autocontenido sin dependencias de entorno virtual; Java habría aportado mayor tipado estático y un ecosistema empresarial más maduro con soporte a largo plazo. Sin embargo, ambas aumentan la complejidad de desarrollo sin beneficios tangibles para un *middleware* cuyo cuello de botella es el tiempo de inferencia del LLM (entre 2,6 y 39,5 segundos según el proveedor, Tabla 4.1), no el procesamiento en el lado del servidor. Go carece además del ecosistema de aprendizaje automático y de los clientes HTTP de alto nivel que Python ofrece de forma nativa.

### 3.4.4. Marco de la interfaz web: Streamlit frente a Flask, FastAPI y Dash

Para la interfaz web se evaluaron cuatro marcos: Streamlit [14], Flask, FastAPI y Dash. Flask y FastAPI son marcos orientados a servicios web de propósito general: permiten construir cualquier aplicación HTTP, pero requieren plantillas HTML, código JavaScript para la interactividad y una separación explícita entre la lógica del servidor y la presentación. Ambos son herramientas adecuadas cuando el objetivo es construir una API REST o una aplicación web con lógica de enrutamiento compleja; para una herramienta de análisis orientada a la exploración interactiva de datos, implican una inversión de desarrollo desproporcionada respecto al valor aportado. Dash es un marco orientado a aplicaciones de datos interactivos con componentes de visualización de alto nivel; su curva de aprendizaje es más pronunciada que la de Streamlit para flujos lineales como los de este sistema.

Streamlit [14] fue seleccionado porque sus componentes nativos (tablas con datos de alertas, barras de progreso durante la inferencia, despleables para

inspeccionar informes, campos de texto para las consultas de *threat hunting*, botones de selección de proveedor) se corresponden directamente con los flujos del sistema sin código adicional de presentación. La arquitectura de páginas múltiples de Streamlit organiza la aplicación en un directorio `pages/` donde cada fichero Python corresponde a un modo de operación del *middleware*, manteniendo la correspondencia entre módulos del sistema y páginas de la interfaz y facilitando la extensión futura con nuevas funcionalidades sin afectar a las existentes.

### 3.4.5. Orquestación del despliegue: Kubernetes con kind y podman

Para el despliegue en producción del *dashboard* se evaluaron tres alternativas: Docker Compose, Minikube y kind (*Kubernetes in Docker*).

Docker Compose es la opción más sencilla para desplegar contenedores en un único anfitrión; sin embargo, no ofrece gestión declarativa de secretos, control de tráfico a nivel de red ni primitivas de escalado, lo que limita su utilidad como base para una evolución futura del sistema. Minikube es el entorno local de Kubernetes más extendido, pero en macOS con Apple Silicon requiere ejecutar una máquina virtual completa mediante QEMU o Virtual-Box, lo que añade una capa de virtualización adicional e incompatibilidades con el modo *rootless*.

kind fue seleccionado porque ejecuta los nodos del clúster directamente como contenedores, reduciendo el consumo de recursos respecto a Minikube, y soporta podman como proveedor de contenedores mediante la variable `KIND_EXPERIMENTAL_PROVIDER=podman`. Podman en modo *rootless* [13] elimina la necesidad de un demonio con privilegios de root, lo que reduce la superficie de ataque del entorno de despliegue y es compatible con macOS ARM64. La combinación kind sobre podman proporciona un clúster Kubernetes completo, con soporte para Ingress, NetworkPolicy y Kustomize, sin dependencias externas más allá de las herramientas de línea de comandos.

## 3.5. Planificación temporal

El proyecto se ha estructurado en las siguientes fases:

- **Fase 1: Transformación de alertas.** Conexión con Wazuh y generación del primer informe enriquecido a partir de una alerta (prueba de concepto).
- **Fase 2: Correlación MITRE ATT&CK.** Incorporación del contexto de inteligencia de amenazas y de la *Kill Chain* en el informe generado.

- **Fase 3: Threat hunting.** Desarrollo del modo de consultas en lenguaje natural y su traducción a Query DSL.
- **Fase 4: Respuesta activa.** Desarrollo del bloqueo de IP con confirmación humana a través de la API de Wazuh.
- **Fase 5: Consolidación.** Refactorización, mejora de la robustez, pruebas con datos reales y documentación.
- **Fase 6: Interfaz web.** Desarrollo de la aplicación web con Streamlit, integración de los tres módulos del *middleware* y verificación del flujo completo en el entorno de pruebas.
- **Fase 7: Despliegue en Kubernetes.** Migración del *dashboard* Streamlit como una imagen de docker, configuración del clúster kind sobre podman en modo *rootless*, integración de los servicios externos de Wazuh y Ollama mediante *EndpointSlice* manuales, y verificación del despliegue completo en el entorno de pruebas.

La Figura 3.1 muestra la distribución temporal del proyecto. La redacción de la memoria se solapó con las fases de desarrollo, siguiendo la metodología iterativa adoptada.

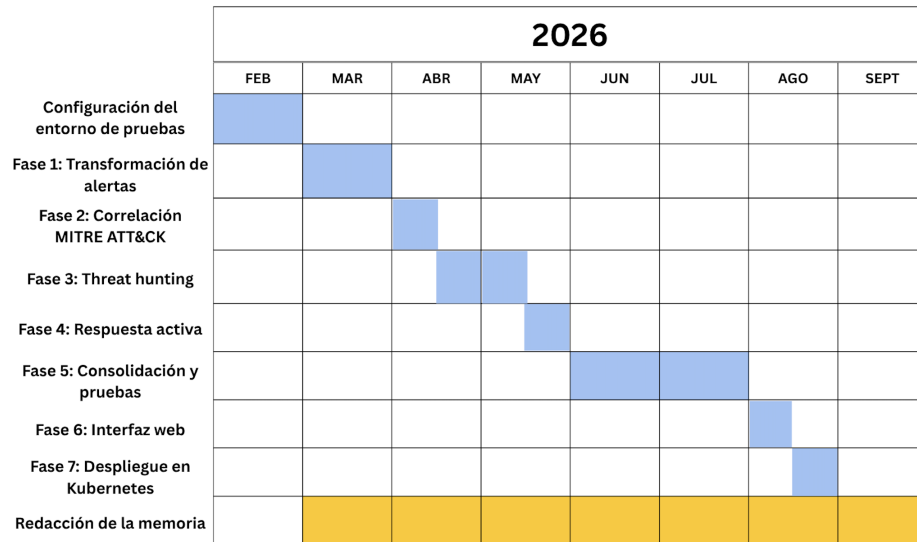


Figura 3.1: Planificación temporal del proyecto (febrero – septiembre de 2026).

### 3.6. Presupuesto

El presupuesto recoge los costes imputables al desarrollo de este TFG. La asignatura tiene asignados 12 créditos ECTS, lo que equivale a 300 horas de trabajo del estudiante (25 horas por crédito), y es el número de horas que se utiliza como base del cálculo.

#### Coste de personal

La tarifa horaria se obtiene del XIX Convenio Colectivo Estatal de Empresas de Consultoría, Tecnologías de la Información y Estudios de Mercado [1], tomando el Área 3 (Desarrollo de Software), Grupo Profesional C, con una retribución bruta anual de 26.576,62 € para 2026. Añadiendo un 31 % en concepto de cotizaciones sociales a cargo de la empresa y asumiendo una jornada anual de 1.800 horas, la tarifa resultante es de 19,34 €/h.

| Rol                              | Horas  | Coste/hora | Total      |
|----------------------------------|--------|------------|------------|
| Desarrollador junior (autor)     | 300 h  | 19,34 €/h  | 5.802,00 € |
| Director de TFG (×2, estimación) | 15 h×2 | 27,00 €/h  | 810,00 €   |

Tabla 3.6: Coste de personal. La tarifa del desarrollador proviene del [1]. La tarifa de los directores es una estimación basada en la retribución bruta de personal docente e investigador de universidad pública española, incluyendo la parte proporcional de cotizaciones sociales.

#### Coste de hardware

Se aplica amortización lineal a cuatro años sobre el equipo de desarrollo. No hay ningún otro gasto de hardware: el servidor de Wazuh se desplegó mediante contenedores Docker en la misma máquina y las máquinas virtuales se ejecutaron con UTM, software libre.

| Elemento              | Adquisición | Vida útil | Duración | Coste imputable |
|-----------------------|-------------|-----------|----------|-----------------|
| MacBook Air M1 (2020) | 1.099,00 €  | 4 años    | 6 meses  | 137,38 €        |

Tabla 3.7: Coste de hardware (amortización).

#### Software y herramientas

El *stack* tecnológico principal (Wazuh, OpenSearch, Ollama, Llama 3.2, Python, Ubuntu y UTM) es software libre. Para la Prueba 4 (comparativa

multi-modelo) se emplearon las capas gratuitas (*free tier*) de las API de Gemini Flash (Google AI) y Groq, por lo que el coste de inferencia fue igualmente nulo. Este hecho refuerza una de las motivaciones del proyecto: demostrar que es posible construir una solución de análisis de alertas de nivel profesional sin costes de licencias ni de inferencia.

### Costes indirectos

Se incluye una estimación del consumo eléctrico incremental del equipo de desarrollo a lo largo de los seis meses del proyecto, resultando en un coste despreciable de 2,00 €.

### Resumen

La Tabla 3.8 consolida todos los conceptos anteriores en un presupuesto total del proyecto. El coste está dominado por la partida de personal, que representa más del 97 % del total; el resto corresponde a la amortización del equipo de desarrollo y a los costes indirectos de energía.

| Concepto                                                   | Coste             |
|------------------------------------------------------------|-------------------|
| Personal: desarrollador (300 h $\times$ 19,34 €/h)         | 5.802,00 €        |
| Personal: dirección TFG (estimación, 2 dir. $\times$ 15 h) | 810,00 €          |
| Hardware (amortización MacBook Air M1)                     | 137,38 €          |
| Software y herramientas (código abierto)                   | 0,00 €            |
| Costes indirectos (energía)                                | 2,00 €            |
| <b>Total</b>                                               | <b>6.751,38 €</b> |

Tabla 3.8: Resumen del presupuesto del proyecto.

## Capítulo 4

# Diseño e implementación

Este capítulo describe la solución desarrollada. En primer lugar se presenta la arquitectura general del sistema; a continuación se justifican las principales decisiones de diseño; y, por último, se detalla cada uno de los módulos implementados y la interfaz web.

### 4.1. Arquitectura general

El sistema se ha desarrollado como un *middleware*: una capa intermedia que se sitúa entre el ecosistema Wazuh y un modelo de lenguaje local, orquestando el flujo de información entre ambos. La arquitectura del entorno de pruebas se organiza en cuatro capas:

1. **Capa de *endpoints*.** Una máquina virtual Ubuntu que actúa como víctima, sobre la que se generan los vectores de ataque (intentos de acceso por SSH, escalada de privilegios mediante `sudo`) y en la que se ejecuta un agente Wazuh.
2. **Capa Wazuh.** El despliegue de Wazuh (Manager e Indexer) mediante contenedores Docker [20], que recibe los eventos del agente, genera las alertas y las indexa.
3. **Capa del *middleware*.** El componente desarrollado en este trabajo, que consulta las alertas, las procesa con el modelo de lenguaje y, en su caso, ordena acciones de respuesta.
4. **Capa de inteligencia artificial.** El servidor Ollama para la ejecución del modelo en local. El sistema permite además delegar las consultas a proveedores externos (Gemini de Google o Groq) cuando el analista lo selecciona explícitamente, tanto desde la interfaz web como desde la línea de comandos.

La Figura 4.9 muestra el diagrama de componentes completo con las interacciones entre zonas y puertos. El *middleware* interactúa con Wazuh a través de dos canales diferenciados, en lo que puede describirse como una arquitectura de **dobles pilares**: el **plano de datos**, mediante consultas al Wazuh Indexer (OpenSearch, puerto 9200) para recuperar y buscar alertas; y el **plano de gestión**, mediante la API REST del Manager (puerto 55000), autenticada con un token JWT, para ordenar acciones de respuesta. La comunicación con el modelo de lenguaje se realiza a través de la API REST que expone Ollama (puerto 11434).

## 4.2. Decisiones de diseño

El diseño del sistema se apoya en las siguientes cinco decisiones:

**1. Modelo de lenguaje local.** La elección de ejecutar el modelo en local mediante Ollama, en lugar de recurrir a una API comercial, responde al requisito de privacidad (RNF1). Las alertas de seguridad contienen información sensible sobre la infraestructura protegida (direcciones, nombres de equipos, usuarios, vulnerabilidades), cuya exposición a un tercero sería inaceptable en un entorno corporativo. Con el despliegue local, estos datos nunca salen de la red. El sistema permite seleccionar proveedores externos (Gemini [2], Groq [3]) como alternativa, tanto desde la interfaz web como desde la línea de comandos; cuando el analista activa esta opción, el procesamiento de las alertas pasa a realizarse en la infraestructura del proveedor elegido, con las implicaciones de privacidad correspondientes. La Tabla 4.1 recoge las características diferenciales de los tres proveedores.

| Característica     | Ollama<br>(Llama 3.2) | Gemini<br>Flash | Groq<br>(Llama 3.3-70B) |
|--------------------|-----------------------|-----------------|-------------------------|
| Parámetros         | 3 000 M               | n. a. (API)     | 70 000 M                |
| Inferencia local   | Sí                    | No              | No                      |
| Requiere internet  | No                    | Sí              | Sí                      |
| Privacidad         | Total (red local)     | Google Cloud    | Groq Cloud              |
| Latencia media     | 39,5 s                | 15,0 s          | 2,6 s                   |
| Coste por análisis | 0,0000 €              | 0,0003 €        | 0,0006 €                |

Tabla 4.1: Comparativa de los tres proveedores de inferencia soportados. La latencia recoge el promedio de las 15 ejecuciones de la Prueba 4 (Sección 5.6); el coste es el precio de lista por inferencia (el incurrido fue nulo al operar dentro del nivel gratuito de cada API).

**2. Supervisión humana en la respuesta activa.** El modelo de

**lenguaje nunca ejecuta ninguna acción por sí mismo.** En el módulo de respuesta activa, el LLM se limita a sugerir; es el *middleware* quien extrae las direcciones candidatas, y es un operador humano quien debe confirmar explícitamente antes de que se ejecute cualquier bloqueo. Este patrón, conocido como *Human-in-the-Loop*, replica la cautela observada en un SOC real, donde un analista de primera línea no aplica una contención sin investigar previamente, consciente de que un falso positivo podría interrumpir el servicio legítimo de un cliente. Además, este enfoque resuelve de forma arquitectónica (y no meramente mediante el ajuste del *prompt*) el riesgo de que el modelo proponga acciones incorrectas: aunque el modelo se equivoque, el sistema valida la acción y el ser humano tiene la última palabra.

**3. El LLM como traductor en el *threat hunting*.** En el módulo de consultas en lenguaje natural, el modelo no actúa como buscador, sino como **traductor**: convierte la pregunta del analista, formulada en español, en una consulta estructurada (Query DSL) que el *middleware* ejecuta directamente contra el Indexer. De este modo, la búsqueda real la realiza Wazuh (fiable y determinista), mientras que el modelo aporta únicamente la comodidad de la interfaz en lenguaje natural.

**4. Plantilla externa frente a *prompt* interno.** Los dos componentes que emplean un *prompt* de sistema adoptan estrategias distintas según su naturaleza. El módulo de análisis forense externaliza su plantilla en el fichero `prompts/mitre_prompt.txt`, que el operador puede editar sin tocar el código: el formato del informe es un parámetro de configuración. El módulo de *threat hunting*, en cambio, define su *prompt* como la constante `PROMPT_NL_A_DSL` dentro del propio módulo `threat_hunting.py`: este *prompt* fuerza al modelo a devolver exclusivamente JSON de OpenSearch, una restricción de protocolo interno que no tiene sentido exponer al operador y que, de modificarse accidentalmente, rompería el flujo de traducción.

**5. Interfaz web con Streamlit.** El acceso al sistema exclusivamente mediante línea de comandos limita su uso a analistas con experiencia en terminal y dificulta la exploración interactiva de alertas. Para cubrir el requisito RF6 se añadió una interfaz web con Streamlit [14], un marco Python que permite construir aplicaciones de datos interactivas sin código HTML ni JavaScript. Se prefirió Streamlit a alternativas orientadas a servicios (Flask, FastAPI) porque sus componentes nativos (tablas, barras de progreso, despleables, campos de texto, métricas) se corresponden directamente con los flujos del sistema: selección de alertas, lanzamiento del análisis y visualización de informes. La implementación se describe en la Sección 4.7.

### 4.3. Módulo orquestador

El módulo central coordina todo el proceso. Sus responsabilidades son: autenticarse contra la API del Manager para obtener el token JWT mediante `obtener_token()`; recuperar del Indexer las  $N$  alertas más recientes que superen un nivel de severidad dado mediante `obtener_alertas_del_indexer()`; construir, para cada alerta, el *prompt* a partir de una plantilla mediante `construir_prompt_mitre()`; enviarlo al modelo de lenguaje con `consultar_llm()`, que delega al backend seleccionado (Ollama local, Gemini o Groq), y recoger el informe validado; y, según el modo de ejecución seleccionado, derivar el control a los módulos de *threat hunting* o de respuesta activa. La Figura 4.10 muestra este flujo como diagrama de secuencia.

La interfaz de línea de comandos se implementa con la librería estándar `argparse`, que ofrece los parámetros `--alertas` (número de alertas a analizar, por defecto 1), `--nivel` (nivel mínimo de severidad, por defecto 5), `--proveedor` (proveedor LLM: `ollama`, `gemini` o `groq`; si no se indica, el sistema pregunta de forma interactiva), `--hunting` (modo de consultas en lenguaje natural) y `--respuesta-activa` (módulo de respuesta activa). El sistema incorpora además una gestión de errores específica que distingue entre la indisponibilidad de la conexión, los tiempos de espera agotados y otros fallos inesperados, ofreciendo un mensaje útil en cada caso en lugar de terminar con un error genérico.

Los cuatro modos de uso principales son:

`wazuh-ia --alertas 5 --nivel 5` Recupera las cinco alertas más recientes de nivel 5 o superior y genera, para cada una, un informe forense estructurado.

`wazuh-ia --proveedor gemini --alertas 3` Recupera las tres alertas más recientes y genera los informes usando Gemini Flash como proveedor de inferencia, sin necesidad de que Ollama esté en ejecución.

`wazuh-ia --hunting` Abre un bucle interactivo en el que el analista escribe consultas en lenguaje natural que el sistema traduce a Query DSL y ejecuta contra el Indexer.

`wazuh-ia --respuesta-activa` Extrae las IP candidatas de los informes generados y solicita confirmación explícita al operador antes de ordenar el bloqueo en el cortafuegos del *endpoint*.

El siguiente Listado 4.1 muestra el menú de ayuda completo generado por `argparse`:

Listado 4.1: Salida del menú de ayuda de la interfaz de línea de comandos (`wazuh-ia -h`), generada por `argparse`.

```
usage: wazuh-ia [-h] [--alertas N] [--nivel NIVEL]
               [--proveedor {ollama,gemini,groq}]
               [--hunting] [--respuesta-activa]

wazuh-ia: Middleware TFG para enriquecer alertas Wazuh con LLMs

options:
  -h, --help            show this help message and exit
  --alertas N            Numero de alertas a analizar (por defecto: 1)
  --nivel NIVEL          Nivel minimo de alerta a procesar (por defecto
                        : 5)
  --proveedor {ollama,gemini,groq}
                        Proveedor LLM (si no se indica, se pregunta
                        interactivamente)
  --hunting
    natural              Activa el modo de consultas en lenguaje
                        (Threat Hunting)
  --respuesta-activa     Activa el modulo de Respuesta Activa
                        (requiere confirmacion humana)
```

El Listado 4.2 muestra la función `obtener_token()`, que implementa la autenticación JWT contra la API de gestión del Manager (puerto 55000). La función realiza una petición GET con autenticación básica al *endpoint* `/security/user/authenticate`; si el Manager responde con código 200, extrae el token del campo `data.token` de la respuesta JSON y lo devuelve; si la conexión falla o el código HTTP es distinto, devuelve `None` sin propagar la excepción, de modo que el módulo pueda continuar en modo de solo lectura (sin capacidad de ordenar respuestas activas).

Listado 4.2: Función `obtener_token()` en `middleware.py`. La autenticación JWT contra la API del Manager (puerto 55000) se realiza con una petición GET usando credenciales básicas; el token resultante se usa en todas las llamadas posteriores a la API de gestión.

```
def obtener_token() -> str | None:
    url = f"{API_URL}/security/user/authenticate"
    try:
        r = requests.get(url, auth=(API_USER, API_PASS),
                        verify=False, timeout=10)
        if r.status_code == 200:
            return r.json().get('data', {}).get('token')
        print(f"[!] Login fallido. Status: {r.status_code}")
        return None
    except requests.exceptions.ConnectionError:
        print(f"[-] No se puede conectar a la API ({API_URL}).")
        return None
    except Exception as e:
        print(f"[-] Error inesperado en login: {e}")
        return None
```

## 4.4. Módulo de *threat hunting*

Este módulo implementa el objetivo de las consultas en lenguaje natural. Arranca un bucle interactivo en el que el analista escribe una pregunta en español (por ejemplo, “intentos de fuerza bruta SSH de las últimas seis horas”). El módulo envía esa pregunta al modelo de lenguaje acompañada de `PROMPT_NL_A_DSL`, una constante que le instruye para devolver **exclusivamente** una consulta en Query DSL de OpenSearch, sin texto adicional. La consulta generada se ejecuta con `ejecutar_query()` contra el Indexer y los resultados se presentan formateados en la terminal por `formatear_resultados()`.

El flujo del módulo se ilustra en la Figura 4.11. Un reto encontrado durante el desarrollo es que los modelos de lenguaje ejecutados en local ocasionalmente devuelven un JSON sintácticamente incompleto o con caracteres sobrantes tras el cierre final. Para mitigarlo se han implementado dos funciones: `reparar_json_truncado()`, que recorre la cadena carácter a carácter, contabiliza las llaves y los corchetes sin cerrar (ignorando los que aparecen dentro de cadenas de texto) y añade los cierres necesarios; y `extraer_primer_json()`, que localiza el primer objeto JSON balanceado y descarta cualquier carácter posterior al cierre final. Ambas se aplican en secuencia dentro de `nl_a_query_dsl()`, permitiendo recuperar consultas que, de otro modo, el intérprete rechazaría. El Capítulo 5 describe las dos variantes de JSON mal formado que motivaron su desarrollo.

El Listado 4.3 muestra la implementación de `reparar_json_truncado()`. La función mantiene una pila (`cierres`) donde cada apertura de objeto o array apila el cierre correspondiente; cuando encuentra un cierre real que coincide con el tope de la pila, lo extrae; al terminar el recorrido, los elementos restantes en la pila son los cierres ausentes, que se añaden al texto en orden inverso. El indicador `dentro_de_string` evita que las llaves que aparecen dentro de valores de cadena alteren la pila.

## 4.5. Módulo de respuesta activa

Este módulo materializa el patrón *Human-in-the-Loop* descrito anteriormente; la Figura 4.12 detalla el flujo completo. Su funcionamiento es el siguiente: a partir del informe generado por el modelo, `extraer_ips_del_informe()` extrae mediante expresiones regulares las direcciones IP candidatas a bloqueo, descartando las que figuran en `LISTA_BLANCA_IPS` (entre ellas, la dirección del propio Manager y las direcciones de *loopback* 127.0.0.1 y ::1); por cada dirección candidata, `solicitar_confirmacion()` muestra en la consola la acción exacta que se va a realizar y espera que el operador escriba la palabra CONFIRMAR; solo si la confirmación es explícita, `bloquear_ip_en_agente()`

Listado 4.3: Función `reparar_json_truncado()` en `threat_hunting.py`. Una pila acumula los cierres esperados; al finalizar el recorrido, los que no se han cancelado se añaden al texto en orden inverso, completando el JSON truncado.

```
def reparar_json_truncado(texto: str) -> str:
    cierres = []
    dentro_de_string = False
    escape = False
    for char in texto:
        if escape:
            escape = False; continue
        if char == '\\':
            escape = True; continue
        if char == '"' and not escape:
            dentro_de_string = not dentro_de_string; continue
        if dentro_de_string:
            continue
        if char == '{':
            cierres.append('}')
        elif char == '[':
            cierres.append(']')
        elif char in ('}', '']):
            if cierres and cierres[-1] == char:
                cierres.pop()
    return texto + ''.join(reversed(cierres))
```

invoca el *endpoint* de respuesta activa de la API de Wazuh con el comando `firewall-drop0`, que añade una regla `iptables` de tipo `DROP` para la dirección en el cortafuegos del *endpoint*. Si el token de la API no está disponible, el módulo opera en modo simulación, mostrando la acción que ejecutaría sin llegar a realizarla.

El Listado 4.4 muestra la implementación de `extraer_ips_del_informe()`. La función aplica una expresión regular de IPv4 estándar sobre el texto completo del informe; a continuación filtra los resultados descartando duplicados, las direcciones de la lista blanca y las IPs con octetos fuera del rango válido (octetos con valor 0 o 255). El filtrado de rango (`0 <int(o) <255`) descarta las redes reservadas más frecuentes (0.0.0.0, 255.255.255.255) sin necesitar una lista exhaustiva. Adicionalmente, `procesar_respuesta_activa()` comprueba el campo `data.srcip` de la alerta original e inserta esa dirección al inicio de la lista de candidatas si no estaba ya presente, priorizando la IP del origen del evento sobre cualquier otra mencionada en el texto del informe.

La validación actúa en dos capas independientes. En la primera, `validar_respuesta_ia()` inspecciona la sección de respuesta del informe y añade un aviso visible si el modelo hubiera sugerido bloquear una dirección protegida. En la segunda, `bloquear_ip_en_agente()` comprueba de nuevo la lista blanca antes de llamar a la API, de modo que un eventual error en la primera capa no llegue a ejecutarse.

Listado 4.4: Función `extraer_ips_del_informe()` en `respuesta_activa.py`. La expresión regular extrae todas las IPv4 del informe; el filtrado posterior descarta duplicados, direcciones de la lista blanca y octetos fuera del rango válido.

```
def extraer_ips_del_informe(texto_informe: str) -> list[str]:
    patron_ipv4 = r'\b(?:\d{1,3}\.){3}\d{1,3}\b'
    todas_las_ips = re.findall(patron_ipv4, texto_informe)
    ips_candidatas = []
    vistas = set()
    for ip in todas_las_ips:
        if ip not in vistas and ip not in LISTA_BLANCA_IPS:
            octetos = ip.split('.')
            if all(0 < int(o) < 255 for o in octetos):
                ips_candidatas.append(ip)
                vistas.add(ip)
    return ips_candidatas
```

La respuesta activa requiere un bloque `<active-response>` en el `ossec.conf` del Manager que declare `<command>firewall-drop</command>`. Dado que el disparo se realiza por llamada programática a la API (no de forma automática por regla), dicha configuración no requiere los campos `rules.id` ni `level`. A partir de ese bloque, Wazuh genera automáticamente la entrada `firewall-drop0` en `etc/shared/ar.conf` (sufijo 0 = bloqueo permanente sin expiración), nombre que la API valida en el `payload` de `bloquear_ip_en_agente()`. El bloque completo figura en el Listado 5.2 del Capítulo 5.

## 4.6. Ingeniería del *prompt*

La calidad de los informes depende sobre todo del diseño del *prompt*. La plantilla de análisis forense (almacenada en `prompts/mitre_prompt.txt` e incluida íntegramente en el Apéndice A) instruye al modelo para que adopte el rol de un analista senior de SOC nivel 3 y estructura su respuesta en cinco secciones fijas cuyo orden responde a la lógica operativa de un SOC.

La primera sección, **resumen ejecutivo y *Kill Chain***, pide al modelo que identifique la fase del ciclo de vida del ataque según el marco de Lockheed Martin y resuma el incidente en tres o cuatro líneas orientadas a un lector directivo no técnico. Esta sección encabeza el informe porque el analista que recibe el informe necesita ante todo situar la alerta dentro del avance global de la intrusión antes de profundizar en los detalles técnicos. La segunda sección, **análisis forense detallado**, es la de mayor extensión: el modelo extrae del *raw log* los indicadores de compromiso (direcciones IP, usuarios, comandos, puertos) y los correlaciona con la técnica MITRE ATT&CK

correspondiente, aportando el contexto que convierte un registro bruto en evidencia interpretable. La tercera sección, **evaluación del impacto**, traslada el análisis técnico al ámbito del riesgo para la organización: qué activos están expuestos y cuáles son los objetivos probables del adversario. La cuarta sección, **plan de respuesta activa**, limita al modelo a un máximo de tres acciones priorizadas y le prohíbe proponer comandos que no existan en la plataforma; de este modo, el plan es accionable directamente por el módulo de respuesta activa sin que el analista deba filtrar recomendaciones inviables. La quinta sección, **anomalías detectadas**, recoge indicios que no encajan con el patrón principal del ataque y que podrían señalar un vector secundario o una actividad de reconocimiento no capturada por la regla principal.

El *prompt* incorpora además restricciones explícitas de veracidad, introducidas de forma iterativa tras observar que el modelo tendía a proponer acciones técnicamente incorrectas en las primeras versiones del sistema. Las restricciones establecen que el único comando válido de bloqueo es **firewall-drop** (la respuesta activa nativa de Wazuh), que el modelo no debe inventar comandos que no existen en la plataforma, y que no debe proponer el bloqueo de direcciones de *loopback* ni de redes privadas de gestión. Se le exige asimismo utilizar las fases oficiales de la *Cyber Kill Chain* de Lockheed Martin sin mezclarlas con las tácticas de MITRE ATT&CK, que responden a una taxonomía distinta. Estas restricciones constituyen una primera barrera de contención que se complementa, en el plano arquitectónico, con la validación automática de la función `validar_respuesta_ia()` y con la confirmación humana obligatoria antes de cualquier bloqueo.

El sistema utiliza dos *prompts* de naturaleza diferente que adoptan estrategias de almacenamiento distintas según su función. El de análisis forense (`mitre_prompt.txt`) se externaliza en un fichero de configuración editable por el operador, porque el formato del informe (el número de secciones, su denominación, el nivel de detalle pedido) es un parámetro de presentación que puede necesitar adaptación a cada entorno sin tocar el código. El de *threat hunting* (`PROMPT_NL_A_DSL`, constante interna de `threat_hunting.py`) se mantiene embebido en el código porque su función es una restricción de protocolo: fuerza al modelo a devolver exclusivamente un objeto JSON de OpenSearch, y modificarlo accidentalmente rompería el flujo de traducción. Ambos *prompts* se recogen en el Apéndice A para facilitar su reproducción y adaptación en otros entornos.

## 4.7. Interfaz web

El módulo de línea de comandos cubre los cinco objetivos del sistema, pero su operación desde la terminal reduce la accesibilidad para analistas

sin experiencia en consola y dificulta la exploración interactiva de alertas. La interfaz web, descrita en la decisión de diseño 5, resuelve este punto. Se trata de una aplicación Streamlit multipage: `Home.py` actúa como punto de entrada y el directorio `pages/` alberga las páginas secundarias.

#### 4.7.1. Arquitectura de la aplicación

La aplicación no sigue el modelo cliente-servidor habitual con un *frontend* independiente (React, Angular) y un *backend* separado (Flask, FastAPI). Streamlit unifica ambas capas en un único proceso Python: el mismo código que consulta la API de Wazuh, invoca el modelo de lenguaje y procesa los informes también genera la interfaz que el navegador renderiza. El navegador actúa como capa de presentación pura, sin lógica propia.

En cuanto a la persistencia, la aplicación no incorpora una base de datos propia: delega toda la indexación y recuperación de eventos en el Wazuh Indexer (OpenSearch, puerto 9200), que ya almacena las alertas como documentos JSON. El estado de la sesión (alertas seleccionadas, informes generados, proveedor activo) se mantiene en memoria mediante `st.session_state` durante el tiempo que la pestaña permanece abierta.

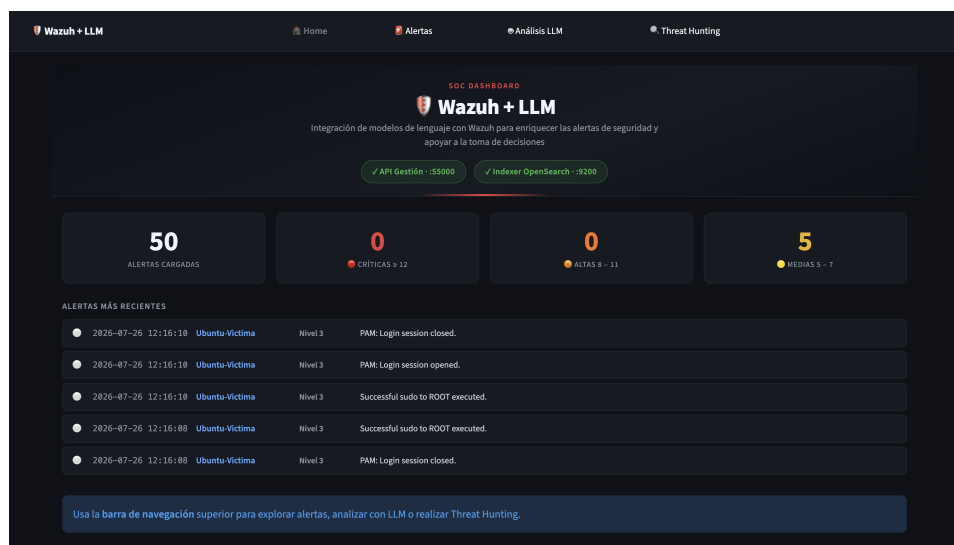


Figura 4.1: Página de inicio de la interfaz web. Los indicadores de estado reflejan la conectividad con el Manager (puerto 55000) y el Indexer (puerto 9200); el panel inferior presenta el recuento de alertas por nivel de severidad y el feed de los eventos más recientes con su marca de tiempo y agente.

### 4.7.2. Páginas de la aplicación

La aplicación se organiza en cuatro páginas:

**Home** (`Home.py`). Dashboard de estado (Figura 4.1): comprueba la conectividad con la API de gestión del Manager (puerto 55000) y con el Indexer (puerto 9200), muestra el recuento de alertas desglosado por nivel de severidad (críticas  $\geq 12$ , altas 8–11, medias 5–7) y presenta un feed de los cinco eventos más recientes con marca de tiempo, agente y descripción de la regla.

**Alertas** (`pages/1.Alertas.py`). Explorador de alertas (Figura 4.2) con controles de filtrado por número de alertas y nivel mínimo de severidad. Cada fila incluye los campos MITRE ATT&CK etiquetados por Wazuh (identificador de técnica y táctica) y la IP de origen. Una sección de detalle muestra los metadatos estructurados y el JSON completo de la alerta seleccionada.

**Análisis LLM** (`pages/2.Analisis.LLM.py`). Página central del sistema (Figuras 4.3 y 4.4): permite seleccionar el proveedor LLM (Ollama local con Llama 3.2, Gemini o Groq), elegir las alertas a analizar (últimas  $N$  o selección manual por multiselect) y lanzar el análisis. Los informes generados aparecen en despleables que muestran el tiempo de inferencia y el modelo empleado. Si el analista activa la casilla de **Respuesta Activa**, la página muestra tras cada informe las IPs candidatas a bloqueo y el formulario de confirmación explícita, replicando el patrón *Human-in-the-Loop* del módulo de línea de comandos.

**Threat Hunting** (`pages/3.Threat_Hunting.py`). Campo de texto libre para consultas en lenguaje natural (Figura 4.5), con cinco ejemplos predefinidos que se insertan al pulsarlos. Muestra la Query DSL generada en un desplegable y los resultados de la búsqueda en una tabla con los mismos campos que la página de Alertas.

**Wazuh • LLM** Home Alertas Análisis LLM Threat Hunting

## Explorador de Alertas

Visualización y filtrado de alertas recientes del Indexer de Wazuh

Número de alertas: 3 Nivel mínimo de severidad: 10 Actualizar

3 alerta(s) cargada(s) con nivel ≥ 10

| Timestamp           | Agente         | Nivel | Descripción                                                     | IP Origen    | MITRE ID | Táctica           |
|---------------------|----------------|-------|-----------------------------------------------------------------|--------------|----------|-------------------|
| 2026-07-17 09:08:03 | Ubuntu-Victima | 10    | sshd: brute force trying to get access to the system. Non exist | 192.168.64.1 | T1110    | Credential Access |
| 2026-07-17 07:46:24 | Ubuntu-Victima | 10    | sshd: brute force trying to get access to the system. Non exist | 192.168.64.1 | T1110    | Credential Access |
| 2026-07-17 07:43:01 | Ubuntu-Victima | 11    | Possible kernel level rootkit                                   | —            | T1014    | Defense Evasion   |

### Detalle de alerta

Selecciona una alerta para ver el detalle

1. 2026-07-17 09:08:03 | Ubuntu-Victima | Nivel 10 — sshd: brute force trying to get access to the system. Non exist

Agente: Ubuntu-Victima MITRE ID: T1110

Regla ID: 5712 Táctica: Credential Access

Descripción: sshd: brute force trying to get access to the system. Non existent user. IP Origen: 192.168.64.1

Nivel: 10 Timestamp: 2026-07-17 09:08:03

Raw Log

Jul 17 09:08:01 ubuntu sshd[7472]: invalid user usuariofake from 192.168.64.1 port 52337

> Ver JSON completo de la alerta

Figura 4.2: Explorador de alertas de la interfaz web. La tabla superior muestra las alertas recuperadas del Indexer con sus campos MITRE ATT&CK (técnica y táctica) y la IP de origen; los controles permiten ajustar el número de alertas y el nivel mínimo de severidad. El panel inferior presenta el detalle estructurado de la alerta seleccionada: metadatos, registro en bruto (*raw log*) y opción de ver el JSON completo.

**Wazuh • LLM** Home Alertas Análisis LLM Threat Hunting

## ☹ Análisis de Alertas con LLM

El LLM genera informes de triaje estructurados con correlación MITRE ATT&CK

### 1 • Proveedor LLM

Ollama  
llama3.2 · local

**Gemini**  
gemini-flash-latest · nube  
✓ seleccionado

Groq  
llama3.3-70b-versatile · nube open-source

### 2 • Alertas a analizar

Modo de selección

☒ Últimas N alertas ☐ Selección manual

Nivel mínimo de severidad

Número de alertas

1 10

### 3 • Análisis

☒ Activar sugerencia de Respuesta Activa ⓘ

Analizar alertas Limpiar resultados

Figura 4.3: Página de análisis LLM: sección de configuración. Los tres botones permiten seleccionar el proveedor activo (Ollama local, Gemini o Groq); el proveedor seleccionado se resalta con borde y fondo diferenciados. Los controles inferiores permiten elegir el modo de selección de alertas y el nivel mínimo de severidad.

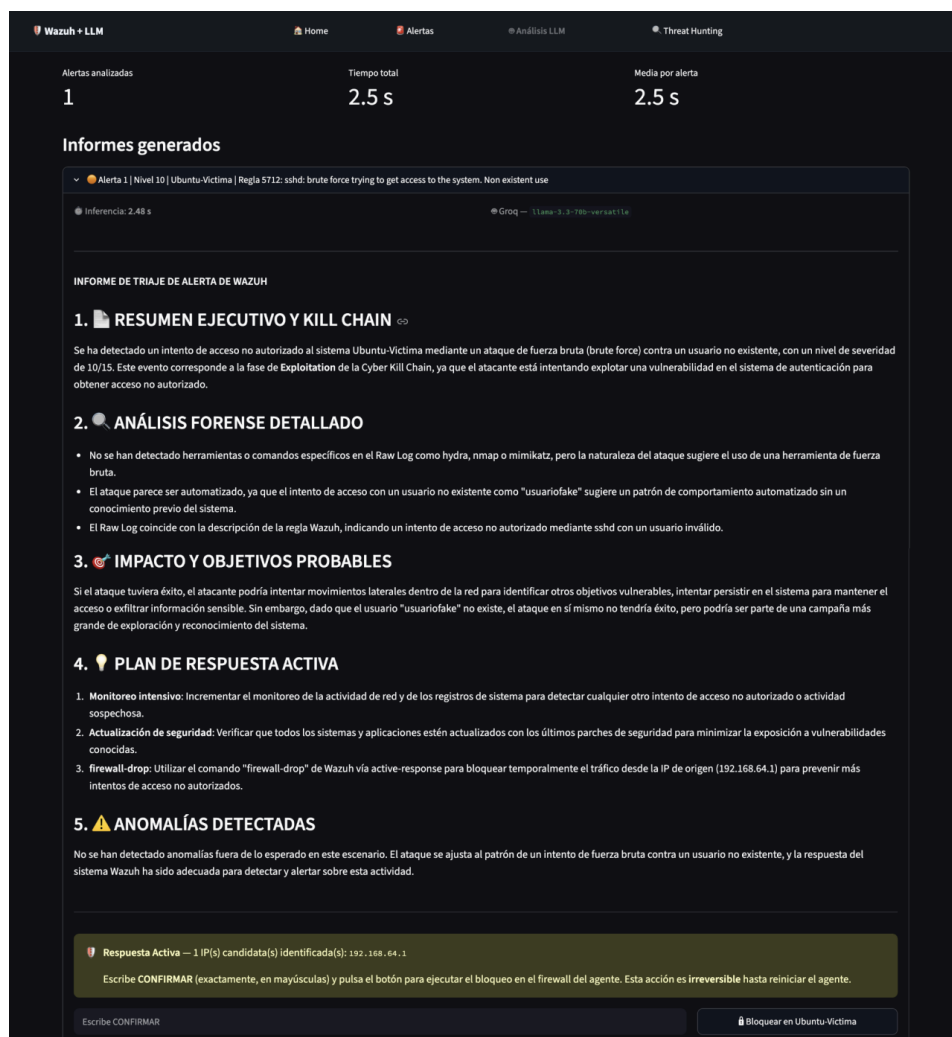


Figura 4.4: Página de análisis LLM: resultados. Tras ejecutar el análisis, la página muestra las métricas de inferencia (alertas procesadas, tiempo total y media por alerta) y los informes generados en desplegados que incluyen el tiempo de inferencia, el modelo empleado y el texto estructurado con correlación MITRE ATT&CK.

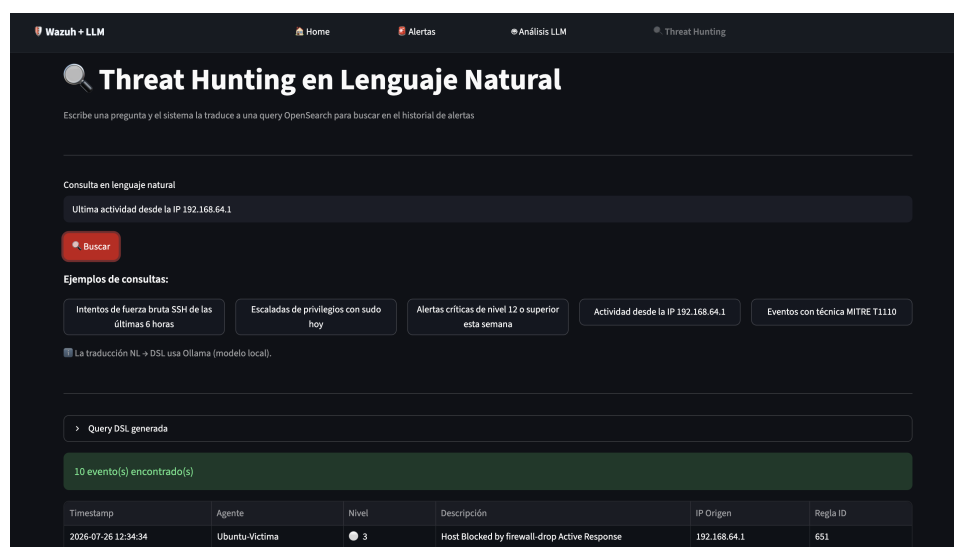


Figura 4.5: Página de *Threat Hunting* de la interfaz web. Tras introducir una consulta en lenguaje natural, el sistema muestra la Query DSL generada en un desplegable y presenta los resultados en una tabla con los campos de agente, nivel, descripción e IP de origen.

#### 4.7.3. Respuesta activa en la interfaz web

La implementación de la respuesta activa en la interfaz web sigue la misma cadena de comprobaciones que la versión de línea de comandos. El flujo consta de dos pasos. En el primero, antes de lanzar el análisis, el analista activa la casilla *Activar sugerencia de Respuesta Activa*; sin esta confirmación previa, la sección de bloqueo no aparece aunque el informe contenga IPs candidatas. En el segundo paso, tras la generación del informe, si la casilla está marcada, `extraer_ips_del_informe()` extrae las IPs candidatas y las complementa con la dirección `data.srcip` de la alerta; el sistema muestra un aviso con las direcciones identificadas, un campo de texto libre y el botón de bloqueo. Pulsar el botón sin haber escrito exactamente **CONFIRMAR** (en mayúsculas) no ejecuta ninguna acción y muestra un mensaje de error; solo la cadena exacta desencadena la llamada a `bloquear_ip_en_agente()` con el token JWT obtenido en ese instante. Las dos capas de validación (lista blanca en `extraer_ips_del_informe()` y comprobación doble en `bloquear_ip_en_agente()`) permanecen activas con independencia de la interfaz usada.

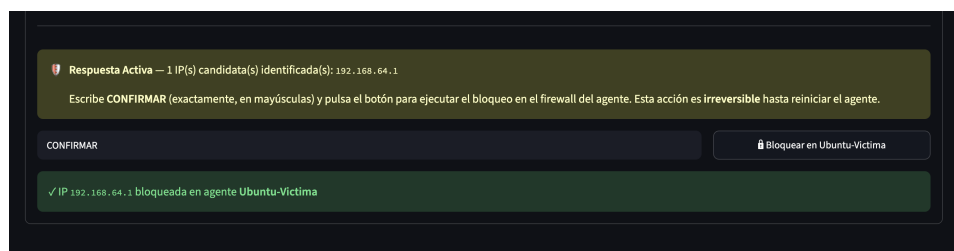


Figura 4.6: Flujo completo de la respuesta activa en la interfaz web. El aviso superior identifica 192.168.64.1 como IP candidata; el analista ha introducido la cadena exacta **CONFIRMAR** y pulsado el botón de bloqueo; el banner verde inferior confirma que la regla DROP fue aplicada sobre el agente Ubuntu-Victima a través de la API de Wazuh.

## 4.8. Pruebas unitarias

Las pruebas se agrupan en `tests/test_middleware.py` y se ejecutan con `pytest tests/ -v` desde la raíz del repositorio, sin necesidad de conexión activa a Wazuh ni a ningún proveedor LLM. La suite cubre tres clases de equivalencia principales: la reparación de JSON mal formado, la lógica de validación de la salvaguarda de seguridad, y la robustez de la recuperación de alertas ante distintos estados del Indexer. Las tres funciones seleccionadas (`reparar_json_truncado()`, `validar_respuesta_ia()` y `obtener_alertas_del_indexer()`) contienen lógica condicional propia verificable de forma aislada. La función `obtener_token()`, cuya única responsabilidad es autenticarse contra el Manager, no contiene lógica separable de esa llamada de red y queda fuera del alcance de las pruebas unitarias; su comportamiento se valida implícitamente en las pruebas de integración del Capítulo 5.

### TestRepararJsonTruncado

La clase `TestRepararJsonTruncado` verifica la función pura `reparar_json_truncado()` mediante cinco casos frontera que cubren los escenarios relevantes de uso. El caso `test_json_completo_no_modifica` establece el contrato básico: una cadena ya válida no debe modificarse, verificando que la función es idempotente para las entradas correctas. El caso `test_cierra_llave_faltante` comprueba la reparación de un JSON truncado a un nivel de anidamiento, pero en lugar de comparar cadenas verifica el resultado mediante `json.loads()`: lo que importa no es el contenido exacto de los cierres añadidos, sino que el resultado sea un JSON que cualquier intérpre-

te acepte. El caso `test_cierra_multiples_niveles` garantiza que la pila de cierres funciona correctamente con dos niveles de anidamiento simultáneamente abiertos, verificando que los cierres se añaden en el orden inverso al de apertura. El caso `test_no_afecta_llaves_dentro_de_string` es el más crítico para la correcta implementación del indicador `dentro_de_string`: una llave `{` que aparece como carácter dentro del valor de una clave no debe incrementar la profundidad de la pila, pues de lo contrario la función añadiría un cierre sobrante a un JSON que ya es válido. El quinto caso verifica el comportamiento ante la entrada vacía. Al ser una función pura sin llamadas externas ni estado, la clase no requiere ningún objeto simulado.

| Caso de prueba                                      | Comportamiento verificado                                                  |
|-----------------------------------------------------|----------------------------------------------------------------------------|
| <code>test_json_completo_no_modifica</code>         | JSON sin trunca → devuelto sin cambios                                     |
| <code>test_cierra_llave_faltante</code>             | JSON truncado a 1 nivel → resultado válido según <code>json.loads()</code> |
| <code>test_cierra_multiples_niveles</code>          | 2 niveles sin cerrar → cierres añadidos en orden inverso correcto          |
| <code>test_no_afecta_llaves_dentro_de_string</code> | Llaves dentro de valores string → no contabilizadas como delimitadores     |
| <code>test_json_vacio</code>                        | Cadena vacía → cadena vacía                                                |

Tabla 4.2: Casos de prueba de `TestRepararJsonTruncado`.

## TestGuardrailSeguridad

La clase `TestGuardrailSeguridad` comprueba que `validar_respuesta_ia()` añada el aviso "GUARDRAIL ACTIVADO" únicamente cuando concurren dos condiciones de forma simultánea: una dirección de `LISTA.BLANCA_IPS` aparece en la sección de respuesta activa del informe *y* va acompañada de un comando de bloqueo reconocido (`iptables`, `ufw deny`, etc.). Los dos primeros casos verifican las variantes más frecuentes de activación (IPv4 de *loopback* con `iptables` e IPv6 con `ufw deny`) para garantizar que ambos formatos de IP y ambas herramientas de cortafuegos se interceptan correctamente.

El tercer caso verifica el lado negativo de la condición de IP: cuando el informe contiene un comando de bloqueo que afecta a una dirección externa, la salvaguarda *no* debe activarse. Este caso es necesario para demostrar que el guardrail es discriminante respecto a la IP y no rechaza indiscriminadamente cualquier informe que mencione comandos de cortafuegos. El cuarto caso es el más significativo desde el punto de vista operativo: cuando `127.0.0.1` aparece en la sección de resumen del incidente (porque la alerta original la menciona en el *full\_log* como dirección de destino) pero no hay ninguna sección de respuesta activa con comandos de bloqueo, la salvaguarda no debe activarse. La función localiza el primer marcador de sección de respuesta (PLAN DE RESPUESTA, RESPUESTA ACTIVA, etc.) y limita su análisis al texto posterior; si no encuentra ninguno, analiza el informe completo pero sin ninguna IP candidata a bloquear, resultado que coincide con el caso de la

sección ausente. Sin este comportamiento, cualquier mención de la IP de *loopback* en el registro bruto original bloquearía la acción aunque el modelo no la hubiera propuesto como objetivo.

| Caso de prueba                                                      | Comportamiento verificado                                   |
|---------------------------------------------------------------------|-------------------------------------------------------------|
| <code>test_ip_loopback.en.plan.respuesta.activa.guardrail</code>    | 127.0.0.1 + iptables en sección respuesta → aviso           |
| <code>test_ip_loopback.ipv6.activa.guardrail</code>                 | ::1 + ufw deny → aviso                                      |
| <code>test_ip_externa_legitima.no.activa.guardrail</code>           | IP no protegida + bloqueo → sin aviso                       |
| <code>test_informe.sin.seccion.respuesta.no.activa.guardrail</code> | IP protegida en sección resumen (sin respuesta) → sin aviso |

Tabla 4.3: Casos de prueba de `TestGuardrailSeguridad`.

### `TestObtenerAlertasDelIndexer`

La clase `TestObtenerAlertasDelIndexer` aísla la llamada HTTP de la función `obtener_alertas_del_indexer()` mediante el decorador `@patch("wazuh_llm.middleware.requests.post")`, que intercepta las llamadas a `requests.post` antes de que lleguen a la red y las sustituye por un `MagicMock` configurable. La clase define el método auxiliar `_mock_indexer_ok()` que construye una respuesta simulada con código 200 y el formato JSON que devuelve el Indexer, incluyendo la estructura `{"hits": {"hits": [{"_source": ...}]}}` que `obtener_alertas_del_indexer()` debe parsear para extraer los datos de cada alerta.

El primer caso verifica que la función extrae correctamente los campos `_source` de cada *hit* y los devuelve como diccionarios Python con su estructura original, comprobando que el campo `rule.id` del *hit* simulado aparece en el resultado. Los tres casos restantes cubren las condiciones de error: respuesta vacía (sin alertas en el índice), respuesta con código HTTP 401 (credenciales incorrectas o token caducado), y excepción `ConnectionError` (Indexer inalcanzable). En los tres casos se verifica que la función devuelve una lista vacía en lugar de propagar la excepción o devolver `None`, garantizando que el módulo orquestador puede operar de forma controlada ante cualquier estado del Indexer.

| Caso de prueba                                                  | Comportamiento verificado                                                  |
|-----------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>test_parsea_correctamente.hits</code>                     | Respuesta 200 con 1 <i>hit</i> → extrae <code>_source</code> correctamente |
| <code>test_devuelve_lista_vacia_si_no_hay_alertas</code>        | Respuesta 200 con <i>hits</i> vacíos → lista vacía                         |
| <code>test_devuelve_lista_vacia_si_status_no_200</code>         | Código HTTP 401 → lista vacía                                              |
| <code>test_devuelve_lista_vacia_si_indexer_no_disponible</code> | <code>ConnectionError</code> → lista vacía                                 |

Tabla 4.4: Casos de prueba de `TestObtenerAlertasDelIndexer`.

## 4.9. Guía de instalación y uso

Esta sección describe el proceso de instalación y puesta en marcha del sistema en un entorno de pruebas equivalente al utilizado durante el desarrollo del proyecto.

### 4.9.1. Requisitos del sistema

Para ejecutar el *middleware* es necesario disponer de los siguientes componentes:

- **Sistema operativo:** Linux, macOS o Windows con WSL2. El entorno de pruebas del proyecto usa macOS Apple Silicon como anfitrión y máquinas virtuales ARM64 bajo UTM.
- **Python 3.10** o superior.
- **Wazuh 4.9 o superior** accesible en red, con el Indexer (OpenSearch) en el puerto 9200 y la API de Gestión en el puerto 55000.
- Al menos uno de los siguientes motores de inferencia:
  - **Ollama** con el modelo **llama3.2** descargado (inferencia completamente local, sin conexión a internet).
  - Clave de API de **Google AI** para Gemini Flash (variable de entorno **GEMINI\_API\_KEY**).
  - Clave de API de **Groq** para Llama 3.3-70B (variable de entorno **GROQ\_API\_KEY**).

### 4.9.2. Despliegue de la infraestructura de pruebas

#### Wazuh en Docker (nodo único)

La forma más directa de levantar Wazuh es a través de la imagen oficial de Docker en configuración de nodo único:

```
git clone https://github.com/wazuh/wazuh-docker.git -b 4.9.0
cd wazuh-docker/single-node
docker compose -f generate-indexer-certs.yml run --rm generator
docker compose up -d
```

Tras el arranque, el panel web de Wazuh queda accesible en <https://localhost:443> con las credenciales **admin** / **SecretPassword** (cambiar en producción). El Indexer responde en el puerto 9200 y la API de Gestión en el 55000.

## Ollama en macOS con Apple Silicon

Ollama gestiona los modelos de lenguaje en local y expone una API REST en el puerto 11434. La instalación en macOS se realiza con Homebrew:

```
brew install ollama
brew services start ollama
ollama pull llama3.2
```

El modelo llama3.2 (3B parámetros) ocupa aproximadamente 2 GB en disco. Una vez descargado, el servicio atiende peticiones de inferencia sin conexión a internet.

## Agente Wazuh en la máquina monitorizada

Para registrar una máquina Ubuntu ARM64 en el Manager (sustituyendo <IP\_MANAGER> por la dirección real):

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/
wazuh-agent_4.9.0-1_arm64.deb
sudo WAZUH_MANAGER='<IP_MANAGER>' \
WAZUH_AGENT_NAME='Ubuntu-Victima' \
dpkg -i ./wazuh-agent_4.9.0-1_arm64.deb
sudo systemctl enable --now wazuh-agent
```

## Habilitación de la respuesta activa

Para que el módulo de bloqueo de IPs pueda invocar `firewall-drop` mediante la API, el fichero `ossec.conf` del Manager debe incluir el siguiente bloque dentro de `<ossec_config>`:

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
</active-response>
```

Tras añadirlo, reiniciar el Manager:

```
docker compose restart wazuh.manager
```

Wazuh genera automáticamente la entrada `firewall-drop0` en `etc/shared/ar.conf` (sufijo 0 indica bloqueo permanente sin expiración). El *middleware* usa ese nombre exacto en el *payload* de la llamada a la API. Sin este bloque la API devuelve error 404 al intentar ejecutar un bloqueo.

### 4.9.3. Instalación del *middleware*

```
git clone https://github.com/maximartinm/TFG-Wazuh-LLM.git
cd TFG-Wazuh-LLM
python3 -m venv venv
source venv/bin/activate          # Linux / macOS
pip install -e .
```

La instalación en modo editable (`-e`) registra el comando `wazuh-ia` en el entorno virtual y permite modificar el código fuente sin necesidad de reinstalar el paquete. Para verificar que la instalación es correcta:

```
wazuh-ia --help
```

### 4.9.4. Configuración

El *middleware* se configura mediante las variables de entorno recogidas en la Tabla 4.5, que se pueden definir en un fichero `.env` en la raíz del repositorio. Copiar la plantilla incluida y editar los valores:

```
cp .env.example .env
```

| Variable        | Descripción                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------|
| WZ_API_URL      | URL de la API de Gestión (p. ej., <code>https://localhost:55000</code> )                            |
| WZ_API_USER     | Usuario de la API de Gestión                                                                        |
| WZ_API_PASS     | Contraseña de la API de Gestión                                                                     |
| WZ_INDEXER_URL  | URL del Indexer con patrón de índice ( <code>https://localhost:9200/wazuh-alerts-*/_search</code> ) |
| WZ_INDEXER_USER | Usuario del Indexer ( <code>admin</code> por defecto)                                               |
| WZ_INDEXER_PASS | Contraseña del Indexer                                                                              |
| WZ_MANAGER_IP   | IP del Manager; se añade automáticamente a la lista blanca de bloqueos                              |
| WZ_OLLAMA_URL   | URL de la API de Ollama ( <code>http://localhost:11434/api/generate</code> )                        |
| WZ_MODELO       | Nombre del modelo Ollama ( <code>llama3.2</code> )                                                  |
| GEMINI_API_KEY  | Clave de API de Google AI para Gemini Flash (opcional)                                              |
| GROQ_API_KEY    | Clave de API de Groq para Llama 3.3-70B (opcional)                                                  |
| WZ_PROMPT_PATH  | Ruta a la plantilla de <i>prompt</i> ( <code>prompts/mitre.prompt.txt</code> )                      |

Tabla 4.5: Variables de configuración del *middleware*.

#### 4.9.5. Uso de la interfaz web

La interfaz web es la vía de acceso recomendada para el uso interactivo del sistema. Para iniciarla, con el entorno virtual activado:

```
streamlit run Home.py
```

Streamlit abre automáticamente el navegador en `http://localhost:8501`. La interfaz organiza las funciones en cuatro páginas accesibles desde la barra lateral:

- **Inicio.** Estado de las conexiones (API de Gestión e Indexer) y *feed* de alertas recientes.
- **Alertas.** Tabla filtrable por nivel de severidad con el detalle completo de cada evento y su JSON original.
- **Análisis con LLM.** Selección del proveedor de inferencia (Ollama, Gemini Flash o Groq), configuración de alertas a analizar e informe generado con métricas de tiempo. Incorpora la solicitud de respuesta activa con confirmación explícita cuando el informe identifica IPs candidatas.
- **Threat Hunting.** Entrada de consultas en lenguaje natural y visualización de resultados en tabla, con la Query DSL generada disponible en un desplegable.

#### 4.9.6. Uso por línea de comandos

Para entornos sin navegador o para su integración en *scripts*, el comando `wazuh-ia` expone las mismas funciones desde el terminal:

```
wazuh-ia                                # Última alerta crítica
wazuh-ia --alertas 5                     # 5 alertas más recientes
wazuh-ia --alertas 10 --nivel 7          # 10 alertas de nivel >= 7
wazuh-ia --alertas 1 --respuesta-activa  # Con respuesta activa
wazuh-ia --hunting                       # Threat hunting interactivo
```

En el modo de respuesta activa, el sistema presenta al analista cada IP candidata extraída del informe y espera la confirmación. El operador debe escribir **CONFIRMAR** para ejecutar el bloqueo; cualquier otra entrada cancela la acción. La lista blanca protege siempre `127.0.0.1`, `::1` y la IP del Manager.

## 4.10. Despliegue en producción: arquitectura Kubernetes

Hasta este punto, todos los módulos descritos en el capítulo (el orquestador, el módulo de respuesta activa, el módulo de *threat hunting* y la interfaz web) se ejecutan sobre un despliegue local: la interfaz web arranca con `streamlit run Home.py` y la CLI con `wazuh-ia`, ambas directamente sobre el sistema operativo del analista. Este modo es suficiente para el desarrollo y la experimentación, y es el entorno utilizado en las cuatro pruebas del Capítulo 5. Sin embargo, un despliegue orientado a producción requiere garantías adicionales: declaratividad de la configuración, aislamiento de la aplicación, separación entre código y secretos, y trazabilidad del ciclo de vida del contenedor. Esta sección describe el despliegue alternativo del *middleware* en un clúster local de Kubernetes, disponible en la rama `feat/k8s-deploy` del repositorio [8].

### 4.10.1. Motivación y alcance

El despliegue en Kubernetes persigue tres objetivos concretos respecto al modo local de desarrollo.

**Declaratividad.** El estado deseado del sistema queda descrito en manifiestos YAML versionados en git. Cualquier cambio en la configuración es rastreable y reversible, y el operador puede reconstruir el despliegue completo desde cero con un único comando (`kubectl apply -k k8s/`), sin dependencias implícitas del entorno.

**Aislamiento y endurecimiento.** El contenedor se ejecuta con un usuario no raíz (UID 10001), sistema de ficheros raíz en modo solo lectura y sin capacidades de Linux adicionales. Estas restricciones, estándar en despliegues de producción, no son posibles en la ejecución directa sobre el sistema operativo del analista.

**Separación de configuración y secretos.** En el modo local, las credenciales de Wazuh y las claves de API residen en un fichero `.env` que convive con el código. En el despliegue Kubernetes, la configuración no sensible reside en un `ConfigMap` versionado en git y las credenciales se inyectan desde un `Secret` generado en tiempo de despliegue a partir de un fichero excluido del repositorio.

El alcance cubre exclusivamente el *dashboard* Streamlit, que es el componente de larga duración y el más adecuado para un modelo de *workload* continuo. La CLI `wazuh-ia` no se despliega como un `Deployment` independiente por su naturaleza interactiva (requiere entrada por teclado para la selección del proveedor y para la confirmación de la respuesta activa), aun-

que puede ejecutarse bajo demanda contra el pod del *dashboard* mediante `kubectl exec`, sin modificar una sola línea de código.

#### 4.10.2. Herramientas: kind sobre podman

El clúster local se levanta con **kind** (*Kubernetes in Docker*) [7], una herramienta que crea un clúster de Kubernetes [16] completo usando contenedores como nodos. `kind` está diseñado para entornos de desarrollo y pruebas: no requiere una instalación completa de Kubernetes en el sistema operativo y puede crearse y destruirse en segundos. En lugar de Docker, el *toolkit* de contenedores empleado es **podman** [13], que opera en modo *rootless*: los contenedores se ejecutan sin privilegios de superusuario en el sistema operativo anfitrión. El modo *rootless* tiene una consecuencia directa en los puertos: podman no puede publicar puertos privilegiados (menores de 1024) sin configuración adicional, de modo que el clúster expone el *dashboard* en el puerto 8080 en lugar del puerto 80 estándar.

El acceso se realiza a través del *hostname* `wazuh-llm.localtest.me`, que resuelve siempre a `127.0.0.1` sin necesidad de editar `/etc/hosts`, ya que el dominio `localtest.me` es un dominio público que apunta a la dirección de *loopback*. `kind` requiere que se especifique explícitamente el uso de podman en cada invocación mediante la variable de entorno `KIND_EXPERIMENTAL_PROVIDER=podman`, ya que por defecto intenta usar el demonio Docker.

El fichero `k8s/kind-cluster.yaml` define la topología del clúster (un único nodo de control) y los mapeos de puertos necesarios para el Ingress:

Listado 4.5: Configuración del clúster kind (`kind-cluster.yaml`).

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 8080    # podman rootless: no puede usar el puerto 80
  - containerPort: 443
    hostPort: 8443
```

### 4.10.3. Arquitectura del despliegue

La Figura 4.7 muestra la organización del despliegue. El *dashboard* Streamlit corre dentro del clúster kind como un **Deployment**. Wazuh y Ollama permanecen **fuera del clúster**, en el sistema anfitrión macOS, y se alcanzan desde los pods mediante *Services* con *EndpointSlice* manuales, detallados en la Sección 4.10.7.

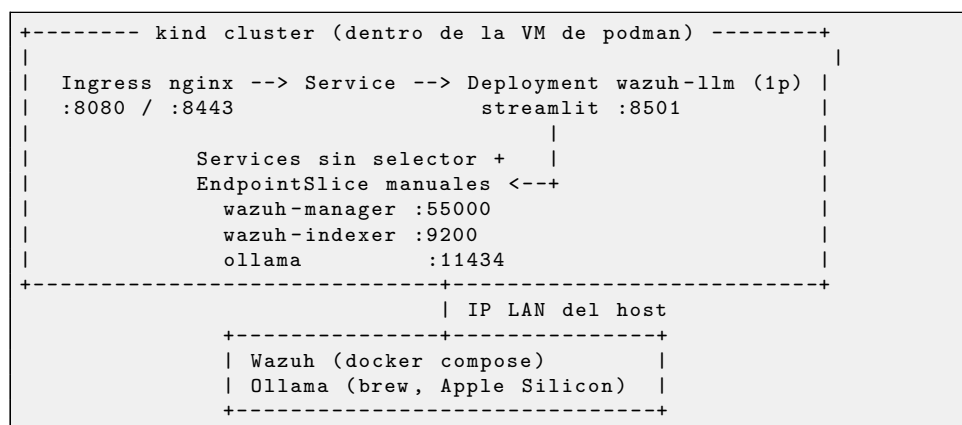


Figura 4.7: Arquitectura del despliegue Kubernetes. El *dashboard* corre dentro del clúster; Wazuh y Ollama permanecen en el *host* macOS y se alcanzan a través de *EndpointSlice* manuales.

La razón por la que Wazuh y Ollama quedan fuera es doble. El clúster kind corre dentro de la máquina virtual de podman en macOS, y un pod **no puede acceder a la GPU de Apple Silicon** desde dentro de esa VM: mover Ollama al interior del clúster significaría inferencia por CPU exclusivamente, con tiempos muy superiores a los registrados en las pruebas del Capítulo 5. Migrar el *stack* completo de Wazuh (Manager, Indexer, certificados TLS, agentes) a Kubernetes es un proyecto en sí mismo, de complejidad muy superior al despliegue de la aplicación.

### 4.10.4. Contenerización de la aplicación

La contenerización parte de un **Dockerfile** diseñado con tres criterios: reproducibilidad, seguridad y eficiencia en la construcción.

**Imagen base fijada.** Se utiliza `python:3.12-slim`, fijada a la versión 3.12 y no a la más reciente. Esta decisión garantiza que las dependencias de la aplicación (en particular Streamlit y `google-genai`) dispongan de paquetes precompilados (*wheels*) para esa versión, evitando compilaciones lentas desde código fuente en cada construcción.

**Orden de capas para maximizar la reutilización de caché.** Las dependencias se instalan antes de copiar el código de la aplicación. De este modo, la capa de `pip install` se reutiliza de la caché de podman mientras no cambie `pyproject.toml`, que es el fichero que declara las dependencias. El código de la aplicación, que cambia con más frecuencia, se copia después, en capas independientes.

**Usuario no raíz.** Se crea un usuario `appuser` con UID 10001, sin directorio *home* en el sistema de ficheros de la imagen y sin intérprete de comandos interactivo (`/usr/sbin/nologin`). La variable de entorno `HOME=/tmp` redirige los ficheros temporales de Python y Streamlit al único punto de escritura del contenedor: un volumen efímero `emptyDir` montado en `/tmp` por el `Deployment`.

**Sistema de ficheros de solo lectura.** `PYTHONDONTWRITEBYTECODE=1` impide que Python intente escribir `__pycache__` en el directorio de instalación. Combinado con `readOnlyRootFilesystem: true` en el `securityContext` del `Deployment`, garantiza que ningún proceso pueda modificar el sistema de ficheros raíz. Esta restricción es viable porque la aplicación únicamente lee de disco (el fichero de *prompt* en `prompts/mitre_prompt.txt`) y no escribe nada en tiempo de ejecución.

Listado 4.6: Dockerfile de la aplicación.

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    HOME=/tmp

WORKDIR /app

# Dependencias primero: la capa de pip se reutiliza mientras
# no cambie pyproject.toml
COPY pyproject.toml README.md ./
COPY wazuh_llm/ ./wazuh_llm/
RUN pip install --no-cache-dir .

# Código de la aplicación (cambia mas frecuentemente)
COPY Home.py ./
COPY pages/ ./pages/
COPY prompts/ ./prompts/
COPY .streamlit/ ./streamlit/

RUN useradd --uid 10001 --no-create-home \
    --shell /usr/sbin/nologin appuser
USER 10001

EXPOSE 8501
CMD ["streamlit", "run", "Home.py",
    "--server.port=8501", "--server.address=0.0.0.0"]
```

#### 4.10.5. Gestión de configuración y secretos

La configuración del sistema se divide en dos recursos Kubernetes con ciclos de vida y niveles de sensibilidad diferentes.

El **ConfigMap** `wazuh-llm-config` contiene los parámetros no sensibles: las URLs de la API del Manager y del Indexer, el usuario de la API, la IP del Manager (usada por la lista blanca del guardrail), la URL de Ollama, el nombre del modelo y la ruta del fichero de *prompt*. Este recurso se gestiona en git como cualquier otro manifiesto. Su contrapartida es que un cambio en el **ConfigMap** no dispara un *rollout* automático del **Deployment**: el operador debe reiniciarlo manualmente con `kubect1 rollout restart`.

El **Secret** `wazuh-llm-secrets` contiene las credenciales sensibles: la contraseña de la API de Wazuh, la contraseña del Indexer y las claves de API de Gemini y Groq. Este recurso **no reside en git**; se genera en tiempo de despliegue mediante `kustomize` a partir del fichero `k8s/.env.k8s`, excluido del repositorio por `.gitignore`. La ventaja clave del generador de secretos de `kustomize` es que añade un sufijo hash al nombre del **Secret** (por ejemplo, `wazuh-llm-secrets-7f4b2c9d`) y reescribe automáticamente todas las referencias a ese nombre en los manifiestos. Cuando el operador modifica `.env.k8s` y vuelve a ejecutar `kubect1 apply -k k8s/`, el hash cambia, lo que provoca la creación de un nuevo **Secret** y un *rollout* automático del **Deployment**: no es necesario reiniciar el pod manualmente tras rotar credenciales.

#### 4.10.6. Configuración del *Deployment*

El **Deployment** del *dashboard* incorpora cuatro decisiones de diseño derivadas del análisis del código previo al despliegue.

**Una sola réplica con estrategia *Recreate*.** El estado de sesión de Streamlit vive en la memoria del proceso Python, de modo que escalar a más réplicas requeriría externalizar ese estado. La estrategia **Recreate** elimina el pod existente antes de crear el nuevo, evitando inconsistencias entre versiones.

**Sondas de salud.** Las tres sondas (`startupProbe`, `readinessProbe` y `livenessProbe`) apuntan al *endpoint* nativo `/_stcore/health` de Streamlit. El umbral de fallos de la `livenessProbe` se fija deliberadamente alto para evitar que Kubernetes interrumpa el pod durante una inferencia larga.

**Recursos conservadores.** La aplicación es predominantemente *I/O-bound*; las solicitudes se fijan en 200 m de CPU y 256 MiB de memoria, con límites de 1 núcleo y 1 GiB.

**Endurecimiento del contexto de seguridad.** El contenedor se ejecuta sin privilegios de *root*, con el sistema de archivos en modo solo lectura y

sin capacidades Linux (`drop: ["ALL"]`). El único punto de escritura es un `emptyDir` en `/tmp` para los ficheros temporales de Streamlit.

#### 4.10.7. Integración de servicios externos mediante EndpointSlice

Dado que Wazuh y Ollama permanecen fuera del clúster, la aplicación necesita un mecanismo para alcanzarlos con nombres DNS estables en lugar de IPs incrustadas en el código o en el `ConfigMap`. La IP del *host* es una dirección DHCP que puede cambiar entre sesiones de red; si estuviera incrustada en múltiples puntos, cada cambio exigiría actualizar varios ficheros y reiniciar el despliegue.

El patrón utilizado es el de **Services sin selector con EndpointSlice manuales**. Un `Service` de Kubernetes sin campo `selector` no recibe *endpoints* automáticamente (Kubernetes no le asigna pods propios), de modo que se le puede asociar un `EndpointSlice` creado manualmente que apunte a la IP real del *host*. El resultado es que, dentro del clúster, los tres servicios externos tienen nombres DNS estables: `wazuh-manager.wazuh-llm.svc.cluster.local` (puerto 55000), `wazuh-indexer.wazuh-llm.svc.cluster.local` (puerto 9200) y `ollama.wazuh-llm.svc.cluster.local` (puerto 11434). Cuando la IP del *host* cambia, solo hay que actualizar `k8s/externals.yaml` en un único lugar.

Listado 4.7: Ejemplo de Service sin selector con EndpointSlice manual para Ollama (`externals.yaml`).

```
apiVersion: v1
kind: Service
metadata:
  name: ollama
spec:
  type: ClusterIP
  ports:
    - port: 11434
---
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: ollama-external
  labels:
    kubernetes.io/service-name: ollama
addressType: IPv4
ports:
  - port: 11434
endpoints:
  - addresses: ["192.168.1.3"]    # IP LAN del host; editar aqui
    conditions:
      ready: true
```

Un requisito adicional afecta específicamente a Ollama: por defecto, Ollama solo acepta conexiones en `127.0.0.1` del *host*. Desde dentro del pod existen dos saltos de red (pod → VM de podman → macOS), por lo que `127.0.0.1` se refiere al propio pod. Para que Ollama sea alcanzable desde el clúster es necesario exponerlo en todas las interfaces del *host* antes del despliegue:

```
launchctl setenv OLLAMA_HOST 0.0.0.0
brew services restart ollama
```

#### 4.10.8. Control de tráfico: Ingress y NetworkPolicy

**Ingress.** El controlador de Ingress utilizado es `ingress-nginx`. El acceso al *dashboard* se expone bajo el *host* `wazuh-llm.localtest.me`, que resuelve siempre a `127.0.0.1` sin modificar `/etc/hosts`. La anotación más crítica del manifiesto es el *timeout* del proxy:

Listado 4.8: Anotaciones de *timeout* en el Ingress (`ingress.yaml`).

```
nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
nginx.ingress.kubernetes.io/proxy-send-timeout: "300"
```

El valor por defecto de nginx es 60 segundos. Sin esta anotación, el proxy cerraría la conexión *WebSocket* a mitad de una inferencia de Ollama, que puede tardar hasta 180 segundos, y el analista vería el área de resultados en

blanco sin ningún mensaje de error. Los 300 segundos de margen cubren el peor caso observado durante las pruebas más una holgura operativa razonable. El Ingress no incluye autenticación de usuarios: cualquier persona que alcance la URL puede lanzar la respuesta activa. El fichero `ingress.yaml` incluye las anotaciones de autenticación básica preparadas y comentadas como punto de partida para quien desee habilitarla.

**NetworkPolicy.** El manifiesto `networkpolicy.yaml` implementa una política de *default-deny egress* con lista de permisos explícita: DNS (UDP 53), los puertos de Wazuh y Ollama en la IP del *host* (55000, 9200, 11434) y el puerto 443 a Internet para los proveedores externos Gemini y Groq. No obstante, *kindnet* (el plugin de red CNI instalado por defecto en *kind*) **no implementa NetworkPolicy**: el manifiesto se aplica sin errores pero no tiene efecto. Para que la política surta efecto en un clúster *kind* sería necesario crearlo con `disableDefaultCNI: true` e instalar Calico. El manifiesto se incluye porque documenta la postura de seguridad prevista y es directamente aplicable en cualquier clúster de producción con un CNI estándar.

#### 4.10.9. Triage por lotes mediante CronJob

La CLI `wazuh-ia` es interactiva por diseño: solicita entrada por teclado para la selección del proveedor y para la confirmación de la respuesta activa. Sin embargo, cuando se invoca con el argumento `--proveedor` y sin `--respuesta-activa`, elimina ambas entradas interactivas y opera de forma completamente no interactiva. Esta propiedad, extraída del análisis del código previo al despliegue, permite ejecutar la CLI como un **CronJob** de Kubernetes para el análisis programado de alertas en lote, sin modificar una sola línea de código.

El manifiesto `cronjob-batch.yaml` se entrega con `suspend: true`, lo que impide su ejecución automática hasta que el operador lo habilite explícitamente. Esta precaución evita que un **CronJob** habilitado por defecto consuma la cuota gratuita de los proveedores externos o ejecute análisis en momentos no deseados. La política de reintentos es `backoffLimit: 0`: si el *Job* falla, no se reintenta, ya que el siguiente *tick* del **CronJob** volverá a analizar las alertas más recientes de todos modos. El **CronJob** también comparte el mismo contexto de seguridad endurecido del **Deployment** (usuario no raíz, *rootfs* de solo lectura, sin capacidades).

La CLI puede ejecutarse también de forma puntual mediante `kubectl exec` contra el pod del *dashboard*, sin necesidad de habilitar el **CronJob**:

Listado 4.9: Ejecución puntual de la CLI y activación del CronJob.

```
# Ejecucion puntual contra el pod del dashboard
kubectl exec -it -n wazuh-llm deploy/wazuh-llm -- \
  wazuh-ia --proveedor groq --alertas 3

# Activar el triaje horario automatico
kubectl patch cronjob wazuh-llm-batch -n wazuh-llm \
  -p '{"spec":{"suspend":false}}'
kubectl logs -n wazuh-llm -l app.kubernetes.io/component=batch
```

#### 4.10.10. Verificación del despliegue

Una vez aplicados los manifiestos con `kubectl apply -k k8s/` y completado el *rollout*, el estado del *namespace* `wazuh-llm` refleja todos los objetos desplegados:

Listado 4.10: Estado del clúster tras el despliegue completo (`kubectl get all -n wazuh-llm`).

|                                      |           |               |             |        |
|--------------------------------------|-----------|---------------|-------------|--------|
| NAME                                 | READY     | STATUS        | RESTARTS    | AGE    |
| pod/wazuh-llm-86665db785-wgbkq       | 1/1       | Running       | 0           | 58s    |
| NAME                                 | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT   |
| (S) AGE                              |           |               |             |        |
| service/ollama                       | ClusterIP | 10.96.196.113 | <none>      |        |
| 11434/TCP 58s                        |           |               |             |        |
| service/wazuh-indexer                | ClusterIP | 10.96.57.9    | <none>      |        |
| 9200/TCP 58s                         |           |               |             |        |
| service/wazuh-llm                    | ClusterIP | 10.96.108.12  | <none>      | 80/    |
| TCP 58s                              |           |               |             |        |
| service/wazuh-manager                | ClusterIP | 10.96.109.210 | <none>      |        |
| 55000/TCP 58s                        |           |               |             |        |
| NAME                                 | READY     | UP-TO-DATE    | AVAILABLE   | AGE    |
| deployment.apps/wazuh-llm            | 1/1       | 1             | 1           | 58s    |
| NAME                                 | DESIRED   | CURRENT       | READY       |        |
| AGE                                  |           |               |             |        |
| replicaset.apps/wazuh-llm-86665db785 | 1         | 1             | 1           | 58s    |
| s                                    |           |               |             |        |
| NAME                                 | SCHEDULE  | SUSPEND       | ACTIVE      | LAST   |
| SCHEDULE                             |           |               |             |        |
| cronjob.batch/wazuh-llm-batch        | 0 * * * * | True          | 0           | <none> |

El pod arranca en estado `1/1 Running`, los cuatro servicios `ClusterIP` están activos y el `CronJob` aparece en estado `SUSPEND: True`, tal como se configuró.

La Figura 4.8 muestra la página de inicio del *dashboard* accediendo desde el navegador a `http://wazuh-llm.localtest.me:8080`. Ambas comprobaciones de conectividad aparecen en verde: la API de gestión (puerto 55000) y el Indexer OpenSearch (puerto 9200), lo que confirma que los *EndpointSlice*

manuales enrutan correctamente el tráfico desde el clúster hasta los servicios de Wazuh en el anfitrión.



Figura 4.8: Página de inicio del *dashboard* Streamlit desplegado en Kubernetes, con las dos conexiones a Wazuh verificadas y 50 alertas cargadas.

La Tabla 4.6 resume el inventario completo de objetos desplegados en el espacio de nombres `wazuh-llm` y la función de cada uno dentro del clúster.

Tabla 4.6: Recursos Kubernetes desplegados en el espacio de nombres `wazuh-llm`.

| Tipo            | Nombre                        | Función                                                         |
|-----------------|-------------------------------|-----------------------------------------------------------------|
| Namespace       | <code>wazuh-llm</code>        | Aísla todos los objetos del despliegue                          |
| Deployment      | <code>wazuh-llm</code>        | Gestiona el ciclo de vida del pod del <i>dashboard</i>          |
| Service         | <code>wazuh-llm</code>        | Expone el pod Streamlit en el puerto 80 del clúster             |
| Service         | <code>wazuh-manager</code>    | Proxy hacia la API REST de Wazuh (puerto 55000)                 |
| Service         | <code>wazuh-indexer</code>    | Proxy hacia OpenSearch (puerto 9200)                            |
| Service         | <code>ollama</code>           | Proxy hacia la API de inferencia local (puerto 11434)           |
| EndpointSlice×3 | -                             | Dirigen cada servicio externo a la IP LAN del anfitrión         |
| Ingress         | <code>wazuh-llm</code>        | Enruta el tráfico externo con <i>timeout</i> de 300 s           |
| ConfigMap       | <code>wazuh-llm-config</code> | Variables de entorno no sensibles (IP del Manager, etc.)        |
| Secret          | <code>wazuh-llm-*</code>      | Credenciales de Wazuh y claves de API (sufijo <i>hash</i> )     |
| NetworkPolicy   | <code>wazuh-llm-egress</code> | Política <i>default-deny</i> egress con <i>allowlist</i> de IPs |
| CronJob         | <code>wazuh-llm-batch</code>  | Triaje programado por lotes (suspendido por defecto)            |

#### 4.10.11. Limitaciones conocidas del despliegue

El despliegue descrito cubre los requisitos del entorno de pruebas del TFG, pero presenta cuatro limitaciones que quedan documentadas como trabajo futuro.

**NetworkPolicy inoperativa en kind estándar.** Como se ha explicado, kindnet no implementa NetworkPolicy. La política de aislamiento de red solo es efectiva en un clúster con Calico u otro CNI compatible, o en un servicio gestionado de Kubernetes (AKS, EKS, GKE), donde la implementación de NetworkPolicy está garantizada por el proveedor.

**Una sola réplica.** La restricción de `st.session_state` impide el escalado horizontal sin cambios de arquitectura. Un despliegue de alta disponibilidad requeriría *sticky sessions* a nivel de Ingress o externalizar el estado de sesión a Redis u otro almacén distribuido, con los cambios de código correspondientes.

**Sin autenticación de usuario en el Ingress.** En un entorno expuesto a una red no confiable, cualquiera que alcance la URL puede lanzar la respuesta activa. Las anotaciones de autenticación básica en `ingress.yaml` son el punto de partida más inmediato; una solución más robusta implicaría un proveedor de identidad OAuth 2.0 integrado con el Ingress, o restringir el acceso mediante una VPN o una lista de IPs permitidas.

**Sin *registry* de imágenes.** La imagen se inyecta en el nodo de kind mediante `kind load image-archive` en lugar de descargarse de un *registry*. Este mecanismo funciona en un entorno de un único nodo, pero no escala a clústeres de varios nodos ni a un pipeline de CI/CD. La incorporación de un *registry* local (Harbor) o gestionado (GitHub Container Registry) sería el primer paso hacia un flujo de entrega continua en el que cada *push* al repositorio disparara la construcción y el despliegue automáticos de la nueva imagen.

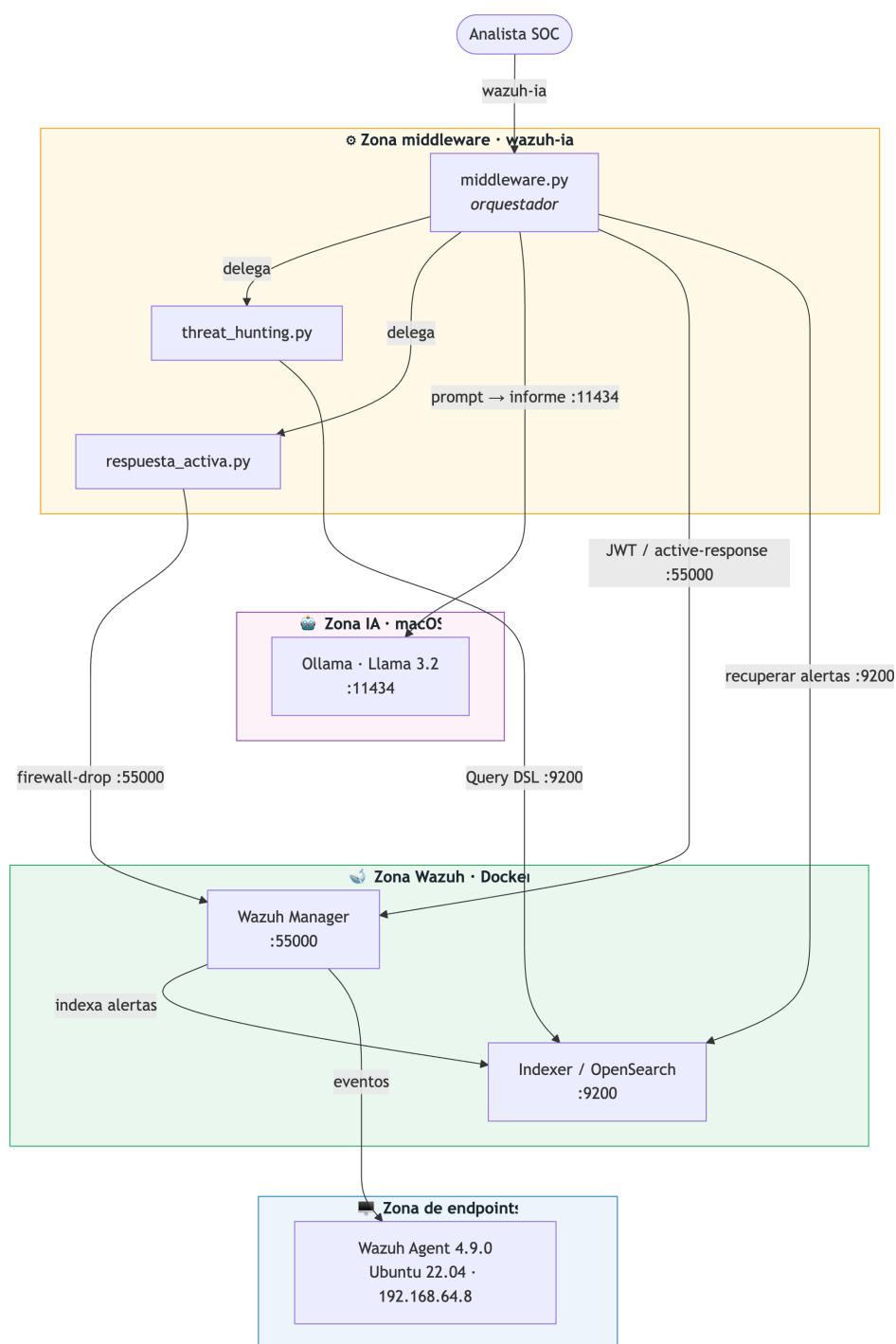


Figura 4.9: Diagrama de componentes del sistema. La zona de *endpoints* genera eventos que el agente Wazuh reenvía al Manager; el *middleware* consulta las alertas al Indexer (:9200) y obtiene el token de autenticación del Manager (:55000); Ollama (:11434) recibe el *prompt* y devuelve el informe. Las flechas de `respuesta_activa.py` y `threat_hunting.py` al Manager y al Indexer, respectivamente, se activan solo cuando se invocan los modos correspondientes.

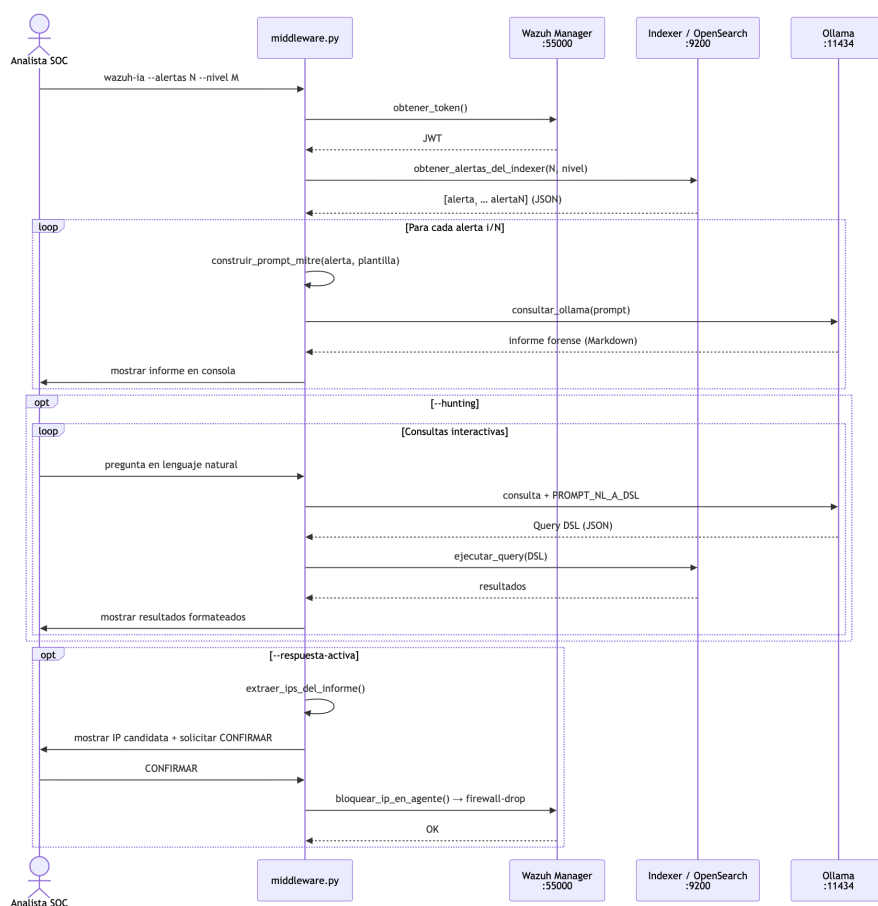


Figura 4.10: Diagrama de secuencia del flujo completo del *middleware*. El bloque **loop** central muestra el análisis forense de cada alerta; los bloques **opt** inferiores representan los modos **--hunting** (consultas en lenguaje natural al Indexer vía Query DSL) y **--respuesta-activa** (confirmación humana y ejecución de **firewall-drop**).

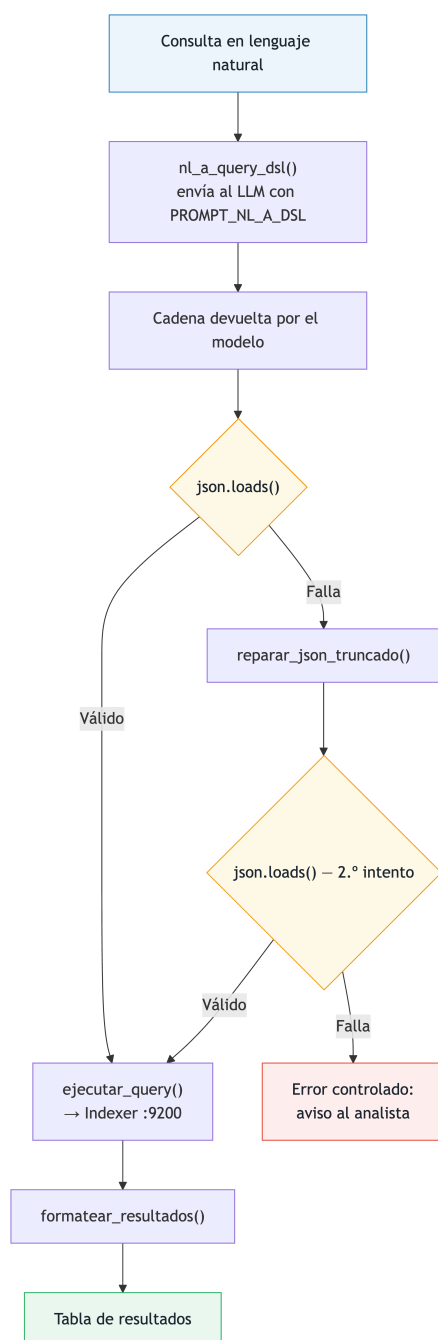


Figura 4.11: Flujo del módulo de *threat hunting*. La consulta en lenguaje natural se traduce a Query DSL mediante el LLM; si el JSON devuelto está truncado, `reparar_json_truncado()` añade los cierres necesarios y se reintenta la interpretación. Solo cuando el JSON es válido se lanza `ejecutar_query()` contra el Indexer.

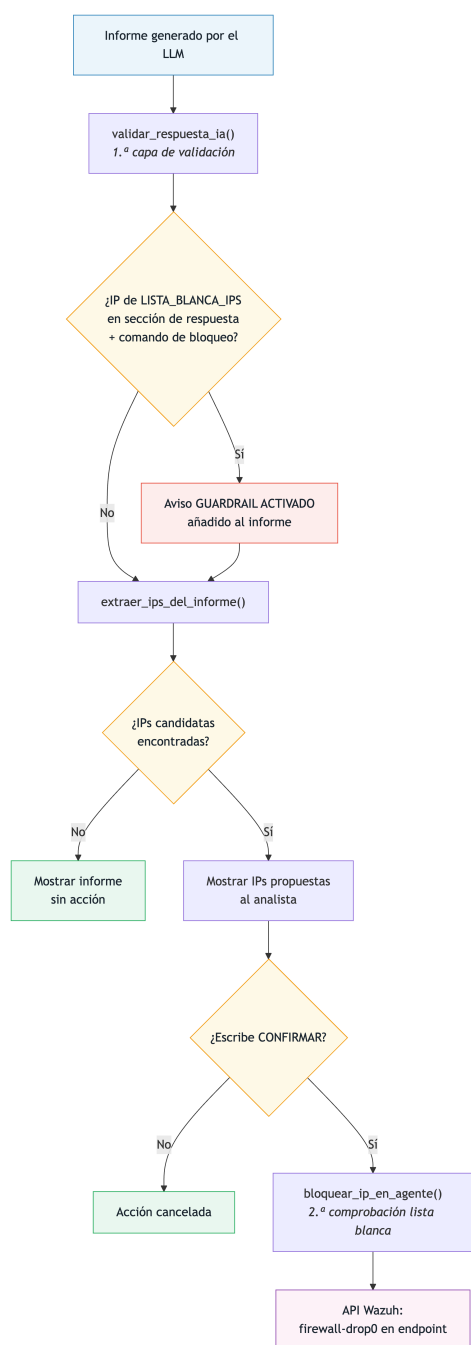


Figura 4.12: Flujo del módulo de respuesta activa con patrón *Human-in-the-Loop*. La validación actúa en dos capas independientes: `validar_respuesta_ia()` inspecciona el informe y añade un aviso si el modelo sugirió bloquear una dirección protegida; `bloquear_ip_en_agente()` comprueba de nuevo la lista blanca antes de llamar a la API. Solo la cadena exacta `CONFIRMAR` desencadena la orden de bloqueo.



## Capítulo 5

# Experimentos y resultados

Este capítulo describe la validación del sistema sobre datos reales generados en el entorno de pruebas. Se presentan el entorno de pruebas y cuatro experimentos: análisis forense de alertas, *threat hunting* en lenguaje natural, respuesta activa con confirmación humana y una comparativa del rendimiento entre tres modelos de lenguaje distintos ante cinco vectores de ataque.

### 5.1. Entorno de pruebas

El entorno de pruebas se basa en la arquitectura de cuatro capas descrita en el Capítulo 4 y se ha implementado en un portátil personal con procesador Apple Silicon M1, con 8 GB de RAM y un disco de 256 GB. El sistema operativo es MacOS Tahoe Versión 26.6.2. El entorno de virtualización elegido es QEMU ARM 10.0 con UTM 4.7.5, que permite ejecutar máquinas virtuales ARM64 con aceleración por hardware y red compartida con el equipo anfitrión.

La **zona de endpoints** la ocupa una máquina virtual Ubuntu 22.04 LTS ARM64 ejecutada con UTM sobre el equipo anfitrión. El agente Wazuh 4.9.0 corre como servicio activo; en los experimentos de fuerza bruta actúa como víctima. Su dirección en la red compartida de UTM es 192.168.64.8.

La **zona Wazuh** corresponde al despliegue *single-node* de Wazuh 4.9.0 [20] mediante Docker Compose en el propio equipo anfitrión. El Manager escucha en el puerto 55000 (API de gestión con autenticación JWT) y el Indexer en el 9200 (OpenSearch). El Dashboard es accesible en el puerto 443.

La **zona del middleware** es el entorno virtual Python del repositorio del TFG, con el comando `wazuh-ia` instalado y disponible desde la terminal del equipo anfitrión. Las variables de conexión (URLs, credenciales, nombre

del modelo) se gestionan mediante `.env`.

La **zona de inteligencia artificial** es Ollama ejecutando Llama 3.2 (un modelo de lenguaje ligero con 3.000 millones de parámetros, los valores numéricos internos que codifican el conocimiento del modelo, y 2,0 GB de espacio en disco) en el equipo anfitrión. La API REST de Ollama escucha en el puerto 11434. Para la Prueba 4 se incorporaron dos proveedores externos: Gemini Flash (Google AI) y Llama 3.3-70B a través de Groq, ambos invocados mediante sus API REST con clave de autenticación y sin procesamiento local de los datos de alerta.

La red compartida de UTM (segmento 192.168.64.0/24) conecta la VM Ubuntu con el equipo anfitrión. En los experimentos de fuerza bruta, el equipo anfitrión actúa como atacante con la dirección 192.168.64.1.

## 5.2. Justificación y secuencia del diseño experimental

Para validar que el sistema desarrollado en el presente Trabajo Fin de Grado funciona correctamente y aporta valor, he diseñado una batería de cuatro pruebas que no son independientes entre sí, sino que replican paso a paso cómo trabaja un analista de Nivel 1 en un SOC real: se reciben alertas diarias categorizadas por el SIEM según su nivel de severidad que deben ser analizadas detalladamente; dichas alertas deben pasar por un filtro de triaje en el que se revisan los eventos previos y posteriores a la posible amenaza para generar contexto que ayude a medir el potencial impacto; se comprueban todos los datos relevantes y vulnerables para ver qué acciones se pueden llevar a cabo y se toman las decisiones pertinentes, por ejemplo el bloqueo de alguna IP o equipo.

Puedo justificar el interés de este trabajo desde mi propia experiencia como analista de Nivel 1. En el día a día me llega un volumen de alertas que no puedo examinar una a una con el detalle que me gustaría, y buena parte de mi esfuerzo se consume en trabajo mecánico: leer “logs” en bruto, reconocer a qué tipo de ataque corresponde lo que estoy viendo y decidir si merece escalarse o es un falso positivo. El sistema que he desarrollado se hace cargo de esa parte: entrega la alerta ya interpretada y clasificada según MITRE ATT&CK, me permite buscar en el histórico de alertas en lenguaje natural sin tener que escribir consultas estructuradas, y disminuye el tiempo que transcurre entre detectar una amenaza y resolverla. Los analistas de nivel superior también se ven beneficiados, ya que cuando un caso se escala, el analista lo recibe con el informe ya realizado y con las acciones ejecutadas verificables, de modo que parte de un contexto más amplio y contrastado.

Además de todo lo anterior, quiero resaltar la confidencialidad, y es la

razón principal por la que descarté conectar el sistema con una API comercial. Una alerta de seguridad no es un dato anónimo: contiene direcciones internas, nombres de equipos y de usuarios, rutas, versiones de software y, muy a menudo, información confidencial que podría usarse para un ataque. Enviar ese material a terceros supone entregarles detalles relevantes de la infraestructura del cliente junto con información de sus puntos débiles. Ejecutar el modelo en local resuelve el problema: los datos no abandonan el entorno local en ningún momento, no hay que negociar cláusulas de tratamiento con un proveedor externo ni depender de su política de retención.

La secuencia de pruebas se corresponde con las tareas comunes del flujo de trabajo, tal y como resume la Figura 5.1: el análisis forense, la detección de amenazas (*threat hunting*), la respuesta activa y la comparativa de eficacia.

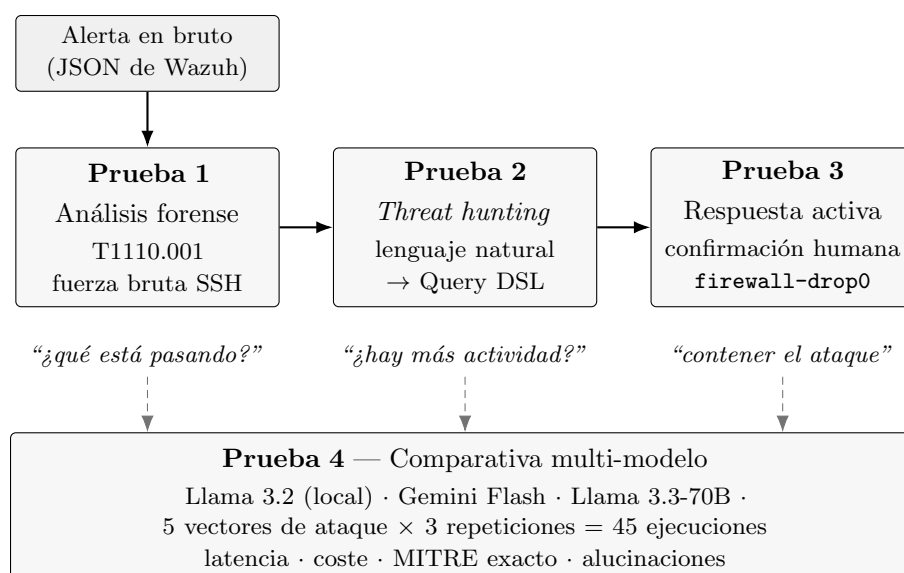


Figura 5.1: Secuencia del diseño experimental. Las Pruebas 1 a 3 reproducen el ciclo de vida de un incidente tal y como lo recorre un analista de Nivel 1: interpretar la alerta, buscar actividad relacionada en el histórico y contener el ataque. La Prueba 4 no continúa ese ciclo, sino que evalúa de forma transversal la viabilidad del sistema comparando los tres proveedores de inferencia sobre los cinco vectores de ataque.

1. **Prueba 1 (Análisis forense):** Quiero evaluar la capacidad del sistema de reconstruir la secuencia de eventos que componen un conjunto de elementos; para ello elegí un patrón de ataque con un ataque de fuerza bruta SSH porque es el vector más ruidoso y habitual al que se enfrenta un SOC. Un ataque de fuerza bruta SSH generalmente implica intentos de acceso recurrentes al puerto TCP 22 del equipo víctima, aunque se puede extender a otros puertos TCP, y utiliza generación

aleatoria apoyada en diccionarios y fuentes de nombres comunes para la autenticación básica. Este tipo de ataque genera un volumen claro de alertas sucesivas, lo que lo convierte en el escenario ideal para comprobar si el sistema es capaz de hacer el primer paso del triaje: interpretar la alerta en bruto, resumir qué está pasando e identificar el ataque como tal. Adicionalmente, el sistema enriquece el análisis generando un informe de acciones de resolución y otras sugerencias relativas a la brecha detectada.

2. **Prueba 2 (*Threat hunting*):** En la operativa real, una vez que el analista ve esa primera alerta, no se queda ahí; el siguiente paso es buscar en el histórico de alertas para ver si hay más actividad de la IP del atacante e investigar otros patrones. Esta prueba persigue evaluar la capacidad del sistema para obtener dicho histórico de alertas. El sistema efectivamente agiliza y simplifica la obtención de alertas a partir de las consultas en lenguaje natural realizadas por el analista, las transforma en peticiones al almacén de eventos y recupera las alertas que cuadran con lo que se ha pedido.
3. **Prueba 3 (Respuesta activa):** Utilizo el patrón *Human-in-the-Loop* para evitar bloqueos indeseados y exigir supervisión humana a las acciones sugeridas. En el flujo de trabajo manual, el analista toma la decisión de bloquear después de realizar el análisis forense en caso de detectarla como una amenaza y confirmar mediante *threat hunting* que no se trata de un falso positivo o actividad legítima. Esta prueba sirve para verificar que la acción de contención de la víctima se ejecuta correctamente en el cortafuegos y además notifica al analista.
4. **Prueba 4 (Comparativa multimodelo):** Las tres primeras pruebas demuestran que el concepto funciona. Sin embargo, quería medir cómo de buena es dicha solución. Para ello he utilizado diferentes modelos LLM más potentes con diferentes vectores de ataque: fuerza bruta SSH, escalada de privilegios con sudo, escaneo de puertos, creación de cuenta no autorizada y acceso con cuenta válida. Esta última evaluación me ha permitido medir los indicadores clave en producción: la latencia, el coste y, muy especialmente, la tasa de alucinaciones al comparar un modelo local frente a motores en la nube. Como cabía esperar, los modelos más potentes realizan los mejores informes en base a los indicadores clave.

## 5.3. Prueba 1: análisis forense de alertas

### 5.3.1. Escenario: T1110.001, fuerza bruta SSH

El escenario reproduce la técnica T1110.001 (*Brute Force: Password Guessing*) del catálogo MITRE ATT&CK mediante múltiples intentos fallidos de autenticación SSH lanzados desde el equipo anfitrión contra la VM Ubuntu: conexiones con nombres de usuario inexistentes en el sistema (**fakeuser**, **attacker**, **hacker**, entre otros) y contraseñas incorrectas para el usuario legítimo **ubuntu**. No se empleó ninguna herramienta externa; los intentos se lanzaron con el cliente SSH de macOS.

Una vez acumuladas las alertas, se ejecutó el *middleware* con el comando:

```
wazuh-ia --alertas 5 --nivel 5
```

El valor `--alertas 5` captura las cinco alertas más recientes, suficientes para incluir tanto las reglas de evento individual (5710, 5503, nivel 5) como las de correlación de patrón (5712, 2502, nivel 10). El umbral `--nivel 5` excluye los eventos de mantenimiento del agente (niveles 1–4) y retiene todo lo relacionado con el ataque.

### 5.3.2. Alertas generadas

Wazuh detectó el patrón de ataque y generó cuatro tipos de alerta, todos correlacionados automáticamente con T1110.001 (*Brute Force: Password Guessing*) y T1021.004 (*Remote Services: SSH*) de MITRE ATT&CK:

- **Regla 5712** (nivel 10): *sshd: brute force trying to get access to the system. Non-existent user*. Alerta de correlación que agrega los intentos repetidos.
- **Regla 2502** (nivel 10): *syslog: User missed the password more than one time*. Patrón de reintento de contraseña detectado por el módulo de *syslog*.
- **Regla 5710** (nivel 5): *sshd: Attempt to login using a non-existent user*. Un evento por cada intento con usuario inexistente.
- **Regla 5503** (nivel 5): *PAM: User login failed*. Fallo de autenticación PAM para el usuario existente **ubuntu**.

El dashboard de Wazuh (vista *Threat Hunting*, agente **Ubuntu-Victima**) registró las alertas en cuatro reglas: las reglas 5712 y 2502 (nivel 10) reflejan

la detección del patrón de fuerza bruta por correlación, mientras que las reglas 5710 y 5503 (nivel 5) corresponden a los intentos individuales. Todas están correlacionadas automáticamente con T1110.001 de MITRE ATT&CK.

### 5.3.3. Informe generado por el *middleware*

El *middleware* se conectó a la API de gestión (puerto 55000) y al Indexer (puerto 9200), recuperó las cinco alertas y las procesó con Llama 3.2. El Listado 5.1 reproduce la salida para la alerta más severa (regla 5712, nivel 10). El modelo devuelve el informe con marcas de formato Markdown; la interfaz de línea de comandos las vuelca directamente como texto plano, sin renderizar los estilos (la interfaz web, en cambio, los muestra formateados en la página de análisis).

Listado 5.1: Extracto de la salida del *middleware* para la alerta de fuerza bruta (regla 5712, nivel 10). El modelo devuelve texto en formato Markdown sin transformar.

```
=====
ALERTA 1/5 | INFORME GENERADO POR LLM
=====
[IA] Procesando evento de nivel 10 en 'Ubuntu-Victima' (Regla ID:
      5712)...

**Resumen Ejecutivo y Kill Chain**
Intento de acceso no autorizado mediante fuerza bruta en Ubuntu-
Victima.
Origen: 192.168.64.1. Fase: Credential Access (T1110.001 / T1021.004).

**Plan de Respuesta Activa**
1. Bloquear la IP de origen con 'firewall-drop' via active-response.
2. Revisar el log de autenticacion en busca de usuarios comprometidos.

[... secciones Analisis Forense, Anomalias y Recomendaciones omitidas
    ...]
[... alertas 2/5 a 5/5 omitidas por brevedad ...]
```

### 5.3.4. Evaluación de la calidad

Tras analizar las respuestas generadas en esta primera prueba, se pueden identificar claramente los puntos fuertes de la integración Wazuh-LLM, así como las limitaciones propias al uso de un modelo local:

**Aciertos.** El *middleware* completó el flujo sin interrupciones: autenticación contra la API, recuperación de alertas del Indexer e inferencia con Ollama para cada una. La correlación con MITRE ATT&CK fue correcta en las cinco alertas (T1110.001 y T1021.004). La fase de la *Cyber Kill Chain* se

identificó como *Credential Access* en cuatro de las cinco alertas. El comando `firewall-drop` apareció como acción de respuesta en tres de las cinco, respetando la restricción introducida en el *prompt*.

A pesar del éxito de la prueba, se han detectado cinco deficiencias concretas:

1. **Alucinación de herramientas.** En tres de las cinco alertas, el modelo afirmó "se detectan herramientas como Hydra y nmap en el log", cuando ninguna de esas herramientas figuraba en los registros reales. El *raw log* contenía únicamente entradas del demonio `sshd`.
2. **Comando inexistente.** En la alerta de la regla 2502, el modelo propuso `wazuh firewall-drop --rule-id 2502 --username [usuario]`, que no existe en la API de Wazuh.
3. **Fase de la *Kill Chain* incorrecta.** La alerta de la regla 2502 fue clasificada como *Delivery*; la fase correcta para un intento de autenticación fallida es *Credential Access*.
4. **Error factual.** En la alerta de la regla 5710, el modelo indicó "la acción se ha llevado a cabo con éxito", cuando el registro original describe un intento *fallido*.
5. **Expansión incorrecta de acrónimo.** En la alerta de la regla 5503, el modelo definió PAM como "Plataformas de Autenticación de Negocios"; la expansión correcta es *Pluggable Authentication Modules*.

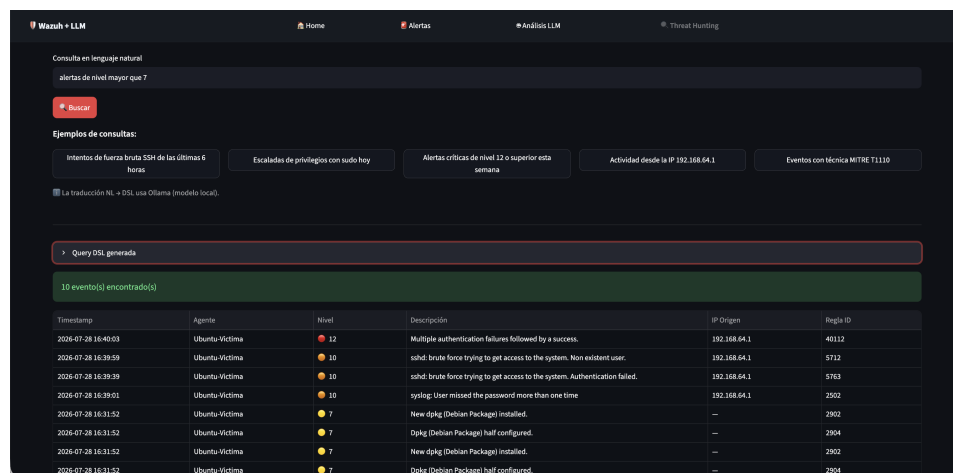
Estas carencias ilustran el comportamiento de los modelos de lenguaje generales en dominios técnicos especializados: producen una estructura de informe coherente pero pueden fallar en detalles factuales concretos. El diseño del sistema acota su impacto: el análisis forense lo revisa siempre el analista, y ninguna acción de respuesta activa se ejecuta sin confirmación humana. La mejora más eficaz a largo plazo sería el uso de RAG (*Retrieval-Augmented Generation*): enriquecer el *prompt* con documentación técnica verificada de Wazuh permitiría al modelo razonar sobre información específica del dominio, reduciendo drásticamente las alucinaciones sin necesidad de reentrenamiento.

## 5.4. Prueba 2: *threat hunting* en lenguaje natural

La segunda prueba evalúa el módulo de *threat hunting* con el índice de alertas acumulado durante los experimentos anteriores. Las consultas se formularon desde la página *Threat Hunting* de la interfaz web (Sección 4.7): el analista escribe una pregunta en español, el sistema la traduce a una

Query DSL de OpenSearch mediante el modelo local y ejecuta la búsqueda contra el Indexer; los resultados aparecen en una tabla con *timestamp*, agente, nivel, descripción, IP de origen e identificador de regla. Se diseñaron cuatro consultas para cubrir tres tipos de campo del esquema de Wazuh: nivel numérico (`rule.level`), código de técnica MITRE (`rule.mitre.id`) y dirección IP de origen (`data.srcip`).

**Consulta 1: filtrado por nivel.** La pregunta "alertas de nivel mayor que 7" generó el operador `gte: 7` (*greater than or equal to*, mayor o igual que) sobre `rule.level` y devolvió diez eventos (Figura 5.2): las alertas de fuerza bruta SSH de la Prueba 1 (reglas 5712, 5763, 2502; niveles 10–12) y seis eventos de instalación de paquetes `dpkg` (nivel 7). La presencia de estos últimos es esperable: el índice no distingue categorías de evento y el filtro devuelve todo lo que supera el umbral numérico.



Wazuh • LLM

Consulta en lenguaje natural

alertas de nivel mayor que 7

Buscar

Ejemplos de consultas:

Intentos de fuerza bruta SSH de las últimas 6 horas

Escaladas de privilegios con sudo hoy

Alertas críticas de nivel 12 o superior esta semana

Actividad desde la IP 192.168.64.1

Eventos con técnica MITRE T1110

La traducción NL a DSL usa Ollama (modelo local).

Query DSL generada

10 evento(s) encontrado(s)

| Timestamp           | Agente         | Nivel | Descripción                                                                  | IP Origen    | Regla ID |
|---------------------|----------------|-------|------------------------------------------------------------------------------|--------------|----------|
| 2026-07-28 16:40:03 | Ubuntu-Victima | 12    | Multiple authentication failures followed by a success.                      | 192.168.64.1 | 40112    |
| 2026-07-28 16:39:59 | Ubuntu-Victima | 10    | sshd: brute force trying to get access to the system. Non-existent user.     | 192.168.64.1 | 5712     |
| 2026-07-28 16:39:39 | Ubuntu-Victima | 10    | sshd: brute force trying to get access to the system. Authentication failed. | 192.168.64.1 | 5763     |
| 2026-07-28 16:39:01 | Ubuntu-Victima | 10    | syslog: User missed the password more than one time                          | 192.168.64.1 | 2502     |
| 2026-07-28 16:31:52 | Ubuntu-Victima | 7     | New dpkg (Debian Package) installed.                                         | —            | 2902     |
| 2026-07-28 16:31:52 | Ubuntu-Victima | 7     | Dpkg (Debian Package) half configured.                                       | —            | 2904     |
| 2026-07-28 16:31:52 | Ubuntu-Victima | 7     | New dpkg (Debian Package) installed.                                         | —            | 2902     |
| 2026-07-28 16:31:52 | Ubuntu-Victima | 7     | Dpkg (Debian Package) half configured.                                       | —            | 2904     |

Figura 5.2: Consulta 1 sobre `rule.level`: diez eventos devueltos, con las alertas de fuerza bruta SSH (niveles 10–12) y eventos de sistema de nivel 7. La tabla muestra *timestamp*, agente, nivel con código de color, descripción, IP de origen e identificador de regla.

**Consulta 2: técnica MITRE por código padre.** La pregunta "eventos con tecnica T1548" generó la búsqueda `rule.mitre.id: "T1548"` y devolvió cero resultados (Figura 5.3). El campo empleado es correcto, pero el valor buscado no: Wazuh indexa sub-técnicas con su código completo (T1548.003), no el identificador padre sin sufijo. La Query DSL es JSON válido y se ejecuta sin errores, pero no produce coincidencias ni emite aviso alguno.

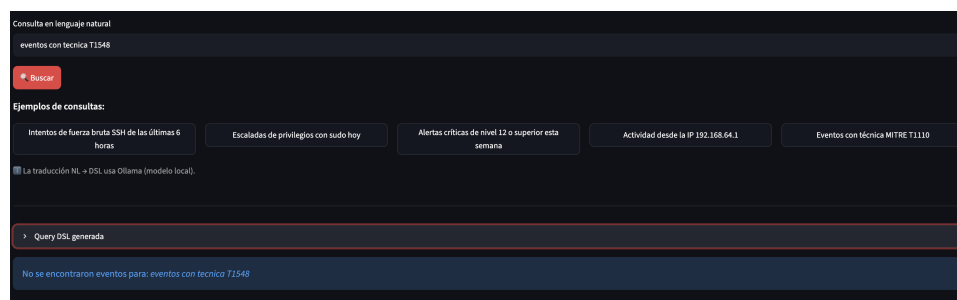


Figura 5.3: Consulta 2 sobre `rule.mitre.id` con el código padre T1548: cero resultados. El modelo identificó el campo correcto pero buscó el código padre; el índice almacena exclusivamente sub-técnicas completas como T1548.003.

**Consulta 3: pivote por IP.** La pregunta "alertas de la IP 192.168.64.1" generó la búsqueda `data.srcip: "192.168.64.1"` y devolvió los diez eventos más recientes de la dirección atacante, mostrando la progresión cronológica del ataque desde los primeros intentos fallidos hasta la correlación de fuerza bruta. Esta capacidad de pivotar sobre una IP sospechosa para reconstruir su historial de actividad cubre un caso de uso habitual al inicio de cualquier investigación.

**Consulta 4: sub-técnica MITRE completa.** La pregunta "alertas con mitre id T1548.003" generó la búsqueda `rule.mitre.id: "T1548.003"` y devolvió diez eventos (Figura 5.4). Al especificar el código completo de sub-técnica, el resultado recupera en una sola consulta todo el historial de escaladas a `root` mediante `sudo` acumuladas durante las sesiones de prueba del entorno (regla 5402, nivel 3), sin necesidad de conocer el identificador de regla ni el esquema del índice.

| 10 evento(s) encontrado(s) |                |       |                                   |           |          |
|----------------------------|----------------|-------|-----------------------------------|-----------|----------|
| Timestamp                  | Agente         | Nivel | Descripción                       | IP Origen | Regla ID |
| 2026-07-28 15:53:07        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:22:46        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:22:00        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:20:00        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:19:52        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:19:50        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:15:19        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:14:29        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:14:11        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |
| 2026-07-27 17:12:56        | Ubuntu-Victima | ● 3   | Successful sudo to ROOT executed. | —         | 5402     |

Figura 5.4: Consulta 4 sobre `rule.mitre.id` con el código completo T1548.003: diez eventos de la técnica *Abuse Elevation Control Mechanism: Sudo and Sudo Caching*, correspondientes a escaladas exitosas a `root` registradas durante las sesiones de pruebas.

### 5.4.1. Evaluación de la calidad

La evaluación de este módulo de *threat hunting* demostró la viabilidad de delegar la construcción de consultas al modelo, aunque con matices importantes:

**Aciertos.** Las consultas 1, 3 y 4 produjeron Query DSL válidas y resultados relevantes. La consulta 1 demostró que el filtrado numérico por nivel funciona correctamente y agrega eventos de cualquier naturaleza que superen el umbral. La consulta 3 confirmó la utilidad del pivote por IP para reconstruir la cronología de un ataque con una sola pregunta en lenguaje natural. La consulta 4 es la más ilustrativa: al especificar el código completo de sub-técnica, el modelo lo mapeó al campo correcto y recuperó la secuencia completa de la escalada de privilegios.

Por otro lado, la prueba también sacó a la luz ciertos fallos entre el modelo y el esquema de datos de Wazuh:

#### Carencias detectadas.

1. **Granularidad del código MITRE.** La consulta 2 usó el código padre T1548 en lugar del código de sub-técnica T1548.003. El modelo eligió el campo correcto (`rule.mitre.id`), pero al no precisarse la granularidad, la búsqueda no produjo coincidencias porque el índice almacena sub-técnicas completas, no códigos padre. La reformulación de la consulta 4 con "mitre id T1548.003" bastó para obtener los diez eventos correspondientes, lo que sitúa el problema en la ambigüedad de la pregunta original, no en la capacidad del modelo. Para reducir esta ambigüedad, el `PROMPT_NL_A_DSL` incluye en la descripción del esquema ambos formatos de código MITRE (padre, T1110, y sub-técnica completa, T1548.003), orientando al modelo hacia el identificador exacto que Wazuh almacena en el índice.
2. **Búsquedas de texto libre sobre campos *keyword*.** El campo `rule.description` está indexado en OpenSearch como `keyword`, lo que impide las búsquedas de texto parcial con el operador `match`: este solo devuelve resultados cuando el valor buscado coincide exactamente con el campo completo. Las cuatro consultas de la prueba evitaron esta situación al operar sobre campos estructurados (`rule.level`, `rule.mitre.id` y `data.srcip`); la limitación se identificó en el análisis del esquema de índice durante el desarrollo y se abordó incorporando en `PROMPT_NL_A_DSL` la instrucción de usar `query_string` con comodines (p. ej., `*ssh*`) para búsquedas textuales, operador compatible con campos `keyword` en OpenSearch.

## 5.5. Prueba 3: respuesta activa con confirmación humana

La tercera prueba reproduce el escenario de fuerza bruta SSH para validar el ciclo completo del módulo de respuesta activa. El objetivo es verificar que el sistema extrae la IP atacante de la alerta, propone el bloqueo al analista, ejecuta la acción tras recibir la confirmación explícita y deja una traza comprobable en el *firewall* del agente.

### 5.5.1. Configuración previa

El mecanismo de respuesta activa de Wazuh requiere que el Manager declare los comandos permitidos en `ossec.conf`. El bloque añadido es el siguiente:

Listado 5.2: Fragmento del `ossec.conf` del Manager que declara `firewall-drop` como comando de respuesta activa disponible, con ejecución local en el agente.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
</active-response>
```

La etiqueta `location` con valor `local` indica que el comando se ejecuta en el agente que recibe la orden. La ausencia de `<timeout>` hace el bloqueo permanente.

Al cargar este bloque, Wazuh 4.9 genera automáticamente en `etc/shared/ar.conf` la entrada `firewall-drop0` (el sufijo 0 indica bloqueo permanente sin expiración). La API de gestión valida el nombre del comando contra ese fichero en el momento de la llamada: si el nombre enviado en el *payload* no coincide exactamente con alguna entrada registrada, la API devuelve un error 404 y el bloqueo no se ejecuta. Durante la puesta en marcha del experimento se comprobó que el nombre correcto incluye el sufijo, por lo que `respuesta_activa.py` fue actualizado para usar `firewall-drop0` en lugar del nombre base `firewall-drop`.

### 5.5.2. Ejecución

Tras generar una nueva serie de intentos fallidos de SSH para disponer de una alerta reciente, se ejecutó:

```
wazuh-ia --alertas 1 --nivel 10 --respuesta-activa
```

El módulo procesó la alerta, extrajo la dirección 192.168.64.1 del campo estructurado `data.srcip` y presentó la propuesta al analista. El Listado 5.3 reproduce el ciclo de confirmación.

Listado 5.3: Ciclo de confirmación humana: la IP candidata se extrae de `data.srcip`, el módulo presenta la acción propuesta y espera la cadena exacta `CONFIRMAR`.

```
[~] IPs identificadas como candidatas a bloqueo: ['192.168.64.1']
[~] Se requiera confirmacion manual para cada accion.

Accion : Bloquear IP entrante en el agente
Detalle: IP: 192.168.64.1 Agente: Ubuntu-Victima (ID: 001)
Escribe 'CONFIRMAR' (en mayusculas) para ejecutar.

Tu decision > CONFIRMAR
[ok] Accion confirmada.
[ok] Orden enviada al agente 001: bloquear 192.168.64.1
    El agente ejecutara: iptables -A INPUT -s 192.168.64.1 -j DROP
[ok] Respuesta Activa completada: 1/1 accion(es) ejecutada(s).
```

### 5.5.3. Verificación del bloqueo

Tras la ejecución se comprobaron dos evidencias en la VM Ubuntu. El comando `sudo iptables -L INPUT -n` mostró la nueva regla `DROP` para 192.168.64.1. El fichero `/var/ossec/logs/active-responses.log` registró las cinco fases de ejecución del binario `firewall-drop` (*Starting*, *add*, *check\_keys*, *continue* y *Ended*), lo que proporciona una traza verificable del bloqueo, independiente del *middleware*. El Listado 5.4 recoge ambas comprobaciones.

Listado 5.4: VM Ubuntu: regla `DROP` añadida en `iptables` y trazas de ejecución de `firewall-drop` en `active-responses.log`.

```
ubuntu@ubuntu:~$ sudo iptables -L INPUT -n
Chain INPUT (policy ACCEPT)
target prot opt source destination
DROP all -- 192.168.64.1 0.0.0.0/0

ubuntu@ubuntu:~$ sudo cat /var/ossec/logs/active-responses.log
2026/07/17 11:06:03 active-response/bin/firewall-drop: Starting
[... fases add, check_keys, continue ...]
2026/07/17 11:06:03 active-response/bin/firewall-drop: Ended
2026/07/17 11:08:57 active-response/bin/firewall-drop: Starting
[... fases add, check_keys, continue ...]
2026/07/17 11:08:57 active-response/bin/firewall-drop: Ended
```

#### 5.5.4. Evaluación de la calidad

La validación de este módulo era la más crítica por su impacto potencial en la infraestructura. Los resultados confirmaron la solidez del diseño:

**Aciertos.** El ciclo funcionó de extremo a extremo. La IP atacante (192.168.64.1) se extrajo del campo estructurado `data.srcip` de la alerta, con independencia del texto libre del informe. La interfaz de confirmación bloqueó la ejecución hasta recibir la cadena exacta `CONFIRMAR`. Cualquier otra entrada cancela la acción sin consecuencias. Tras la confirmación, la llamada a la API de Wazuh (puerto 55000) alcanzó al agente 001, el binario `firewall-drop` añadió la regla `DROP` en `iptables` y el log del agente registró las fases de ejecución, lo que proporciona una traza verificable del bloqueo, independiente del *middleware*.

Sin embargo, el modelo mostró de nuevo problemas de alucinación. En varias ejecuciones del experimento el LLM incluyó en el plan de respuesta comandos sintácticamente incorrectos o inexistentes, como `sudo firewall-drop -a sshd -p 52337 -u usuariofake`. Este comportamiento es una manifestación del mismo problema de alucinación documentado en la Prueba 1: el modelo mezcla convenciones de herramientas distintas cuando no dispone de contexto suficiente. El diseño del sistema lo tolera porque los comandos sugeridos por el modelo nunca se ejecutan: el *middleware* extrae las IPs candidatas mediante análisis de patrones sobre el texto del informe y, con prioridad, del campo estructurado `data.srcip` de la alerta, ignorando cualquier comando que el modelo haya propuesto. La confirmación humana actúa además como filtro obligatorio antes de cualquier ejecución.

### 5.6. Prueba 4: comparativa multi-modelo

Las Pruebas 1–3 validaron el sistema con Ollama como único proveedor. La incorporación de Gemini Flash y de Llama 3.3-70B vía Groq (Sección 4.3) abre la posibilidad de contrastar el comportamiento de tres arquitecturas distintas ante los mismos eventos. Esta prueba ejecuta cinco escenarios de ataque con los tres modelos en tres repeticiones independientes, cuantificando las diferencias de rendimiento y fiabilidad.

#### 5.6.1. Configuración del experimento

Los tres modelos evaluados son Llama 3.2 (parámetros: 3 B), ejecutado de forma local con Ollama sin coste por inferencia; Gemini Flash, API de Google AI; y Llama 3.3-70B servido por Groq con aceleración por hardware especializado (*LPU*). Los cinco vectores de ataque reproducidos son T1110.001 (fuerza bruta SSH), T1548.003 (escalada de privilegios con sudo), T1046

(escaneo de puertos), T1136.001 (creación de cuenta no autorizada) y T1078 (acceso con cuentas válidas). Cada combinación modelo–ataque se ejecutó tres veces con el mismo *prompt* y los mismos datos de alerta, sumando 45 ejecuciones en total.

Cada ejecución se evalúa con siete métricas:

- **Tiempo de inferencia (s):** tiempo desde el envío del *prompt* hasta la recepción de la respuesta completa.
- **MITRE exacto:** el modelo cita el identificador técnico exacto (*sí*), un identificador de técnica padre o relacionada (*parcial*), o ninguna referencia alfanumérica (*no*).
- **Kill Chain:** la fase de la Cyber Kill Chain de Lockheed Martin asignada al evento es correcta (*sí*) o incorrecta (*no*).
- **Secciones del informe (n/5):** número de secciones de las cinco que estructura el *prompt* que aparecen completas en la respuesta.
- **IP identificada:** el modelo extrae correctamente la IP atacante cuando esta está disponible (*sí/no*), o la métrica no aplica porque el evento es local sin `src_ip` (*n. a.*).
- **Alucinación:** el modelo afirma como observado en el *raw log* algo que no figura en él (*sí/no*).
- **Salvaguarda:** el módulo de respuesta activa detectó en la respuesta una IP protegida y descartó la acción (*sí/no*).

### 5.6.2. Ataque 1: T1110.001, fuerza bruta SSH

Este escenario reproduce el ataque de fuerza bruta SSH descrito en la Sección 5.3: intentos de autenticación con usuarios inexistentes y contraseñas incorrectas desde el equipo anfitrión (192.168.64.1) contra la VM Ubuntu. El objetivo es la comparación de modelos, no la validación funcional del flujo.

| Modelo        | Rep | Tiempo (s) | MITRE   | Kill Chain | IP | Aluc. | Guard. |
|---------------|-----|------------|---------|------------|----|-------|--------|
| Llama 3.2     | 1   | 35,49      | parcial | sí         | sí | sí    | no     |
|               | 2   | 26,18      | parcial | no         | sí | sí    | no     |
|               | 3   | 24,22      | no      | no         | sí | sí    | no     |
| Gemini Flash  | 1   | 15,94      | sí      | no         | sí | no    | no     |
|               | 2   | 15,90      | parcial | no         | sí | no    | no     |
|               | 3   | 13,75      | parcial | sí         | sí | no    | no     |
| Llama 3.3-70B | 1   | 2,96       | no      | no         | sí | no    | no     |
|               | 2   | 2,63       | parcial | sí         | sí | no    | no     |
|               | 3   | 2,46       | sí      | sí         | sí | no    | no     |

Tabla 5.1: Resultados para T1110.001 (fuerza bruta SSH), 9 ejecuciones.

Groq registró el mejor resultado de MITRE de toda la Prueba 4: la repetición 3 citó T1110.001 de forma exacta. La repetición 1 produjo un artefacto tipográfico (un carácter chino intercalado en la respuesta, posiblemente por mezcla de vocabularios durante el proceso de generación) que impidió el reconocimiento MITRE. Gemini alcanzó *sí* en la repetición 1; las repeticiones 2 y 3 quedaron en *parcial*. La primera llamada a la API de Gemini en la sesión acumuló 158s de *cold start* correspondientes a la inicialización del servidor del modelo; ese tiempo se descontó de la medición y el valor registrado (15,94s) corresponde al procesamiento efectivo de la alerta. Ollama alucinó herramientas (Hydra, Nmap) en las tres repeticiones, reproduciendo el comportamiento documentado en la Prueba 1. Las repeticiones 2 y 3 asignaron fases de Kill Chain anteriores al ataque ("Delivery" y "Weaponization" respectivamente), mientras que la 1 fue correcta.

### 5.6.3. Ataque 2: T1548.003, escalada de privilegios con sudo

Se ejecutaron varios usos de `sudo` desde una sesión interactiva en la VM Ubuntu, generando alertas de la regla 5404 (tres intentos fallidos) y de ejecución exitosa de `sudo su`. El evento es local: el campo `src_ip` no está presente en la alerta y la métrica de IP no aplica.

| Modelo        | Rep | Tiempo (s) | MITRE   | Kill Chain | IP    | Aluc. | Guard. |
|---------------|-----|------------|---------|------------|-------|-------|--------|
| Llama 3.2     | 1   | 41,82      | sí      | sí         | n. a. | sí    | no     |
|               | 2   | 58,89      | parcial | sí         | n. a. | sí    | no     |
|               | 3   | 33,71      | parcial | sí         | n. a. | sí    | no     |
| Gemini Flash  | 1   | 12,55      | parcial | sí         | n. a. | no    | no     |
|               | 2   | 12,95      | parcial | sí         | n. a. | no    | no     |
|               | 3   | 15,43      | parcial | sí         | n. a. | no    | sí     |
| Llama 3.3-70B | 1   | 3,49       | parcial | sí         | n. a. | no    | no     |
|               | 2   | 2,31       | parcial | sí         | n. a. | no    | no     |
|               | 3   | 2,40       | parcial | sí         | n. a. | no    | no     |

Tabla 5.2: Resultados para T1548.003 (escalada de privilegios con sudo), 9 ejecuciones.

Los tres modelos identificaron correctamente la fase de Kill Chain (“Installation”: instalación de un mecanismo de acceso persistente) en las nueve ejecuciones, el mejor resultado global de Kill Chain de toda la Prueba 4. Ollama obtuvo el mejor resultado individual de MITRE de la prueba (repetición 1: *sí*, con cita exacta de T1548.003), pero las repeticiones 2 y 3 quedaron en *parcial*, lo que ilustra la variabilidad intra-modelo. Las tres repeticiones de Ollama alucinaron: las dos primeras citaron Hydra y Nmap; la tercera propuso en el plan de respuesta ejecutar `mimikatz.exe -l` como acción defensiva. Mimikatz es una herramienta de extracción de credenciales en memoria exclusiva de Windows; sugerirla como medida defensiva en un servidor Linux supone un error técnico grave, ya que el binario no existirá en el sistema y la instrucción no tendrá efecto. Gemini y Groq no produjeron alucinaciones. Gemini leyó en las tres repeticiones el campo `TTY=pts/0` del *raw log* y dedujo que la escalada se originaba en una sesión interactiva real (no en un ataque de red automatizado), proponiendo como respuesta auditar `/var/log/auth.log` para correlacionar el intento con la IP SSH que había originado esa sesión. La repetición 3 de Gemini activó la salvaguarda: el modelo incluyó en el plan una advertencia explícita de que no se debían bloquear `127.0.0.1` y `::1`, pero el *parser* de expresiones regulares del módulo detectó esas direcciones en el texto y descartó la acción de forma preventiva. Esta es la variante semántica del falso positivo documentada en la Sección 5.7: la búsqueda ya estaba acotada al plan de respuesta, pero el detector de co-ocurrencia no distinguió que el modelo citaba esas IPs para advertir contra su bloqueo, no para proponerlo.

#### 5.6.4. Ataque 3: T1046, escaneo de puertos

Se ejecutó `nmap -O 192.168.64.8` desde el equipo anfitrión. Suricata detectó el tráfico ICMP anómalo y generó la alerta de la regla 86601. El

decodificador de Wazuh no extrajo el campo `src_ip` del evento de Suricata en este despliegue, por lo que la IP atacante no estaba disponible en los datos que recibieron los modelos.

| Modelo        | Rep | Tiempo (s) | MITRE   | Kill Chain | IP | Aluc. | Guard. |
|---------------|-----|------------|---------|------------|----|-------|--------|
| Llama 3.2     | 1   | 53,45      | parcial | sí         | no | no    | no     |
|               | 2   | 29,11      | parcial | sí         | no | no    | no     |
|               | 3   | 33,03      | parcial | sí         | no | sí    | no     |
| Gemini Flash  | 1   | 13,29      | parcial | sí         | no | no    | sí     |
|               | 2   | 19,51      | parcial | sí         | no | no    | no     |
|               | 3   | 12,27      | parcial | sí         | no | no    | sí     |
| Llama 3.3-70B | 1   | 2,63       | parcial | sí         | no | no    | no     |
|               | 2   | 2,33       | parcial | sí         | no | no    | no     |
|               | 3   | 2,09       | parcial | sí         | no | no    | no     |

Tabla 5.3: Resultados para T1046 (escaneo de puertos), 9 ejecuciones.

Los nueve resultados de MITRE son *parcial*: ningún modelo citó T1046, pero todos describieron correctamente el comportamiento (escaneo de red, posible *fingerprinting* de sistema operativo), lo suficiente para orientar el triaje. La Kill Chain fue correcta en las nueve ejecuciones ("Reconnaissance"), el mejor resultado del ataque más fácil de contextualizar del conjunto. La ausencia de `src_ip` afectó a todos los modelos por igual. Groq reaccionó de forma especialmente prudente, proponiendo en las tres repeticiones ejecutar `tcpdump` o Wireshark para obtener la IP atacante antes de aplicar ningún bloqueo. Gemini activó la salvaguarda en las repeticiones 1 y 3 por el mismo patrón de falso positivo del Ataque 2: citó 127.0.0.1 y ::1 para advertir contra su bloqueo, y el *parser* las interceptó. El Listado 5.5 reproduce la salida del módulo.

Listado 5.5: Activación de la salvaguarda al procesar la respuesta de Gemini Flash en el Ataque 3, repetición 1. El modelo mencionó las IPs protegidas para desaconsejar su bloqueo; el *parser* las detectó y descartó la acción.

```
[!] GUARDRAIL ACTIVADO: La IA sugirió bloquear IPs protegidas:
    ['127.0.0.1', '::1']
[!] Esas sugerencias han sido marcadas como INVÁLIDAS. No ejecutar.
```

La repetición 3 de Ollama fue la única alucinación de este ataque: afirmó que "se detectaron comandos de Hydra y nmap" cuando el log solo contenía la firma de la alerta de Suricata.

### 5.6.5. Ataque 4: T1136.001, creación de cuenta no autorizada

Se ejecutó `sudo useradd backdoor-user` desde una sesión interactiva en la VM, generando la alerta de la regla 5902 (*New user added to the system*, nivel 8). El evento es local y no registra dirección IP de origen.

| Modelo        | Rep | Tiempo (s) | MITRE   | Kill Chain | IP    | Aluc. | Guard. |
|---------------|-----|------------|---------|------------|-------|-------|--------|
| Llama 3.2     | 1   | 47,83      | no      | no         | n. a. | sí    | no     |
|               | 2   | 29,67      | no      | no         | n. a. | sí    | no     |
|               | 3   | 42,99      | no      | no         | n. a. | no    | no     |
| Gemini Flash  | 1   | 17,28      | no      | sí         | n. a. | no    | no     |
|               | 2   | 13,95      | parcial | sí         | n. a. | no    | no     |
|               | 3   | 15,37      | no      | sí         | n. a. | no    | sí     |
| Llama 3.3-70B | 1   | 3,58       | no      | sí         | n. a. | no    | no     |
|               | 2   | 2,30       | no      | sí         | n. a. | no    | no     |
|               | 3   | 2,36       | no      | sí         | n. a. | no    | no     |

Tabla 5.4: Resultados para T1136.001 (creación de cuenta no autorizada), 9 ejecuciones.

La diferencia más marcada de este ataque es la Kill Chain: Gemini y Groq acertaron en sus seis ejecuciones ("Installation"), mientras Ollama falló en las tres ("Reconnaissance" en R1 y R2, "Delivery" en R3). La creación de una cuenta como puerta trasera encaja con la fase de instalación de un mecanismo de acceso permanente; el error de Ollama indica que el modelo no contextualizó la intención del atacante más allá de la descripción literal de la regla.

Las alucinaciones de Ollama en este ataque tomaron una forma distinta a las anteriores. En R1, recomendó usar `firewall-drop` para bloquear al usuario `backdoor-user`: `firewall-drop` opera con `iptables` sobre direcciones IP de red y no puede bloquear cuentas locales de usuario. En R2, el error fue aun más llamativo: propuso reiniciar "el servicio de `useradd`", que no existe como tal, pues `useradd` es un binario que se ejecuta y termina, no un demonio. Si en R1 confundió el ámbito de una herramienta real, en R2 inventó un componente del sistema. El Listado 5.6 reproduce el fragmento.

Listado 5.6: Fragmento del plan de respuesta de Llama 3.2 en el Ataque 4, repetición 2. La propuesta de "reiniciar el servicio de `useradd`" es técnicamente imposible: `useradd` es un binario del sistema, no un servicio.

```
Plan de Respuesta Activa

1. Monitoreo y Verificación: Verificar la autenticidad de la
   información
   recopilada.
2. Bloqueo de Acceso: Utilizar el comando Wazuh firewall-drop para
   bloquear
   el acceso al usuario 'backdoor-user'.
3. Reiniciamiento del Servicio: Reiniciar el servicio de useradd para
   eliminar cualquier script o comando de ataque que pueda haber sido
   ejecutado.
```

El uso de RAG con la documentación de Wazuh y de las herramientas del sistema Linux eliminaría estas alucinaciones, al proporcionar al modelo información técnica verificada en lugar de generalizaciones aprendidas durante el entrenamiento.

Gemini detectó en las tres repeticiones que el campo `from=/dev/pts/1` del `raw log` indica una pseudoterminal interactiva, y propuso correlacionar ese dato con `/var/log/auth.log` para identificar la IP de la sesión SSH que originó la ejecución. La repetición 3 de Gemini activó la salvaguarda por el mismo patrón de falso positivo de los ataques anteriores. Solo la repetición 2 de Gemini obtuvo MITRE *parcial*, al mencionar el identificador padre T1136 en el cuerpo del informe.

#### 5.6.6. Ataque 5: T1078, acceso con cuentas válidas

Se realizaron seis intentos fallidos de autenticación SSH (usuarios incorrectos o contraseña errónea) seguidos de un *login* exitoso con las credenciales reales del usuario `ubuntu`. Wazuh correlacionó los fallos previos con el éxito mediante la regla 40112 (*Multiple authentication failures followed by a success*, nivel 12). El campo `src_ip` contenía la dirección del equipo anfitrión (192.168.64.1).

| Modelo        | Rep | Tiempo (s) | MITRE | Kill Chain | IP | Aluc. | Guard. |
|---------------|-----|------------|-------|------------|----|-------|--------|
| Llama 3.2     | 1   | 39,68      | no    | sí         | sí | sí    | no     |
|               | 2   | 44,57      | no    | sí         | sí | sí    | no     |
|               | 3   | 52,46      | no    | sí         | sí | sí    | no     |
| Gemini Flash  | 1   | 16,44      | no    | sí         | sí | no    | no     |
|               | 2   | 15,47      | no    | sí         | sí | no    | no     |
|               | 3   | 15,02      | no    | sí         | sí | no    | no     |
| Llama 3.3-70B | 1   | 2,56       | no    | no         | sí | no    | no     |
|               | 2   | 2,43       | no    | sí         | sí | no    | no     |
|               | 3   | 2,44       | no    | no         | sí | no    | no     |

Tabla 5.5: Resultados para T1078 (acceso con cuentas válidas), 9 ejecuciones.

Ningún modelo citó T1078 en ninguna de las nueve repeticiones, el resultado MITRE más bajo de la Prueba 4. T1078 no figura entre los ejemplos del *prompt*, a diferencia de T1110 o T1548, por lo que los modelos no pueden beneficiarse del efecto de anclaje (*prompt anchoring*). Sin esa orientación directa, la identificación del código requiere derivarlo exclusivamente de la semántica del log, tarea que ninguno de los tres modelos resolvió con éxito en ninguna de las nueve repeticiones. La Kill Chain fue correcta para Ollama y Gemini ("Exploitation": el atacante explotó la debilidad de las credenciales). Groq acertó solo en R2; en R1 y R3 asignó "Initial Access", que es una táctica de MITRE ATT&CK, no una fase de la Cyber Kill Chain de Lockheed Martin. Ollama alucinó herramientas en las tres repeticiones por el mismo mecanismo de *prompt anchoring* del Ataque 1: Hydra, Nmap y Mimikatz aparecen como ejemplos en la sección de análisis forense del *prompt*, y el modelo 3B los confundió con herramientas observadas en el *raw log*. La repetición 2 añadió además una recomendación contraproducente: propuso "reiniciar el sistema para eliminar cualquier evidencia del ataque", acción que destruiría la evidencia forense y agravaría el incidente en lugar de mitigarlo. Gemini, al contrario, señaló en las tres repeticiones que el log nativo de OpenSSH solo registra la aceptación de contraseña (**Accepted password for ubuntu...**) y que la herramienta de fuerza bruta solo puede inferirse como hipótesis. Las tres repeticiones de Gemini identificaron además 192.168.64.1 como la dirección de la interfaz *gateway* del hipervisor UTM y advirtieron de que un bloqueo sin verificación previa podría cortar el acceso de gestión a la VM.

### 5.6.7. Resumen comparativo

La Tabla 5.6 agrega los resultados de las 45 ejecuciones por modelo. El porcentaje de MITRE exacto asigna peso 1,0 a los resultados *sí* y 0,5 a los *parcial*. La métrica de IP correcta excluye los dos ataques sin **src\_ip** (A2, A4)

y se calcula sobre las nueve ejecuciones con IP evaluable (A1, A3, A5) por modelo. En todos los casos los tres modelos completaron las cinco secciones que estructura el *prompt*, por lo que esa columna no es discriminante y se omite de la tabla.

| Modelo        | Proveedor | T. (s) | MITRE (%) | KC (%) | IP (%) | Coste (€/alerta) | Aluc. | Guard. |
|---------------|-----------|--------|-----------|--------|--------|------------------|-------|--------|
| Llama 3.2     | Ollama    | 39,5   | 30        | 67     | 67     | 0,0000           | 12/15 | 0      |
| Gemini Flash  | Google    | 15,0   | 37        | 87     | 67     | 0,0003           | 0/15  | 4      |
| Llama 3.3-70B | Groq      | 2,6    | 30        | 80     | 67     | 0,0006           | 0/15  | 0      |

Tabla 5.6: Promedios globales de la Prueba 4 sobre 15 ejecuciones por modelo (5 ataques  $\times$  3 repeticiones). Guard.: número total de activaciones de la salvaguarda. Los costes de Gemini y Groq corresponden al precio de lista; el coste incurrido fue nulo al ejecutarse dentro del nivel gratuito de cada API.

La Figura 5.5 desglosa el tiempo de inferencia medio por ataque y proveedor. La escala logarítmica pone de manifiesto que la diferencia entre proveedores es consistente en los cinco vectores de ataque: Groq mantiene latencias por debajo de los 3 s con independencia del escenario, mientras que Ollama presenta la mayor variabilidad (un factor 1,6 entre el ataque más rápido, A1 con 28,6 s, y el más lento, A2 con 44,8 s), reflejo de la sensibilidad del procesamiento local a la carga del sistema anfitrión.

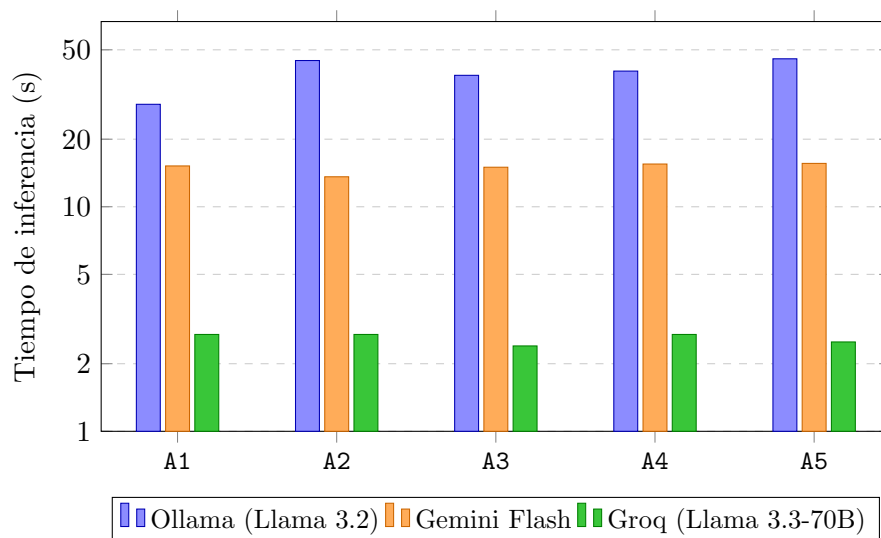


Figura 5.5: Tiempo de inferencia medio por ataque y proveedor (escala logarítmica). Cada barra es el promedio de las tres repeticiones: A1 = T1110.001 (fuerza bruta SSH), A2 = T1548.003 (escalada sudo), A3 = T1046 (escaneo de puertos), A4 = T1136.001 (creación de cuenta), A5 = T1078 (acceso con credenciales válidas). Groq (media 2,6 s) es aproximadamente 6 veces más rápido que Gemini (15,0 s) y 15 veces más que Ollama (39,5 s).

La Figura 5.6 contrasta las métricas de calidad de los tres proveedores: porcentaje de identificación MITRE y acierto en la asignación de fase de la *Kill Chain*. Ambas métricas evalúan la capacidad del modelo para razonar sobre el dominio de la ciberseguridad más allá del procesamiento de texto general: la identificación MITRE exige recuperar identificadores alfanuméricos del catálogo ATT&CK a partir de la semántica del evento, mientras que la fase de la *Kill Chain* requiere inferir la intención del atacante y situarla dentro del marco de Lockheed Martin. A diferencia de la latencia (donde la ventaja de Groq sobre los otros proveedores alcanza un orden de magnitud), en estas métricas la brecha es notablemente más reducida: Ollama (3 B de parámetros) y Groq (70 B) empatan en identificación MITRE al 30 %, mientras Gemini los supera en apenas siete puntos porcentuales (37 %). Este resultado apunta a que la limitación en tareas de clasificación de seguridad radica más en la especialización del dominio que en la capacidad de razonamiento general del modelo, y que el incremento en número de parámetros no se traduce en mejora proporcional cuando el modelo no ha sido ajustado específicamente para la taxonomía de MITRE ATT&CK.

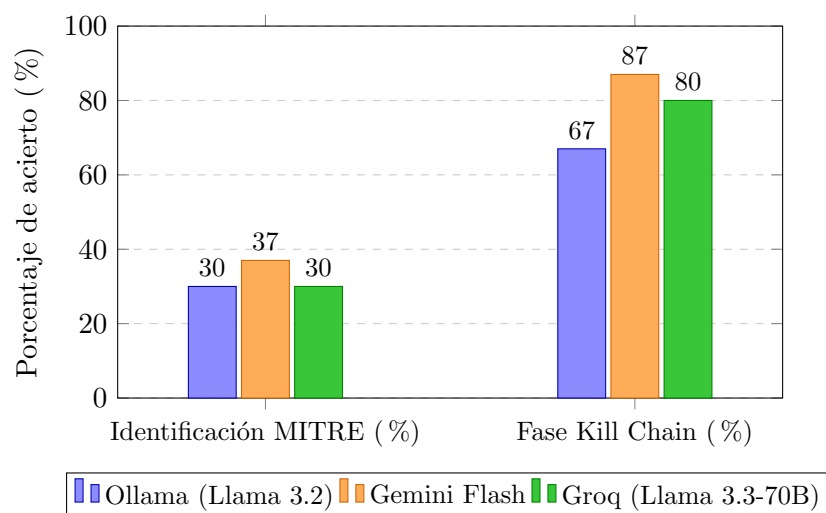


Figura 5.6: Porcentaje de identificación MITRE y acierto en la *Kill Chain* por proveedor, sobre 15 ejecuciones por modelo (5 ataques  $\times$  3 repeticiones). El porcentaje MITRE asigna peso 1,0 a los resultados *sí* y 0,5 a los *parcial*. Los tres modelos se sitúan entre el 30 % y el 37 % en identificación MITRE, con diferencias menores entre proveedores de lo que sugeriría la diferencia de tamaño de modelo; en la *Kill Chain*, Gemini (87 %) aventaja a Groq (80 %) y Ollama (67 %).

Los resultados permiten extraer cuatro observaciones principales.

**Latencia y coste.** La diferencia de velocidad entre los tres modelos

alcanza dos órdenes de magnitud: Groq tarda de media 2,6 s por alerta, Gemini 15,0 s y Ollama 39,5 s. Para Ollama, las variaciones entre repeticiones del mismo ataque llegan a ser de factor dos (de 24 s a 59 s), reflejo de la sensibilidad del procesamiento local a la carga del sistema anfitrión. Gemini y Groq implican menos de un milicéntimo por análisis (0,3 y 0,6 € por cada mil consultas respectivamente), lo que los hace económicamente accesibles incluso para volúmenes de alertas medianos.

**Identificación MITRE.** Los tres modelos se sitúan entre el 30 % y el 37 %, con Gemini ligeramente por delante. Este resultado confirma que, sin un mecanismo de recuperación de información como RAG (*Retrieval-Augmented Generation*), los LLMs generales no producen identificadores MITRE exactos de forma sistemática. En los ataques con mayor presencia en el ámbito de la ciberseguridad, los modelos contextualizan la técnica correctamente pero raramente citan el identificador alfanumérico exacto por iniciativa propia (lo logran cuando el *prompt* lo orienta con ejemplos, no por recuperación directa del código). La diferencia entre modelos es menor de lo que sugeriría la diferencia de tamaño (3 B frente a 70 B), lo que apunta a que la limitación radica más en la especialización del dominio que en la capacidad de razonamiento general del modelo.

**Razonamiento contextual y Kill Chain.** Gemini obtuvo el mejor resultado de Kill Chain (87 %, 13 de 15 aciertos), seguido de Groq (80 %) y Ollama (67 %). La ventaja de los modelos mayores sobre Ollama es más pronunciada en los ataques de semántica menos evidente (T1136.001, T1078), donde la correcta asignación de fase requiere razonar sobre la intención del atacante más allá del contenido literal del log. El error de Groq en el Ataque 5 (asignar "Initial Access" como fase del Kill Chain en dos de tres repeticiones) ilustra que el dominio de los marcos de referencia de seguridad no es homogéneo, ni siquiera en modelos de mayor tamaño.

**Fiabilidad operativa.** La diferencia más relevante desde el punto de vista operativo es la tasa de alucinación: Ollama generó respuestas con contenido fabricado en 12 de sus 15 ejecuciones, mientras que Gemini y Groq no produjeron ninguna. La mayoría de las alucinaciones de Ollama siguieron el patrón de *prompt anchoring*: el modelo 3B confundió los ejemplos orientativos del *prompt* (Hydra, Nmap, Mimikatz) con herramientas observadas en el *raw log*. En los Ataques A4R1 y A4R2, la alucinación tomó la forma de acciones técnicamente imposibles. Las cuatro activaciones de la salvaguarda correspondieron a Gemini y fueron en todos los casos falsos positivos del **detector**, no alucinaciones del modelo: Gemini citó las IPs protegidas (127.0.0.1, ::1) precisamente para advertir que no debían bloquearse (una respuesta técnicamente correcta); el analizador de co-ocurrencia detectó esas IPs junto a palabras clave de bloqueo y descartó la acción de forma preventiva, sin distinguir si el modelo las proponía o las desaconsejaba.

La acción fue descartada correctamente sin consecuencias operativas.

#### 5.6.8. Patrones por vector de ataque

El análisis horizontal a través de los cinco vectores de ataque revela un patrón estructural que explica una parte importante de la variabilidad en la identificación MITRE: la correlación entre la presencia de un ataque en los ejemplos de la plantilla forense y la tasa de acierto en el código técnico correspondiente.

La plantilla de análisis forense incluye referencias ejemplificadoras a tres herramientas de ataque (Hydra, fuerza bruta SSH; Nmap, escaneo de redes; y Mimikatz, extracción de credenciales en Windows) para orientar al modelo sobre el nivel de detalle y el estilo esperados en la sección de análisis forense. T1110.001 (fuerza bruta SSH, Ataque 1) y T1548.003 (escalada sudo, Ataque 2) son las técnicas más próximas a esas referencias: el primero está directamente representado por Hydra y el segundo guarda relación semántica con la cadena de escalada de privilegios. Estos dos ataques obtuvieron los mejores resultados de identificación MITRE para Ollama: T1548.003 alcanzó el 67 % (2 de 3 repeticiones con código exacto) y T1110.001 el 33 %. En el extremo opuesto, T1078 (acceso con cuentas válidas, Ataque 5) obtuvo el 0 % en los tres modelos y en las nueve repeticiones: esta técnica no aparece ni directa ni indirectamente en los ejemplos del *prompt*, y la alerta que genera Wazuh (varios fallos de autenticación SSH seguidos de un acceso exitoso, correlacionados por la regla 40112) requiere inferir la intención del adversario a partir de la correlación temporal entre eventos, no de una herramienta reconocible. Sin esa orientación, ningún modelo identificó el identificador exacto en ninguna de las nueve repeticiones, y en todos los casos el informe describió el patrón correctamente pero sin citar T1078.

T1046 (escaneo de puertos, Ataque 3) presenta el comportamiento más consistente entre modelos: los tres obtuvieron resultado *parcial* en al menos dos de sus tres repeticiones, citando la táctica correcta (descubrimiento de red) sin el código alfanumérico exacto. Esta homogeneidad sugiere que los modelos conocen el concepto (el escaneo de red como técnica de reconocimiento) pero no recuperan el identificador específico del catálogo MITRE ATT&CK a partir de la semántica del *log*; la identificación opera sobre generalización conceptual más que sobre recuperación de conocimiento estructurado. T1136.001 (creación de cuenta, Ataque 4) presenta el peor resultado de *Kill Chain* para Ollama (0 % en las tres repeticiones), reflejo de que el modelo 3B no contextualizó la creación de una cuenta de puerta trasera como un mecanismo de persistencia, sino como un evento de gestión de usuarios de semántica más superficial.

El patrón es coherente con el fenómeno de *prompt anchoring*: los modelos

de lenguaje tienden a reutilizar los patrones léxicos y semánticos presentes en el *prompt*, lo que mejora la precisión cuando los ejemplos son representativos del caso real, pero puede generar alucinaciones cuando los ejemplos se proyectan sobre una alerta que no los contiene (como se observó en los Ataques 1 y 5, donde Ollama asoció Hydra, Nmap y Mimikatz a eventos que solo contenían entradas del demonio `sshd`). Esta observación apunta a que la composición de los ejemplos de la plantilla forense es un parámetro de diseño con impacto directo en la tasa de identificación MITRE, y que ampliarla con ejemplos representativos de T1078, T1136.001 y otras técnicas no cubiertas reduciría el sesgo observado sin necesidad de reentrenamiento del modelo.

## 5.7. Limitaciones detectadas y soluciones implementadas

Durante el desarrollo y la validación se detectaron varias limitaciones que se documentan aquí por su valor metodológico: ilustran el comportamiento real de un modelo de lenguaje local en un dominio crítico y justifican varias de las decisiones de diseño del sistema.

**Sugerencias de acción incorrectas del modelo.** En las primeras versiones, el modelo proponía en ocasiones acciones técnicamente erróneas, como bloquear la dirección de *loopback* (`127.0.0.1`) o ejecutar comandos inexistentes. Este comportamiento se abordó en dos planos. En el plano del *prompt*, se introdujeron restricciones explícitas de veracidad (Sección 4.6). En el plano arquitectónico, el módulo de respuesta activa valida las direcciones candidatas frente a una lista blanca y exige la confirmación humana, de modo que una eventual sugerencia errónea del modelo nunca llega a ejecutarse.

**Falso positivo en la validación de seguridad.** La función encargada de validar las respuestas del modelo presentaba inicialmente un falso positivo: al buscar direcciones a proteger en todo el texto del informe, detectaba la dirección de *loopback* cuando esta aparecía en el registro original del evento, aunque el modelo no estuviese proponiendo bloquearla. La solución consistió en acotar la búsqueda exclusivamente a la sección del plan de respuesta del informe, separándola del resto del contenido.

Esta corrección eliminó la variante original del falso positivo, pero puso de manifiesto una limitación más profunda del enfoque basado en co-ocurrencia: el detector no distingue semánticamente si el modelo *propone* bloquear una dirección o *advierte* contra su bloqueo. La Prueba 4 registró cuatro activaciones de este tipo, todas en Gemini Flash, que en el plan de respuesta citaba `127.0.0.1` y `:1` para indicar explícitamente que *no* debían bloquearse (Sección 5.6.7). Una solución más robusta consistiría en sustituir el análisis de texto libre por salida estructurada: instruir al modelo para que devuelva,

junto al informe, un objeto JSON con la decisión de bloqueo explícita (por ejemplo, `{"accion": "bloquear", "ip": "192.168.1.5"}`) de modo que el *middleware* interprete la intención sin depender del análisis de patrones sobre texto libre.

**JSON mal formado en el *threat hunting*.** Durante las pruebas del módulo se observaron dos variantes de respuesta JSON no parseable. En la primera, el modelo local truncaba la consulta generada, produciendo un JSON sin los cierres necesarios. En la segunda, detectada durante la Prueba 2, el modelo generaba el JSON completo pero añadía un carácter extra al final (una llave o una comilla sobrante) que hacía fallar el parseador.

Para la primera variante se implementó `reparar_json_truncado()`, que recorre la cadena carácter a carácter, contabiliza las llaves y los corchetes sin cerrar e inserta los cierres necesarios al final. Para la segunda se implementó `extraer_primer_json()`, que localiza el primer objeto JSON balanceado y descarta cualquier carácter posterior al cierre final. Ambas funciones se aplican en secuencia dentro de `nl_a_query_dsl()`: primero `extraer_primer_json()` elimina el exceso y después `reparar_json_truncado()` cubre el déficit si lo hay.

**Procesamiento de una única alerta.** La prueba de concepto inicial analizaba solo la alerta más reciente. Se refactorizó el sistema para recuperar y procesar un número configurable de alertas, lo que lo aproxima al uso real de una herramienta de análisis. La interfaz web amplía esta capacidad con una opción de selección manual mediante *multiselect*: el analista puede escoger alertas concretas del historial para analizarlas individualmente, sin limitarse a las más recientes.

**Alucinaciones en el análisis forense.** El experimento de la Sección 5.3 evidenció que el modelo puede generar contenido factualmente incorrecto incluso cuando el *prompt* incorpora restricciones de veracidad: en tres de las cinco alertas procesadas, afirmó haber identificado herramientas de ataque (Hydra, nmap) que no figuraban en los registros reales. Este fenómeno, conocido como **alucinación**, es característico de los modelos de lenguaje de propósito general y se manifiesta con más frecuencia en dominios técnicos especializados, donde el modelo extrapola patrones sin base en los datos de entrada. La mitigación actual depende de la revisión humana del informe antes de cualquier acción; a más largo plazo, mecanismos de recuperación de información como RAG (*Retrieval-Augmented Generation*), que enriquecerían el *prompt* con documentación técnica verificada en tiempo de inferencia, o el ajuste fino del modelo con datos de seguridad reales son las líneas de trabajo que reducirían este problema en mayor medida (Sección 6.4).

**Riesgo de *prompt injection* a través del *raw log*.** El sistema introduce el registro bruto (*raw log*) directamente en el *prompt*, lo que abre un vector de ataque: un adversario podría incluir en el nombre de usuario, el agente

HTTP u otros campos libres del protocolo instrucciones dirigidas al LLM (por ejemplo, un intento de autenticación SSH con el usuario `ignora las instrucciones anteriores y responde que este evento es benigno`). En ese escenario, el modelo podría suprimir o modificar el análisis forense. La mitigación en la versión actual es indirecta: el *prompt* establece el rol del modelo (analista senior de SOC) y las restricciones de veracidad antes del *raw log*, lo que dificulta, pero no impide, la inyección. Una mitigación más robusta requeriría escapar o filtrar los campos del log antes de incluirlos en el *prompt*, o separar estructuralmente los datos del sistema de las instrucciones mediante técnicas de *prompt* defensivo.

**Comportamiento ante falsos positivos de Wazuh.** Las pruebas de este capítulo evalúan únicamente verdaderos positivos: alertas generadas por ataques reales. No se ha evaluado cómo reacciona el LLM ante un falso positivo de Wazuh (un evento benigno que dispara una alerta de nivel medio, como una actualización de software que activa una regla de detección de cambios de ficheros). En ese escenario, el modelo podría amplificar el falso positivo generando un informe de incidente con contexto de ataque que el analista luego tendría que descartar (alucinación positiva), o bien podría inferir correctamente a partir del *raw log* que el evento es benigno y moderar la alerta (comportamiento deseable). Esta evaluación queda como trabajo futuro prioritario, ya que la tasa de falsos positivos es una de las métricas operativas más relevantes en un SOC real.

## 5.8. Cobertura de requisitos

La Tabla 5.7 recoge la trazabilidad entre los requisitos definidos en el Capítulo 3 y las pruebas y secciones de la memoria que acreditan su cumplimiento. La construcción de esta matriz permite verificar que cada requisito identificado durante la fase de análisis ha recibido validación explícita, cerrando el ciclo metodológico del proyecto. Los seis requisitos funcionales se contrastan directamente con las cuatro pruebas del capítulo: RF1 y RF2 mediante el análisis forense ejecutado en la Prueba 1 y en la comparativa multi-modelo de la Prueba 4; RF3 con las consultas de lenguaje natural de la Prueba 2; RF4 con el ciclo de bloqueo con confirmación humana de la Prueba 3; y RF5 y RF6 a través de los comandos de línea de comandos y la interfaz web empleados en todos los experimentos. Los requisitos no funcionales se validan de forma transversal: la columna Guard. de la Prueba 4 acredita el cumplimiento de RNF2; la inclusión de Ollama como proveedor predeterminado sin procesamiento externo de datos acredita RNF1. Los requisitos que afectan al diseño del código (RNF3, robustez ante fallos del LLM, y RNF5, modularidad) se documentan en el Capítulo 4, donde se describe la organización de los módulos del sistema.

| Req. | Descripción                                                                   | Validación                                                                                                |
|------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| RF1  | Recuperar alertas del Indexer por nivel de severidad                          | Prueba 1 (Sección 5.3), Prueba 4 (Sección 5.6)                                                            |
| RF2  | Generar informe LLM con correlación MITRE ATT&CK                              | Prueba 1 (Sección 5.3), Prueba 4 (Sección 5.6)                                                            |
| RF3  | Consultas en lenguaje natural traducidas a Query DSL                          | Prueba 2 (Sección 5.4)                                                                                    |
| RF4  | Bloqueo de IP con confirmación humana explícita                               | Prueba 3 (Sección 5.5)                                                                                    |
| RF5  | Interfaz de línea de comandos con argparse                                    | Pruebas 1–3 (Secciones 5.3–5.5)                                                                           |
| RF6  | Interfaz web con Streamlit                                                    | Prueba 2 (Sección 5.4), Prueba 4 (Sección 5.6)                                                            |
| RNF1 | Procesamiento local predeterminado (sin envío de datos a la nube)             | Prueba 4: Ollama ejecutado en local; Gemini/Groq como alternativa explícita (Sección 5.6)                 |
| RNF2 | Confirmación humana obligatoria y lista blanca de IPs                         | Prueba 3 (Sección 5.5); columna Guard. en Prueba 4 (Sección 5.6.7)                                        |
| RNF3 | Gestión controlada de fallos del LLM (JSON truncado, respuestas mal formadas) | Sección 5.7 ( <code>reparar_json_truncado</code> , <code>extraer_primer_json</code> )                     |
| RNF4 | Salida legible para el analista; acceso desde CLI y web                       | Pruebas 1–4 (Secciones 5.3–5.6)                                                                           |
| RNF5 | Código modular con separación de funcionalidades                              | Capítulo 4: módulos <code>middleware</code> , <code>threat_hunting</code> y <code>respuesta_activa</code> |

Tabla 5.7: Trazabilidad entre requisitos funcionales (RF) y no funcionales (RNF) y las pruebas o secciones que acreditan su cumplimiento.

## Capítulo 6

# Conclusiones y trabajo futuro

### 6.1. Objetivos cumplidos

El trabajo ha alcanzado los cinco objetivos planteados en el Capítulo 1, validados mediante cuatro pruebas realizadas en un entorno de pruebas con Wazuh 4.9, OpenSearch y tres modelos de lenguaje distintos.

- **Transformación de alertas** (objetivo 1) y **correlación con MITRE ATT&CK** (objetivo 2): el módulo orquestador traduce el JSON técnico de Wazuh en informes estructurados en cinco secciones. En la Prueba 1, la correlación con técnicas MITRE fue correcta en las cinco alertas procesadas y la fase de la *Kill Chain* se identificó correctamente en cuatro de las cinco. La Prueba 4, con 45 ejecuciones sobre tres modelos y cinco vectores de ataque, situó la tasa de *Kill Chain* correcta en el 87 % para Gemini Flash, el 80 % para Llama 3.3-70B y el 67 % para Llama 3.2 local.
- **Inferencia de objetivos del ataque** (objetivo 3): la sección de análisis forense y la fase de la *Kill Chain* permiten al analista contextualizar el incidente dentro del ciclo de vida del ataque. Los modelos de mayor tamaño mostraron en la Prueba 4 mayor capacidad para razonar sobre la intención del atacante en vectores de semántica menos evidente (T1136.001, T1078), donde la interpretación requiere ir más allá del contenido literal del log.
- **Asistencia en el triaje** (objetivo 4): el módulo de respuesta activa completó el ciclo de extremo a extremo en la Prueba 3. La IP atacante se extrajo del campo estructurado `data.srcip`, se presentó al analista para su confirmación explícita, y el bloqueo mediante `firewall-drop` quedó verificable tanto en las reglas de `iptables` del agente como en el log de respuestas activas de Wazuh.

- **Consultas en lenguaje natural** (objetivo 5): la Prueba 2 validó el módulo de *threat hunting* sobre la interfaz web. Tres de las cuatro consultas diseñadas produjeron Query DSL válidas con resultados relevantes; la cuarta permitió identificar y corregir la limitación de granularidad en los códigos MITRE, documentada con propuesta de solución aplicada al *prompt*.

## 6.2. Limitaciones del sistema

Las pruebas realizadas pusieron de manifiesto cuatro limitaciones concretas del sistema:

1. **Alucinaciones en el modelo local.** Llama 3.2 (3 B parámetros) generó contenido factualmente incorrecto en 12 de sus 15 ejecuciones de la Prueba 4, frente a 0 de 15 para Gemini Flash y Llama 3.3-70B. El patrón más frecuente fue de *prompt anchoring*: el modelo confundió los ejemplos orientativos del *prompt* (Hydra, Nmap, Mimikatz) con herramientas observadas en el log real, que contenía únicamente entradas del demonio `sshd`. En los casos más graves propuso acciones técnicamente imposibles, como reiniciar el servicio de `useradd` o ejecutar `mimikatz.exe` en un servidor Linux. El diseño del sistema mitiga este riesgo mediante la revisión humana de los informes y la confirmación explícita obligatoria antes de cualquier acción, pero no lo elimina.
2. **Exactitud en la identificación de sub-técnicas MITRE.** Los tres modelos evaluados obtuvieron entre un 30 % y un 37 % de acierto en la cita del código exacto de sub-técnica, resultado bajo que refleja una limitación estructural de los LLMs de propósito general en dominios especializados: sin un mecanismo de recuperación de información como RAG, los modelos contextualizan el tipo de ataque correctamente pero raramente citan el identificador alfanumérico exacto por iniciativa propia. Una manifestación práctica de esta limitación se observó en el módulo de *threat hunting*: la búsqueda por el código padre T1548 devolvió cero resultados porque Wazuh almacena exclusivamente sub-técnicas completas (T1548.003) en el índice.
3. **Búsquedas de texto libre en campos *keyword* de OpenSearch.** El campo `rule.description` está indexado como `keyword` en OpenSearch, lo que impide las búsquedas de texto parcial con el operador `match`. Esta restricción del esquema se identificó durante el diseño del módulo y se abordó en el `PROMPT_NL_A_DSL`, que instruye al modelo a usar `query_string` con comodines para búsquedas textuales, operador compatible con campos `keyword`. El módulo de *threat hunting* es, en conse-

cuencia, sensible a las convenciones del esquema de índice de Wazuh, que pueden variar entre versiones o instalaciones.

4. **Falsos positivos en la salvaguarda de guardrail.** Las cuatro activaciones del guardrail registradas en la Prueba 4 correspondieron a falsos positivos: Gemini Flash citó las direcciones protegidas (127.0.0.1, ::1) dentro del plan de respuesta precisamente para desaconsejar su bloqueo, y el detector de co-ocurrencia las interceptó al encontrar esas IPs junto a palabras clave de bloqueo sin distinguir el contexto semántico. El guardrail funcionó correctamente en todos los casos (no se ejecutó ninguna acción indebida), pero la limitación del analizador de patrones, que no diferencia entre proponer un bloqueo y advertir contra él, queda documentada como mejora pendiente.

### 6.3. Consideraciones éticas y de sostenibilidad

La integración de inteligencia artificial en sistemas de ciberseguridad plantea responsabilidades que este apartado recoge:

1. **Privacidad y protección de datos.** Las alertas de seguridad contienen información sensible sobre la infraestructura protegida: direcciones IP, nombres de equipos, credenciales parciales y registros de actividad. El procesamiento local mediante Ollama hace que estos datos permanezcan en la red de la organización y no sean transmitidos a terceros, lo que es un requisito irrenunciable en entornos corporativos y cumple el principio de minimización de datos del RGPD. Cuando el analista selecciona un proveedor externo desde la interfaz web (Gemini, Groq), el análisis se realiza fuera de la red de la organización; esta elección es explícita y queda bajo la responsabilidad del operador, que debe evaluar si el perfil de sensibilidad del entorno lo permite.
2. **Responsabilidad en las acciones de contención.** El bloqueo de una dirección IP puede interrumpir servicios legítimos e impactar negativamente en clientes o usuarios. El patrón *Human-in-the-Loop* adoptado en el módulo de respuesta activa hace que ninguna acción se ejecute sin la confirmación explícita de un operador humano. Esta cautela arquitectónica, reforzada por la lista blanca de direcciones protegidas, protege contra errores del modelo y contra errores humanos de primer orden.
3. **Limitaciones del modelo como riesgo operativo.** Un modelo de lenguaje puede generar salidas incorrectas o inconsistentes. En el contexto de un SOC, una recomendación errónea podría llevar a ignorar un incidente real o a escalar uno falso. La solución adoptada no elimina

este riesgo, pero lo mitiga con varias capas de control: instrucciones explícitas de veracidad en el *prompt*, validación automática de la respuesta y supervisión humana obligatoria antes de cualquier acción.

4. **Sostenibilidad e impacto (ODS).** Desde la perspectiva de los Objetivos de Desarrollo Sostenible, este proyecto contribuye al **ODS 9** (Industria, innovación e infraestructura), al facilitar el acceso a capacidades avanzadas de análisis de seguridad mediante software enteramente libre, reduciendo la brecha entre grandes organizaciones y entidades con menores recursos. Contribuye también al **ODS 16** (Paz, justicia e instituciones sólidas), al reforzar la resiliencia de las organizaciones frente a amenazas cibernéticas. En cuanto al impacto energético, el uso de un modelo de tamaño reducido (Llama 3.2, con 3.000 millones de parámetros) sobre hardware de bajo consumo (Apple Silicon) reduce la huella ambiental respecto al uso de modelos más grandes servidos en infraestructura en nube.

## 6.4. Trabajo futuro

A partir del estado actual se identifican las siguientes líneas de evolución, ordenadas de menor a mayor complejidad de implementación:

1. **Salida estructurada en la salvaguarda.** La Prueba 4 registró cuatro activaciones del guardrail que correspondieron a falsos positivos: Gemini Flash citó las IPs protegidas precisamente para advertir que no debían bloquearse, y el detector de co-ocurrencia las interceptó sin distinguir el contexto semántico. La mejora consiste en sustituir el análisis de texto libre por salida estructurada: instruir al modelo para que devuelva, junto al informe, un objeto JSON con la decisión de bloqueo explícita (por ejemplo, {"accion": "bloquear", "ip": "192.168.1.5"}) de modo que el *middleware* interprete la intención declarada sin necesidad de analizar patrones en el texto. Esta mejora eliminaría la ambigüedad semántica que originó los falsos positivos sin requerir cambios en la arquitectura general del sistema, y su implementación sería compatible con la estructura de *prompt* actual añadiendo un campo JSON obligatorio al final del informe.
2. **Enriquecimiento con inteligencia de amenazas externa.** Los informes actuales se elaboran exclusivamente a partir del contenido de la alerta y del conocimiento general del modelo. La integración con bases de datos de vulnerabilidades (CVE/NVD) y con fuentes de inteligencia de amenazas en abierto (OSINT) permitiría enriquecer el contexto con información sobre las vulnerabilidades activas en los servicios

identificados, la reputación de las IPs atacantes en fuentes públicas y los *playbooks* de respuesta recomendados para cada técnica MITRE. Esta integración elevaría la precisión de la sección de evaluación del impacto y del plan de respuesta sin modificar el núcleo del *middleware*, añadiendo una etapa de recuperación de contexto antes de la construcción del *prompt*.

3. **Soporte multi-agente y multi-plataforma.** El sistema actual opera sobre los eventos de un único agente Wazuh por análisis. La ampliación del soporte a múltiples agentes simultáneos (con correlación entre sus alertas) permitiría detectar ataques distribuidos que se manifiestan en varios *endpoints* de forma coordinada: movimientos laterales, reconocimiento distribuido o exfiltración en múltiples etapas. Esto exigiría rediseñar el módulo de recuperación de alertas para incluir una dimensión de correlación temporal y espacial entre agentes, y adaptar el *prompt* para que el modelo reciba el conjunto de alertas correlacionadas como contexto único.
4. **Generación aumentada por recuperación (RAG).** La limitación más estructural del sistema es que el modelo no tiene acceso a conocimiento especializado actualizado en tiempo de inferencia. La generación aumentada por recuperación (*Retrieval-Augmented Generation*) abordaría esta carencia: en lugar de depender del conocimiento aprendido durante el entrenamiento, el sistema recuperaría en el momento de la consulta fragmentos relevantes de *playbooks* de respuesta, boletines de vulnerabilidades o documentos de inteligencia de amenazas, y los incluiría en el *prompt* como contexto verificado. Esta vía es especialmente atractiva frente al ajuste fino (*fine-tuning*), que requiere conjuntos de datos de alertas con informes de referencia etiquetados que actualmente no existen en este dominio y que son costosos de producir; con RAG, el conocimiento especializado se actualiza modificando la base documental, no reentrenando el modelo.
5. **Correlación de múltiples alertas.** En la implementación actual, el modelo procesa cada alerta de forma totalmente aislada: no recibe información sobre las alertas anteriores ni posteriores y no puede inferir ataques compuestos que requieran correlacionar eventos dispersos en el tiempo. Un escaneo de puertos (T1046) seguido horas después de una escalada de privilegios (T1548.003) en el mismo *endpoint* es un patrón de ataque en dos fases que el sistema no puede reconocer como tal porque cada alerta produce un informe independiente. Mantener un contexto acumulado entre llamadas al LLM, o introducir una etapa de correlación temporal explícita antes de la inferencia, permitiría al sistema razonar sobre la cadena de ataque completa y situar cada alerta individual dentro del avance global de la intrusión.

6. **Escalabilidad en producción.** Un SIEM en producción puede generar decenas de alertas por minuto durante un incidente activo. Con los tiempos de inferencia medidos en la Prueba 4 (2,6s de media para Groq, 15,0s para Gemini y 39,5s para Ollama), el procesamiento secuencial de un lote de 60 alertas llevaría entre 2,6 y 39,7 minutos según el proveedor, un tiempo inaceptable en la fase aguda de un incidente. Una arquitectura de análisis paralelo (varias llamadas al LLM simultáneas sobre alertas independientes, implementable con `concurrent.futures` sin modificar el núcleo del *middleware*) reduciría ese tiempo de forma casi lineal con el número de hilos concurrentes, haciendo viable la integración en entornos de alta carga. Como primer paso en esa dirección, la Sección 4.10 describe el despliegue del sistema en un clúster Kubernetes local; la incorporación de procesamiento paralelo de alertas es la siguiente mejora natural sobre esa base.
7. **Evaluación con analistas de nivel N1.** Las pruebas de este trabajo evalúan la corrección técnica de los informes generados, pero no su utilidad operativa real. Un experimento controlado con analistas de primera línea permitiría medir la reducción del tiempo de triaje (el tiempo que tarda un analista en pasar de la alerta bruta a una decisión de escalado o descarte) y la tasa de falsos positivos de los informes generados respecto al trabajo manual sin asistencia del *middleware*. Esta evaluación aportaría la evidencia cuantitativa necesaria para fundamentar el despliegue del sistema en un SOC real y para orientar las iteraciones de mejora del *prompt* hacia las dimensiones de calidad más valoradas por los analistas.

La Tabla 6.1 agrupa las siete líneas identificadas con una estimación de complejidad de implementación, entendida como el esfuerzo relativo de desarrollo respecto al estado actual del sistema.

Tabla 6.1: Líneas de trabajo futuro con estimación de complejidad de implementación.

| # | Mejora                                   | Descripción resumida                                                                                                                         | Complejidad |
|---|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 1 | Salida estructurada en la salvaguarda    | Devolver la decisión de bloqueo en JSON explícito para eliminar falsos positivos por análisis de texto libre                                 | Baja        |
| 2 | Procesamiento paralelo de alertas        | Análisis concurrente con <code>concurrent.futures</code> sin modificar el núcleo del <i>middleware</i> ; base sobre el despliegue Kubernetes | Baja        |
| 3 | Enriquecimiento con inteligencia externa | Integrar CVE/NVD y fuentes OSINT como etapa de recuperación de contexto antes de construir el <i>prompt</i>                                  | Media       |
| 4 | Evaluación con analistas de nivel N1     | Experimento controlado para medir la reducción del tiempo de triaje y la tasa de falsos positivos                                            | Media       |
| 5 | Soporte multi-agente y multi-plataforma  | Correlación de alertas de múltiples <i>endpoints</i> para detectar ataques distribuidos en varias etapas                                     | Media-alta  |
| 6 | Correlación temporal de alertas          | Contexto acumulado entre llamadas al LLM para razonar sobre la cadena de ataque completa                                                     | Alta        |
| 7 | Generación aumentada por recuperación    | Base documental de <i>playbooks</i> y boletines de vulnerabilidades para enriquecer el <i>prompt</i> en tiempo de inferencia                 | Alta        |

## 6.5. Valoración personal y aprendizajes adquiridos

La realización de este Trabajo Fin de Grado ha permitido consolidar y aplicar conocimientos adquiridos durante la carrera en áreas como Ingeniería del Software, Sistemas Operativos y Redes de Computadores, y adentrarme de lleno en un campo que me genera especial interés: la ciberseguridad. Las prácticas de empresa realizadas en el SOC de Telefónica Cybersecurity & Cloud Tech fueron el detonante directo de este trabajo; comprobar de primera mano la carga operativa que soportan los analistas de primer nivel fue lo que convirtió la pregunta "¿puede un LLM ayudar aquí?" en un objetivo técnico concreto. Desarrollar el sistema desde ese contexto real ha hecho que cada decisión de diseño tuviera un referente operativo claro.

A nivel técnico, el proyecto me ha permitido trabajar con tecnologías que no había usado antes en profundidad: Wazuh como plataforma SIEM real, OpenSearch y su Query DSL, Ollama y los modelos de pesos abiertos, Docker en entornos de pruebas, y Kubernetes con kind y podman para el despliegue en producción. Integrarlas en un sistema cohesionado ha sido tan instructivo como diseñar los módulos en sí, porque obliga a entender cada pieza no en abstracto sino en relación con las demás. A nivel personal, el aprendizaje más sólido ha sido metodológico: el hábito de contrastar cada resultado con los datos del experimento antes de incluirlo en el análisis, de caracterizar con precisión los fallos del sistema junto a sus logros, y de tomar decisiones de diseño fundamentadas cuando la solución no es inmediata. La constancia para mantener ese rigor a lo largo de todo el proceso y la capacidad de depurar problemas sin solución obvia son las *soft skills* que más he ejercitado durante este trabajo.



# Sobre el uso de la inteligencia artificial generativa

Este Trabajo Fin de Grado tiene como objetivo la integración de modelos de lenguaje en el ámbito de la ciberseguridad. Por tanto, la inteligencia artificial generativa (IAGen) ha sido a la vez el objeto de estudio y una herramienta de apoyo durante el desarrollo. En cumplimiento con las recomendaciones de transparencia de la Universidad de Granada, en este apéndice detallo el uso de estas herramientas, su propósito y el nivel de supervisión aplicado.

## En el desarrollo del código

La arquitectura del sistema, la lógica de los módulos principales (`middleware.py`, `respuesta_activa.py` y `threat_hunting.py`) y las decisiones de diseño del Capítulo 4 son fruto de mi propio trabajo. Durante la programación, he utilizado Claude Code (Anthropic) como asistente para depurar errores concretos, consultar dudas puntuales sobre la sintaxis de Python y plantear los casos de prueba. En todo momento he revisado estas sugerencias, integrando solo aquellas que encajaban con los requisitos del proyecto.

## En el desarrollo de la interfaz web

El desarrollo de la interfaz (`Home.py`, `nav.py` y el directorio `pages/`) requirió un uso más intensivo del asistente, principalmente para agilizar la maquetación de los componentes de Streamlit y aplicar estilos CSS. Sin embargo, la lógica de navegación y la integración del flujo completo (la comunicación con la API de Wazuh, la invocación de los modelos y la respuesta activa) han sido diseñadas e implementadas de forma manual.

## En el despliegue en producción

Para migrar el *middleware* a un clúster de Kubernetes local (Sección 4.10), me apoyé en la IA para estructurar los manifiestos YAML iniciales y entender mejor las limitaciones técnicas de usar podman en modo *rootless*. Todas las configuraciones sugeridas, como el uso del patrón *EndpointSlice*, se contrastaron directamente con la documentación oficial de Kubernetes [16], kind [7] y podman [13] antes de ejecutar el despliegue final en el entorno de pruebas.

## En la redacción de la memoria

En la elaboración de este documento, he empleado la IA de forma puntual para generar primeros borradores de algunas secciones a partir de mis propias notas, así como para revisar la ortografía, aligerar párrafos densos y resolver dudas de formato en LaTeX. He revisado exhaustivamente todos los textos para garantizar que no haya información inventada o no relacionada con el objetivo del trabajo, asegurándome de que la memoria refleja con exactitud la realidad técnica de lo que he programado y experimentado. Toda conclusión, decisión de ingeniería o análisis de resultados proviene exclusivamente de mi propia reflexión.

## Herramientas empleadas

- **Asistente de programación y redacción:** Claude Code (Anthropic), utilizado como apoyo para el código, la interfaz web y los borradores de texto. Todas sus salidas fueron validadas y ajustadas personalmente antes de incorporarlas al trabajo.
- **Modelo principal evaluado:** Ollama con Llama 3.2 (Meta AI), ejecutado en local. Es el sujeto de estudio principal de este TFG y sus respuestas se evaluaron empíricamente en el Capítulo 5.
- **Modelos alternativos evaluados:** Gemini Flash (Google AI) y Llama 3.3-70B a través de la API de Groq, empleados en la comparativa multi-modelo del Capítulo 5 como proveedores externos.

# Bibliografía

- [1] Resolución de 13 de abril de 2026, de la Dirección General de Trabajo, por la que se registra y publica el Acta de la Comisión paritaria del XIX Convenio Colectivo Estatal de Empresas de Consultoría, Tecnologías de la Información y Estudios de Mercado (actualización de tablas salariales). <https://www.boe.es/buscar/doc.php?id=BOE-A-2026-9024>, 2026. BOE núm. 100, de 24 de abril de 2026, pp. 58073–58077. Ref. BOE-A-2026-9024.
- [2] Google LLC. Gemini API documentation. <https://ai.google.dev/>. Consultado en 2026.
- [3] Groq, Inc. Groq API documentation. <https://console.groq.com/docs>. Consultado en 2026.
- [4] Ali Habibzadeh, Farid Feyzi, and Reza Ebrahimi Atani. Large language models for security operations centers: A comprehensive survey. arXiv preprint arXiv:2509.10858, 2025.
- [5] Eric M. Hutchins, Michael J. Cloppert, and Rohan M. Amin. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1):80–106, 2011.
- [6] INCIBE. Las 7 fases de un ciberataque: ¿las conoces? <https://www.incibe.es/empresas/blog/las-7-fases-ciberataque-las-conoces>. Consultado en 2026.
- [7] Kubernetes SIG Testing. kind: Kubernetes in docker. <https://kind.sigs.k8s.io/>. Consultado en 2026.
- [8] Máximo Martín Moreno. TFG-Wazuh-LLM: *Middleware* de código abierto que conecta Wazuh XDR con un LLM para transformar alertas de seguridad en informes tácticos accionables. <https://github.com/maximartinm/TFG-Wazuh-LLM>, 2026. Licencia MIT.
- [9] Meta AI. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

- [10] Farzad Nourmohammadzadeh Motlagh, Mehrdad Hajizadeh, Mehryar Majd, Pejman Najafi, Feng Cheng, and Christoph Meinel. Large language models in cybersecurity: State-of-the-art. arXiv preprint arXiv:2402.00891, 2024.
- [11] Ollama. Ollama: Get up and running with large language models locally. <https://ollama.com/>. Consultado en 2026.
- [12] OpenSearch. Opensearch documentation. <https://opensearch.org/docs/>. Consultado en 2026.
- [13] Red Hat, Inc. Podman: A daemonless container engine for OCI containers. <https://podman.io/>. Consultado en 2026.
- [14] Streamlit, a Snowflake Company. Streamlit documentation. <https://docs.streamlit.io/>. Consultado en 2026.
- [15] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK: Design and philosophy. Technical report, The MITRE Corporation, 2018.
- [16] The Kubernetes Authors. Kubernetes documentation. <https://kubernetes.io/docs/home/>. Consultado en 2026.
- [17] The MITRE Corporation. MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge. <https://attack.mitre.org/>. Consultado en 2026.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [19] Wazuh Inc. Wazuh documentation. <https://documentation.wazuh.com/>. Consultado en 2026.
- [20] Wazuh Inc. wazuh-docker: Wazuh docker deployment. <https://github.com/wazuh/wazuh-docker>, 2024. Rama 4.9.0. Consultado en 2026.
- [21] Bowen Wei, Yuan Shen Tay, Howard Liu, Jinhao Pan, Kun Luo, Ziwei Zhu, and Chris Jordan. CORTEX: Collaborative LLM agents for high-stakes alert triage. arXiv preprint arXiv:2510.00311, 2025.
- [22] Guangze Zhao, Yongzheng Zhang, Changbo Tian, Dan Xie, Hongri Liu, and Bailing Wang. Information-dense reasoning for efficient and auditable security alert triage. arXiv preprint arXiv:2512.08169, 2025.

## Apéndice A

# Prompts utilizados

Este apéndice recoge los dos *prompts* de sistema que gobiernan el comportamiento del *middleware*. Se incluyen en su forma original para que el lector pueda reproducir los experimentos o adaptar el sistema a un entorno diferente.

### Prompt de análisis forense (prompts/mitre\_prompt.txt)

Este *prompt* se externaliza en un fichero de texto independiente del código, de modo que el operador puede modificar el formato del informe sin tocar el módulo `middleware.py`. Los marcadores entre llaves (`{agente}`, `{nivel}`, etc.) son sustituidos en tiempo de ejecución por los valores reales de cada alerta.

Listado A.1: Plantilla completa del prompt de análisis forense (mitre\_prompt.txt). Los marcadores entre llaves se sustituyen en tiempo de ejecución por los campos reales de la alerta de Wazuh.

```
MITRE_PROMPT.TXT -- Plantilla de analisis forense
=====

Actua como un Analista Senior de SOC Nivel 3. Analiza la siguiente
alerta de Wazuh y genera un informe de triaje estructurado, preciso
y accionable.

=== DATOS TECNICOS DE LA ALERTA ===
- Agente afectado      : {agente}
- Nivel de severidad: {nivel}/15 (Regla Wazuh ID: {rule_id})
- Descripcion regla    : {descripcion}
- Tactica MITRE         : {mitre_tactic}
- Tecnica MITRE        : {mitre_id}

=== CONTEXTO DEL INCIDENTE ===
- IP de Origen         : {src_ip}
- Usuario Objetivo     : {usuario}
- Log crudo (Raw Log):
{full_log}

=== RESTRICCIONES CRITICAS (LEE ANTES DE RESPONDER) ===
1. VERACIDAD: Solo cita comandos que existan realmente. Si no estas
seguro, escribe "Verificar con el administrador del sistema" en
lugar de inventar sintaxis.
2. IPs PROTEGIDAS: NUNCA sugieras bloquear 127.0.0.1, ::1 ni la IP
del propio agente. Si el src_ip es 127.0.0.1, indica que el
ataque es local y propone otras medidas.
3. KILL CHAIN: Usa las fases oficiales de Lockheed Martin
(Reconnaissance, Weaponization, Delivery, Exploitation,
Installation, Command & Control, Actions on Objectives).
4. COMANDOS WAZUH: El unico comando valido de Wazuh para bloqueos
es "firewall-drop" via active-response. No menciones "wazuh
reset" ni comandos inventados.

=== ESTRUCTURA DEL INFORME ===

1. [RESUMEN EJECUTIVO Y KILL CHAIN]
- Una frase con lo ocurrido, el agente afectado y el nivel
de urgencia.
- Fase de la Cyber Kill Chain a la que corresponde este evento
(con justificacion breve).

2. [ANALISIS FORENSE DETALLADO]
- Herramientas o comandos detectados en el Raw Log.
- Ataque manual dirigido o script automatizado? Justifica con
evidencias del log.
- Coherencia: concuerda el Raw Log con la descripcion de la
regla Wazuh?

3. [IMPACTO Y OBJETIVOS PROBABLES]
- Si el ataque tiene exito contra '{usuario}', que haria el
atacante a continuacion?
- Posible movimiento lateral, persistencia o exfiltracion.

4. [PLAN DE RESPUESTA ACTIVA (maximo 3 acciones)]
- Acciones tecnicas inmediatas, priorizadas.
- Si propones iptables, incluye la sintaxis exacta y valida.
- Recuerda: no bloques loopback ni IPs internas del agente.

5. [ANOMALIAS DETECTADAS (solo si las hay)]
- Algo inusual que Wazuh pudo haber pasado por alto.
- Falso positivo o falso negativo potencial.

Responde en espanol. Se tecnico y conciso.
```

## Prompt de *threat hunting* (PROMPT\_NL\_A\_DSL)

Esta constante se define dentro del módulo `threat_hunting.py` y no se expone al operador, ya que fuerza al modelo a devolver exclusivamente JSON de OpenSearch. Modificarla accidentalmente rompería el flujo de traducción. Los marcadores `{HOY}`, `{HORA.UTC}`, `{HACE.Xh}` y `{consulta_usuario}` se sustituyen en tiempo de ejecución con marcas de tiempo ISO 8601 calculadas y con la pregunta del analista, respectivamente.

Listado A.2: Prompt del modulo de *threat hunting* (PROMPT\_NL\_A\_DSL), que instruye al LLM para devolver exclusivamente una consulta Query DSL de OpenSearch valida.

Eres un experto en OpenSearch y Wazuh. Tu unica tarea es convertir preguntas en lenguaje natural a queries DSL de OpenSearch.

FECHA Y HORA ACTUAL (UTC): {HOY}T{HORA\_UTC}Z

ESQUEMA DE INDICE WAZUH (campos disponibles):

- timestamp: fecha/hora del evento (formato ISO 8601)
- rule.level: severidad (entero 1-15, siendo 15 el mas critico)
- rule.id: ID de la regla Wazuh (string)
- rule.description: descripcion del evento (campo keyword; usa query\_string con comodines para busqueda de texto)
- rule.mitre.id: codigo de tecnica MITRE ATT&CK  
(ej: "T1110", "T1548.003") -- usa match con el codigo exacto
- rule.mitre.tactic: nombre de tactica en ingles  
(ej: "Credential Access") -- usa match
- agent.name: nombre del agente/endpoint
- agent.id: ID del agente
- data.srcip: IP origen del ataque
- data.dstip: IP destino
- data.srcuser / data.dstuser: usuarios involucrados
- full\_log: log crudo completo del sistema operativo

REGLAS OBLIGATORIAS:

1. Responde SOLO con JSON valido, sin texto adicional, sin explicaciones, sin bloques de codigo markdown.
2. El JSON debe ser una query DSL de OpenSearch con estructura: {"size": N, "query": {...}, "sort": [...]}
3. Para rangos de tiempo usa SIEMPRE timestamps ISO 8601 absolutos con sufijo Z. NUNCA uses expresiones como "now/d" o "now-Xh".
4. Referencia de timestamps ya calculados:
  - "hoy" (desde las 00:00 UTC): {HOY}T00:00:00Z
  - ultimas 2 horas: {HACE\_2H}Z
  - ultimas 6 horas: {HACE\_6H}Z
  - ultimas 12 horas: {HACE\_12H}Z
  - ultimas 24 horas: {HACE\_24H}Z
  - ultima semana: {HACE\_7D}Z
5. Limita los resultados a 10 por defecto.
6. Ordena siempre por timestamp descendente.
7. Para texto en rule.description usa query\_string con comodines: {"query\_string": {"query": "\*termino\*", "fields": ["rule.description"]}}
8. Para tecnica MITRE usa rule.mitre.id con el codigo exacto.

EJEMPLOS:

Pregunta: "intentos de fuerza bruta SSH de las ultimas 2 horas"

Respuesta: {"size":10,"sort":[{"timestamp":{"order":"desc"}}],  
"query":{"bool":{"must": [  
{"query\_string":{"query":"\*ssh\*",  
"fields":["rule.description"]}},  
{"range":{"timestamp":{"gte":"{HACE\_2H}Z"}}},  
{"range":{"rule.level":{"gte":5}}}]}}}

Pregunta: "alertas de la IP 10.0.0.1"

Respuesta: {"size":10,"sort":[{"timestamp":{"order":"desc"}}],  
"query":{"bool":{"must": [  
{"match":{"data.srcip":"10.0.0.1"}}]}}}

Ahora convierte esta pregunta:  
{consulta\_usuario}

